BEGIN TODAY

Start here

Your first two hours

Run a program, predict its output, make a small change and complete the first two lesson quizzes. No game engine or electronics purchase is needed.

First-day plan / Tested starter code / Lessons 01.1–01.2

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

A first session for today

Goal: run a small program, predict its behavior and explain the difference between input, state and output. No game engine, graphics SDK, electronics kit or paid account is needed today. The learner's age, prior programming and mathematics experience were not supplied, so this starting path assumes no programming experience and permits acceleration by demonstrated competence.

Extract the complete course ZIP into a folder you can find again. Open `CURRICULUM.html` in a browser for searchable lessons, or use the PDFs in `volumes`. The browser edition stores no information on a server. Assigned external readings still require internet access; they are linked, not copied into the package.

Time budget	Activity	Evidence to keep
0-15 minutes	Read Lesson 01.1 and trace its five movement commands on paper.	Your predicted positions.
15-40 minutes	Follow the setup instructions below and run `day01_motion.py`.	The command, Python version and output.
40-60 minutes	Explain every line using the walkthrough.	A short explanation in your own words.
60-80 minutes	Change speed to 10, predict the output, then run. Restore it and change the timestep to 0.5.	Predictions, observations and differences.
80-100 minutes	Complete the three questions after 01.1 and the three after 01.2 before reading the answer key.	Six answers, marked afterward.
100-120 minutes	Write a journal entry and optionally play the grid game.	One question to investigate next session.

These are session-planning estimates, not a deadline. Setup may take longer. Take breaks, stop at a working checkpoint and resume without treating slower progress as failure. Completing the reading and practical check matters more than matching the clock.

Open a shell and find the starter folder

A **terminal** is the window; a **shell** is the command interpreter inside it. PowerShell is a common Windows shell. Terminal is a common macOS terminal application. Linux has several terminal applications. Python is a different program launched from that shell.

Open the extracted `starter` folder in a terminal. Some file managers and editors offer an “Open in Terminal” option. Otherwise type `cd` followed by the quoted folder path and press Enter. The path must be the actual path on the learner's computer; do not copy a fictional username from an example.

On macOS/Linux, `pwd` shows the current folder and `ls` lists its files. In PowerShell, `Get-Location` and `Get-ChildItem` do the same. Confirm that `day01_motion.py` is listed before

continuing.

On **macOS or Linux**, first check the interpreter, then run the file:

```
python3 --version
python3 day01_motion.py
```

On **Windows**, a common Python launcher command is:

```
py -3 --version
py -3 day01_motion.py
```

When `py` is unavailable but `python --version` reports a supported Python 3 installation, use `python day01_motion.py`. Use one verified interpreter command consistently. Do not use Python 2. The supplied code was tested with Python 3.13.5; that is the tested environment, not a claim that this particular release is always the newest or the only usable release.

When Python is missing, use the official Python downloads and setup documentation, then reopen the terminal and repeat the version check. Official entry points: Python downloads and Python setup and usage. On a managed computer, use the approved installation process rather than changing security settings. You can complete the paper trace while installation is arranged. [R04]

The first program, explained

Open `starter/day01_motion.py` in a plain-text code editor. Save changes to that file, not to a word-processing document or a file ending in `.py.txt`.

```
position_units = 0.0
speed_units_per_second = 6.0
timestep_seconds = 0.25

for tick in range(1, 6):
    position_units = position_units + speed_units_per_second * timestep_seconds
    print(f"tick {tick}: x={position_units:.2f} units")
```

`position_units = 0.0` creates the starting position. An assignment evaluates the right side and binds the result to the name on the left. It does not make an algebraic equation that must remain true forever.

`speed_units_per_second = 6.0` says the player travels six units during one second. `timestep_seconds = 0.25` makes each update represent one quarter of a second. The names carry units so the calculation can be checked.

`range(1, 6)` supplies 1, 2, 3, 4 and 5. The upper endpoint is excluded. The indented lines execute once for each value. The movement line adds `6.0 * 0.25`, or 1.5 units, to the previous position. The print line displays the current state; it does not move the player. The formatting `:.2f` displays two digits after the decimal point. It changes the text presentation, not the underlying movement rule. [R03, R04]

Expected output:

```
tick 1: x=1.50 units
tick 2: x=3.00 units
```

```
tick 3: x=4.50 units
tick 4: x=6.00 units
tick 5: x=7.50 units
```

Before each change, write the five expected positions. With speed 10 and the original timestep, each update adds 2.5 units and the last position is 12.5. With the original speed 6 and timestep 0.5, each update adds 3 units and the last position is 15. Change one variable at a time so a disagreement has a clear cause.

Common first-day problems

Symptom	What to check
Command not found, or Python is not recognized	Confirm Python is installed through the approved route; reopen the shell; check the available launcher command.
Cannot open file	List the current directory. Confirm the exact filename and that the terminal is inside `starter`.
A >>> prompt is visible	That is the interactive Python prompt, not the ordinary shell. Enter `exit()` to return, then run the file command in the shell.
Indentation error	Preserve consistent indentation beneath the `for` line. Do not mix arbitrary tabs and spaces.
Syntax error	Read the indicated line and the preceding one. Check quotes, parentheses and the colon after `for`.
Program runs but changes do not appear	Save the edited file and confirm that the command runs that same file, not another copy.

Never respond to a setup error by running an unexplained administrator command, deleting system folders, disabling protections or sharing passwords. The supplied starter programs need none of those actions.

A playable reward and later test commands

After the motion exercise, run the tiny terminal game:

```
python3 grid_game.py
```

Use `py -3` instead of `python3` on Windows when that is the verified launcher. Move with W/A/S/D to reach G; Q exits. A wall or boundary blocks movement but still consumes a turn. The supplied game uses classes and dataclasses that are taught later; playing it today does not require understanding every implementation detail.

The following command replays a complete route:

```
python3 grid_game.py --replay SSSSDDDDDD
```

The demonstration reaches position (6,4) on turn 10. Find a different route after learning the rules. In Module 04, run the supplied automated tests and write additional tests of your own:

```
python3 -m unittest discover -v
```

The package's 23 Python tests and C++ motion-contract test passed in the recorded Linux environment. They validate the supplied starter, not the learner's future work and not every operating system. See `starter/TESTED_ENVIRONMENT.md` for the exact scope.

A two-week starting rhythm

This is a sample at roughly 10-15 study hours per week, not a promise that every reading or lab will fit. Repeat a session when the practical check is incomplete. Do not skip a concept merely to keep the dates aligned.

Session	Focus
Day 1	Lessons 01.1-01.2, paper trace, motion program and six quiz questions.
Day 2	Lesson 01.3: predict assignments before execution; repair a failed swap.
Day 3	Lesson 01.4: units, frame intervals and an engineering journal.
Day 4	Lesson 02.1: values, types and text-to-number conversion.
Day 5	Review Module 01; explain the motion program without looking at notes.
Days 6-7	Optional gentle practice, catch-up or rest. No new topic quota.
Day 8	Lesson 02.2: conditions and boundary tests.
Day 9	Lesson 02.3: loops, progress and invariants.
Day 10	Lesson 02.4: functions, return values and input contracts.
Day 11	Repair missed questions and add your own function test cases.
Day 12	Start Module 02's small resource-gathering simulation.
Day 13	Review the module project; demonstrate one unfamiliar change.
Day 14	Catch-up, planning and rest before Module 03.

Begin each later session with a five-minute recall exercise. End with the exact command that runs the current project, one unresolved question and the next smallest testable step. Use `templates/LAB_JOURNAL.md` rather than relying on memory.

How to know a lesson is learned

Answer the three questions **before** opening the answer key. Award 0 for incorrect/missing, 1 for partly correct or unsupported and 2 for correct with a clear explanation. A suggested formative threshold is at least **5/6**, plus the lesson's practical pass condition and completed assigned study. A quiz score alone does not establish engineering competence.

For a missed question, identify the mistaken idea, solve a changed example and retest after a delay. Do not merely copy the answer. A reviewer should occasionally ask for an unfamiliar variation or a live code change. AI can explain or critique, but the learner must predict, test and defend the result independently.

Next: complete the two full lessons included on the following pages, then continue with Volume 1. `PROGRAM_GUIDE.pdf` contains placement, prerequisites, hour estimates, all ten checkpoint projects, graduate expectations and the resource directory. `ANSWER_KEY.pdf` is deliberately separate from the printable lessons.

LESSON 01.1 • MODULE 01

What a program actually does

Learning goal Explain state, instructions, input and output using a tiny game.

A program is a set of instructions that transforms information. Its state is the information that must persist: a player position, a health value, or a list of enemies. An input is a new observation or command. An output might be a picture, sound, file or network message. A game repeatedly reads inputs, changes state and produces outputs. The picture is a representation of the state, not the state itself.

A processor executes machine instructions; an operating system manages resources and access to devices. A game engine is software that organizes recurring tasks such as rendering and asset loading. A graphics processor is hardware that executes suitable workloads. Keep these layers distinct: changing a game rule is not the same activity as designing a processor.

WORKED EXAMPLE

For a player at x = 2, the command move-right by 3 units produces x = 5. Drawing the character twice must not move it twice. Rendering observes state; the simulation changes it.

PRACTICE + PASS CRITERION

On paper, trace five commands: right 3, left 1, right 0, left 4, right 2, starting at x = 2. Draw boxes for input, simulation state and output. Save the trace. Pass: every state transition is explainable without running code.

ASSIGNED STUDY

R03: introductory functions/variables lecture; R15: Basics of Information.

CHECK YOUR UNDERSTANDING

1. What is the difference between state and a displayed image?

2. What final position results from the five lab commands?

3. Why should drawing a character not update its position?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.2 • MODULE 01

Files, terminals and reproducible execution

Learning goal Run a saved Python program and distinguish a shell from an interpreter.

A source file stores the instructions you write. A terminal presents a command-line interface; its shell starts programs and manages paths. The Python interpreter is a program that runs Python instructions. A shell prompt is not a Python prompt. Commands such as cd belong in the shell; expressions such as print(2 + 3) belong in Python code.

A working directory tells a process where relative paths begin. Many apparent programming errors are actually wrong-directory or wrong-file errors. Reproducibility starts by recording the exact command, file and tool version. Use a dedicated course folder. Do not run copied administrative commands or upload private files merely to make a tutorial work.

WORKED EXAMPLE

With day01_motion.py in the current folder, python3 day01_motion.py runs that file on many macOS/Linux setups. On Windows, py -3 day01_motion.py is a common launcher form. First confirm the available command with its --version option.

PRACTICE + PASS CRITERION

Follow START_HERE. Run the included program twice from the same folder; save the command and output in your journal. Intentionally misspell the filename, read the error, then repair it. Pass: you can explain which program produced the error.

ASSIGNED STUDY

R04: Using the Python Interpreter; installation links in START_HERE.

CHECK YOUR UNDERSTANDING

1. What does a relative filename depend on?

2. Is cd a Python statement in an ordinary script?

3. What three facts help another person reproduce a run?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

PROGRAM GUIDE

The path from first code to research

Computer science + computer engineering for games

Read this guide to choose a starting point, follow prerequisites, plan study hours and assess real engineering work.

200 lessons / 600 quiz questions / 10 volumes

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

What this program is

This is a guided self-study curriculum for software engineers who want to understand and build game engines, work close to gaming hardware, or progress toward hardware architecture and digital design. It contains 50 modules, 200 lessons and 600 lesson-quiz questions across ten volumes. Every lesson supplies an outcome, an original teaching note, a worked example, a practical assignment with a pass condition, assigned reading and three questions. Explanatory answers are in a separate key and expandable in the browser edition.

A lesson is a **multi-session study unit**, not a 20-minute substitute for a university subject. The teaching notes introduce the concepts; the assigned chapters, lectures, problem sets, implementations and reviews provide the depth. Reading this package without doing that work will not produce undergraduate or graduate competence. External textbooks and courses are linked rather than redistributed, so the package is not an entirely offline replacement for those materials.

The breadth was cross-checked against ACM/IEEE-CS/AAAI CS2023 and the ACM/IEEE computer-engineering curriculum report CE2016. This is not an accredited degree, a university-approved equivalency, professional licensure, security certification or authorization to sign off production silicon. An institution or employer may assess the work independently under its own rules. [R01, R02]

The default route assumes no programming experience and basic reading and arithmetic. The learner's age, equipment, mathematics background and weekly availability were not supplied. No age-dependent assumptions are made. A beginner can start with the supplied first-day program; a more experienced learner can use the placement rules below. Electronics begins in simulation, with physical extensions only after a qualified safety review.

Learning outcomes and professional direction

By the undergraduate software checkpoint, the learner should be able to design, implement, test, profile and distribute a small original game and engine; explain the relevant mathematics; and reason about memory, algorithms, concurrency, networks and resource ownership. By the undergraduate hardware checkpoint, the learner should be able to specify and verify a digital subsystem, connect it to software, interpret synthesis/timing evidence and distinguish modeled claims from physical measurements.

Graduate work adds derivation, formal or numerical reasoning, strong baselines, research reproduction, controlled experiments, uncertainty analysis and a defended thesis-style contribution. The full route covers both engine and hardware breadth; the thesis should pursue **one primary specialization**, not attempt a whole commercial engine and a new console simultaneously.

Direction	Areas to emphasize after the shared foundation	Strong portfolio evidence
Engine / systems engineer	Volumes 3-6; Modules 37-39, 42, 44-45	Resource-safe engine, task graph, profiler, reproducible release.
Rendering / GPU engineer	Mathematics; Modules 21-23, 37-38, 41-43, 45	Renderer, derived sampling method, correct GPU primitive and equal-quality comparison.
Platform / performance engineer	Volumes 4, 7-8; Modules 42, 46, 48	Latency trace, memory investigation, driver model and reliable cross-platform boundary.
CPU / GPU / accelerator designer	Volumes 2, 4, 7-8; Modules 41-42, 46-48	Verified RTL, architectural model, timing evidence and complete host interface.
Tools / compiler engineer	Modules 11-15, 20, 25, 41, 45, 50	Asset pipeline, editor commands, typed DSL and semantics-preserving optimization.

These are emphasis choices, not permission to bypass a listed prerequisite. Engine-focused learners do not need to wait until the thesis to start building relevant portfolio work. Hiring standards vary; no completion point here guarantees a job.

Program map and planning hours

All hour figures are **author planning estimates**, not measured completion times or transferable academic credits. They include assigned study, problem solving, labs, review and the corresponding checkpoint project. Do not add all project hours again on top of the table. Prior preparation, mentorship, debugging and project scope can move actual time substantially. Volume 10 includes the 400-800 hour thesis planning range.

Volume	Modules	Main level and focus	Estimated hours
1	01-05	Beginner programming, debugging, testing and first game	220-340
2	06-10	Discrete math, linear algebra, calculus, probability and signals	500-800

Volume	Modules	Main level and focus	Estimated hours
3	11-15	C++, ownership, algorithms, data structures and languages	400-650
4	16-20	Architecture, OS, concurrency, networking and engineering	380-600
5	21-25	CPU/GPU graphics, materials, engine architecture and tools	420-650
6	26-30	Physics, animation, audio, AI, multiplayer and release	420-650
7	31-35	Circuits, logic, RTL, processors and FPGA subsystems	450-720
8	36-40	Boards, GPU architecture, performance and co-design	350-560
9	41-45	Graduate numerics, GPU algorithms, rendering and compilers	550-850
10	46-50	Graduate hardware, verification, research thesis and defense	810-1,380
Full dual-breadth route	01-50	All 200 lessons and Gates A-J	4,500-7,200

At 30 focused hours per week for 46 weeks per year, the full estimated range is about **3.3-5.2 years**. At 15 hours per week for 46 weeks, it is about **6.5-10.4 years**. These are arithmetic illustrations, not a promised schedule. A specialist route, substantial prior knowledge or separately completed university subjects can change the workload, but skipping evidence requirements does not create equivalent mastery.

Treat this as four broad stages: foundations (Volumes 1-4), undergraduate engine practice (5-6), undergraduate computer engineering (7-8), then graduate specialization and thesis (9-10). A conventional institution may distribute the subjects differently. The sequence here optimizes for one learner building an integrated portfolio, not for a particular university's catalog.

Prerequisites and sequencing

The default sequence is numerical: 01.1 through 50.4. The digits before the decimal identify the module; the final digit identifies its lesson. Each module's opening page states prerequisites. Complete the practical checkpoint at the end of each volume before using its results as a foundation for the next stage.

Mathematics can run in parallel with early programming after Module 02, but the programming prerequisites of individual math labs must still be met. Do not begin advanced graphics without vectors, transforms and the relevant calculus/probability. Do not begin concurrency without memory and lifetime reasoning. Do not begin physical board work before the circuits and safety review. Graduate modules require the undergraduate evidence, not only recognition of terminology.

When a prerequisite is weak, return to its lesson and solve a new problem. Use a prerequisite repair log: missing skill, smallest review source, transfer exercise, result and date. A tutor or peer reviewer can help identify the actual gap. Starting at the beginning is not required for someone who can demonstrate the listed work independently.

Entry diagnostic: locate the starting point

Complete these without searching or using AI. This is placement information, not a pass/fail judgment. The answers are in the answer-key appendix under “Entry diagnostic,” separate from the 600 lesson questions.

1. Evaluate 3 + 4 x 2 using ordinary arithmetic precedence.
2. Convert 0.25 seconds to milliseconds.
3. Solve 3x + 2 = 14.
4. Evaluate 2^8.
5. Add vectors (-2,5) and (3,-1).
6. Find the length of vector (3,4).
7. Start at position 4, move right 3, then left 1. What is the final position?
8. For integer values: x=2; y=x+3; x=9. What is y?
9. What values does Python range(1,4) supply?
10. In the list [10,20,30], what is the value at zero-based index 1?
11. A while loop repeats while remaining>0 but never changes remaining or exits. What can go wrong?
12. Evaluate True and False.
13. A door opens at level 5 or above. Give three particularly useful boundary-test levels.
14. A program cannot find a relative filename. What environment fact should be checked first?
15. A program is fast on one input but produces a wrong result on another supported input. Is that an acceptable optimization?

With fewer than five correct answers on questions 1-6, schedule an algebra/trigonometry bridge using OpenStax Precalculus 2e while beginning Volume 1. Start with arithmetic, equations, functions, exponents, coordinate geometry and trigonometry; then retest. Questions 5-6 can also be learned in Module 07. This result must not block the first programming session. [R40]

With fewer than five correct answers on questions 7-12, begin at Lesson 01.1. Strong results can justify moving faster, but not skipping the labs automatically. Questions 13-15 probe engineering habits that should develop throughout the course.

Prior-learning challenge process

A self-reported skill list is not enough to skip a volume. Attempt its lesson quizzes without notes, complete the checkpoint artifact or a comparable existing artifact, and explain a new variation to a reviewer. Require the same practical standards as the ordinary route. Record what was assessed and what was not.

For the math volume, the challenge must include a proof, a transform derivation, calculus/ODE reasoning and a probability or statistical interpretation; a multiple-choice vocabulary test is insufficient. For systems, include a lifetime bug, a concurrency argument, a network failure and a measured performance explanation. For digital design, include RTL semantics, interface verification, setup/hold reasoning and an explicit reset policy. Graduate credit within this self-study plan requires research-level evidence, not only professional seniority.

The study loop and assessment rules

For each lesson: retrieve prerequisite ideas from memory, read the teaching note, predict the worked example, complete the assigned source study, perform the lab, answer the quiz, then check the key. Save evidence rather than only checking a completion box. Required source readings are identified by R-codes linked in the resource directory. A chapter's explanation does not replace its exercises.

A suggested study block is 60-90 minutes with a break: roughly 10 minutes recall, 20-30 minutes reading/derivation, 25-40 minutes implementation or problems, then a brief journal entry. This is a planning suggestion, not a medical or cognitive prescription. Use a pace and breaks appropriate to the learner.

Each short quiz has three questions scored 0-2 each. Use 0 for absent/incorrect, 1 for partial or unexplained and 2 for correct with reasoning. A formative pass is **5/6 or better**, completed assigned study and the practical pass condition. The questions are deliberately short; they are not psychometrically validated exams or sufficient evidence of degree-level mastery by themselves.

For each missed concept, write the misconception and solve a changed example. Recheck after 24-72 hours when practical. At the end of a module, demonstrate a new input, requirement or failure without following the original walkthrough. An evaluator may accept a different technically sound answer than the key when the assumptions and reasoning are explicit.

Required depth beyond the short quizzes

At minimum, each programming/systems module should produce the stated project, **6-10 additional exercises**, and **five new edge or failure cases** beyond copied examples. Every mathematics module should include **12-20 written problems**, at least two derivations or proofs where appropriate, and a computational check. Select problems covering each named reading topic; use official solutions only after an independent attempt.

For graphics and simulation modules, preserve reference images or trajectories, numerical tests, a performance/quality comparison and one failure case. For digital-design modules, include a written interface, directed boundary tests, reproducible randomized tests, assertions, and tool reports appropriate to that module. For each graduate technical module, add **6-10 substantial analytical problems**, a critical reading of at least one original paper or equivalent graduate chapter, and a reproducible experiment with a baseline.

These are minimum planning requirements, not a reason to stop when understanding is incomplete. A mentor may substitute a comparably demanding assignment. Problems should come from the referenced course or book where possible and be logged by source, section and exercise number; this avoids pretending a one-page introduction replaces a semester's mathematical practice.

The mathematics extensions named in Volume 2 mean: R40 for algebra/trigonometry, R41 for multivariable derivatives and constrained differentials, and R42 for the later graduate optimization treatment. Complete the relevant R41 sections before gradient-based graduate work. Use R54 for deeper numerical methods. [R40-R42, R54]

For hardware/firmware direction, add a mandatory C-language bridge after Module 15: compile a small C source file as C, expose a C-compatible interface to C++, document allocation/free ownership, inspect structure layout and test error returns. C and C++ are distinct languages; do not treat compiling C code as C++ as proof of compatibility. Review the chosen compiler's C documentation and the target RTOS examples before Module 36. [R11, R53]

Review rubrics and academic integrity

Use this common project rubric, adapted to the project: correctness and requirements 30 points; tests/verification 25; reasoning and design tradeoffs 15; quantitative quality/performance/error analysis 10; reproducibility 10; communication and provenance 10. Suggested pass: **80/100**, with all essential practical checks complete and no unresolved critical defect inside the claimed operating envelope. A high visual-quality score cannot compensate for an unsafe memory interface or a false correctness claim.

For graduate artifacts: contribution/question clarity 10; mathematical or technical rigor 20; implementation correctness 20; evaluation/baselines/uncertainty 25; reproducibility 15; writing and defense 10. Suggested pass: 80/100 plus independent reproduction of the central claim. Publication acceptance and positive results are not required. Clear scope, honest failure analysis and defensible evidence are required.

Use AI as a tutor, reviewer or brainstorming aid, not an unexamined answer source. Predict before asking for code, identify generated sections, check provenance, test outputs and explain

the implementation. Never upload private employer code, proprietary SDK material, credentials or other people's personal data without authorization. Supplied starter code is a learning reference; running it does not complete the learner's own project requirements.

Gate A: a complete first game

After Volume 1. Build a terminal grid or resource game with a clear initial state, commands, valid/invalid input behavior, a success/failure condition and restart. Keep the simulation separate from display. Use the included game as a reference, then implement meaningful changes rather than submitting it unchanged.

Deliver the source, README, a rule specification, at least 12 learner-written tests, a saved input sequence and its expected final state, and a short journal explaining two bugs. Include tests for boundaries, blocked movement, invalid commands, restart and repeated rendering without state changes. Replaying the same inputs in the pinned environment must produce the expected result. A second person should run a complete loop from the README.

The oral check asks the learner to change one rule, predict affected tests and repair the implementation. Passing does not require graphics. Do not add networking, a database or a large engine merely to make the project look advanced.

Gate B: a game-mathematics notebook

After Volume 2. Produce a versioned notebook or scripts plus written derivations. Required artifacts are a logic/invariant proof, vector and matrix operations, a transform-chain example, a one-dimensional dynamics model, a sampling/aliasing demonstration and a small statistically analyzed experiment.

Tests must include a known transform, a zero-vector edge case, an analytic motion comparison, a timestep refinement study and a probability calculation checked by simulation. Show units and numerical tolerances. Do not normalize a zero vector silently or treat a random experiment as proof. Complete and log the required source problem sets.

The oral check supplies a changed coordinate convention, a new timestep and a misleading average. The learner must explain what changes and what remains invariant. A screenshot of a plot without derivation or reproducible data is insufficient.

Gate C: native simulation and a tiny language

After Volume 3. Implement the core simulation in C++ with explicit ownership and a build system, then add a small expression language or bytecode interpreter. It may evaluate movement or material expressions; it must have a specified grammar, type/error behavior and resource limits.

Deliver a clean build, tests, memory/lifetime checks available on the toolchain, a complexity analysis and a comparison of two appropriate data structures. Include empty input, malformed tokens, invalid bytecode or expressions, lifetime boundaries, container invalidation and numerical limits. Add the C-language bridge for the hardware direction.

The oral check includes one pointer/lifetime defect, a data-structure choice and a new grammar case. The learner should explain the entire compile/link/run path. A library may supply support utilities, but not replace the central learning implementation without an explicit comparison exercise.

Gate D: a defensible systems application

After Volume 4. Build a local job-based application that reads validated work, performs bounded processing and stores results. A small local client/server or task-processing tool is enough; public deployment is unnecessary.

Deliver a requirements sheet, architectural decision record, thread/resource ownership map, protocol/schema, threat model, automated build/test script, dependency inventory and recovery report. Test malformed messages, duplicate requests, cancellation, partial writes, worker failure and shutdown under load. Run relevant memory/concurrency diagnostics where supported and record their limitations.

The oral check asks why the design is race-free under its stated model, how retries avoid duplicated durable effects and what evidence identifies the measured bottleneck. A functioning happy path alone does not pass.

Gate E: a small 3D engine

After Volume 5. Build an engine demonstration containing a CPU reference renderer, one supported GPU backend where available, a fixed-step simulation, explicit entity/resource ownership, an asset cooker, an inspector and versioned save/load. A software-only execution path is acceptable while GPU access is unavailable, but untested GPU claims must remain excluded.

Deliver a fixed scene, source assets with provenance, build/cook commands, validation logs, headless tests, screenshot or image-difference evidence, frame-time data and a peak-memory report. The budget is chosen for the declared test machine; 60 fps is an example target, not a universal pass requirement. Test near-plane crossing, resize/minimize, stale handles, malformed assets and resource shutdown.

A second person must build from a clean checkout or extracted source tree. The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. Using an existing engine for comparison is encouraged; using it in place of the central engine implementation changes the learning objective and needs a separate implementation project.

Gate F: undergraduate software capstone

After Volume 6. Release a small original game built on the engine. Include one finished gameplay loop; collision/physics; animation or articulated motion; bounded audio; an inspectable AI behavior; settings; save/load; and a tested package. The multiplayer requirement can be a separate local two-client demonstration with delay, loss and replay tests rather than a public online service.

Evidence includes a compatibility matrix, remappable controls, readable interface tests, alternative presentation for a critical audio cue, pseudo-localization, a clean-folder launch and a technical postmortem. Preserve CPU/GPU or software-renderer timing boundaries, raw frame distributions and memory measurements. Test interrupted saves, invalid mods if modding is exposed, controller/device absence and exit during loading.

A reviewer must finish the loop, reproduce a bug test and observe a live change. The quality bar is a coherent, testable small system. It is not commercial art quality, a large open world or a revenue target.

Gate G: undergraduate digital-design capstone

After Volume 7. Implement one RTL subsystem: a synchronous queue with a small processor interface, a sprite compositor, a fixed-point vector block or another similarly bounded game-related function. The module-level processor exercise remains required even when the capstone centers on a different block.

Deliver the interface specification, software reference, RTL, automated testbench, directed and randomized cases, assertions, reset review and synthesis reports. Test empty/full, simultaneous operations, backpressure, overflow/rounding and reset while work is pending where applicable. Label simulation, synthesis and physical measurements separately.

No board purchase is required. A board demonstration is optional and follows the safety plan. Passing FPGA operation does not establish ASIC readiness. The oral check asks for a waveform explanation, a nonblocking-assignment trace and setup versus hold reasoning.

Gate H: undergraduate hardware/software co-design capstone

After Volume 8. Choose one measured engine workload. Compare a competent software baseline with at least two architecture alternatives, implementing one alternative as RTL, a supported simulation model or a bounded analytical prototype with clear limitations.

Required evidence includes workload distributions, numerical-quality limits, host/device ownership, setup and transfer costs, resource/timing estimates, failure handling and a sensitivity analysis. Evaluate small and large workloads and one adverse case. A negative conclusion is acceptable: proving that a software optimization is preferable is valid co-design work.

The review must identify the maximum application benefit, the dominant bottleneck, which results are modeled and which are measured, and what changes under a tighter memory or latency constraint. Do not compare an optimized accelerator against an intentionally weak software baseline.

Gate I: graduate reproduction and extension

After Volume 9. Select a bounded published or graduate-level method in rendering, simulation, GPU algorithms or compilation. Reconstruct the method, reproduce one central result and add a clearly identified extension, new setting or rigorous failure analysis. The extension need not be globally novel; its relation to prior work must be honest.

Deliver a claim-evidence map, derivation or semantic argument, implementation, independent reference or baseline, raw data, analysis scripts, one ablation and one parameter sensitivity study. Use equal-quality comparisons when quality changes with speed. Report uncertainty at the appropriate trial unit and avoid training/tuning on the final evaluation set without disclosure.

A reviewer should reproduce the main table or result. Unresolved discrepancies must be explained and narrow the claim. This checkpoint is the prerequisite for the thesis, not a replacement for it.

Gate J: thesis, defense and handoff

After Volume 10. Complete one primary thesis-scale project. Plan roughly 400-800 focused hours within the volume's estimate, but adjust the scope through review rather than treating the hours as a requirement to spend. A 30-60 page thesis-style report plus technical appendices is a useful planning target; evidence quality matters more than length.

Deliver a literature map, reviewed proposal, running baseline, risk register, versioned decisions, final method, tests, evaluation, reproducible artifact and maintainer handoff. Hold a 30-45 minute defense with a qualified reviewer or peer panel, including a derivation, a failed case and a live implementation question. Independently reproduce the central result and complete revisions.

One thesis is enough. The learner need not invent both a new engine and a new GPU. Completion records should say self-study curriculum and describe the actual artifacts, not claim an accredited bachelor's or master's degree.

Thesis project menu: choose one primary question

Area	Example question	Essential evidence
Rendering	Can a sampling/reconstruction change reduce equal-error time on a defined moving-scene set?	Estimator derivation, reference images, motion failures, total cost and uncertainty.
Engine scheduling	Can a bounded scheduling policy reduce p99 frame time during streaming bursts without exceeding memory limits?	Task traces, competent baselines, burst workloads, memory peaks and ablations.

Area	Example question	Essential evidence
Physics	How do timestep, iteration count and compliance affect error and cost in a small deformable simulation?	Equations, gradient/constraint checks, reference solution and sensitivity study.
GPU algorithms	Does a compaction or wavefront strategy improve a specified irregular workload after queue overhead?	Work/span reasoning, termination and bounds tests, skewed workloads and end-to-end timings.
Accelerator design	Does a verified fixed-point block meet a game-quality tolerance and improve end-to-end latency or energy?	Numerical error, RTL tests/properties, host interface, PPA assumptions and transfer cost.
Architecture / compilers	Does a cache, layout or specialization policy improve a stated game-workload family?	Model validation, semantic correctness, held-out workloads and Pareto tradeoffs.

These are question templates, not claims that the techniques are new or guaranteed to succeed. Review relevant newer research when the project begins. The 2016 XPBD and 2020 ReSTIR references are stable historical anchors for reproduction, not a survey of the latest work. [R44, R55]

Hardware, software and safety policy

Start with the computer already available. The first-day Python work and early programming do not require a dedicated GPU, FPGA, oscilloscope, cloud subscription or game-engine license. Use a code editor, a supported Python 3 interpreter and later Git. At Module 11, add a C++20-capable compiler and CMake. Record the actual versions rather than upgrading tools in the middle of a reproducibility experiment. [R04, R05, R11, R12]

At graphics, choose one supported path: Vulkan, native Direct3D 12 on a compatible Windows setup, or Metal on a compatible Apple setup. Check actual device and tutorial capabilities before installation. A moving “latest” tutorial may assume newer features than the learner's device supports. SDL3 examples must use SDL3 APIs rather than mixing SDL2 instructions. Keep the CPU renderer as a reference and fallback. [R21, R22, R24, R25]

CUDA is an optional vendor-specific route, not a requirement for learning GPU algorithms. Use compute shaders or a supported alternative where appropriate, and explicitly limit claims when a GPU is unavailable. Use official platform documentation rather than leaked console SDKs. No supplied lesson grants proprietary access or distribution rights. [R21, R24, R25, R37]

For digital design, begin with supported RTL simulation and synthesis tools such as Verilator, cocotb, Yosys and SymbiYosys; compatibility depends on versions and language features. Board selection should follow the project and toolchain, not precede them. For architecture and

physical design, use gem5 and an educational OpenROAD-compatible flow only within their documented model and library scope. [R32-R34, R38, R39, R50]

Physical experiments require a qualified mentor, exact board/peripheral documentation, compatible voltage levels and a low-voltage safety plan. Keep protective mechanisms enabled. Do not open or modify mains power supplies, CRTs, high-energy battery packs or high-voltage equipment. Do not design a first project around mains circuitry. Do not assume a low nominal voltage removes short-circuit or thermal risks. Use reviewed commercial educational boards and approved supplies. Hardware procurement and fabrication are optional extensions, not first-day requirements.

Source use, maintenance and limitations

R-codes identify the authoritative reading directory that follows. The lesson teaching notes and quizzes are original instructional synthesis; cited resources provide primary course material, official specifications, author textbooks or original research. Resource descriptions are not claims that every linked page is a complete beginner course. For example, the Python tutorial is a reference for people with some programming familiarity; CS50 Python is the recommended beginner lecture companion. [R03, R04]

Use the source sections named in each lesson, and log the actual chapter and problem numbers studied. Some course sites are historical archives, intentionally chosen for durable foundations. Software documentation is often a moving target. Record a release or access date and use matching documentation. Where an exact page moves, navigate from the organization's official documentation rather than silently replacing it with an unreviewed tutorial.

The source directory was assembled for this edition dated September 28, 2026. It is not a guarantee that every future link, tool feature or licensing term remains unchanged. Original books, papers, software and courses retain their own licenses. They are linked, not included as copied books or bundled proprietary software. All hardware and performance examples must be revalidated for the learner's chosen system.

This curriculum emphasizes essential technical depth for game engines and gaming hardware. It does not replace a university's full general-education requirements, supervised laboratories, formal academic assessment or the specialized mentorship required for production ASIC signoff. Its strength is a coherent sequence of explicit learning outcomes and evidence-producing work, not a credential claim.

Reference directory

The readings below provide depth beyond these lesson primers. Links lead to the authors, universities, standards organizations or tool maintainers. Record an exact release, edition and access date in each project. “Latest” documentation can change; archived university courses remain useful for foundations. Links require internet access.

R01 ACM / IEEE-CS / AAAI

CS2023 knowledge areas

Breadth checklist, not a claim of accreditation.

https://csed.acm.org/knowledge-areas/

R02 ACM / IEEE Computer Society

Computer Engineering Curricula 2016

Computer-engineering breadth and laboratory expectations; historical curriculum report.

https://ieeecs-media.computer.org/assets/pdf/ce2016-final-report.pdf

R03 Harvard University

CS50 Introduction to Programming with Python

Beginner lectures and practice; complete exercises rather than only watching.

https://cs50.harvard.edu/python/

R04 Python Software Foundation

Python tutorial and language documentation

Reference companion; the tutorial itself assumes some programming familiarity.

https://docs.python.org/3/tutorial/

R05 Scott Chacon and Ben Straub

Pro Git

Version control and collaboration.

https://git-scm.com/book/en/v2

R06 MIT OpenCourseWare

6.042J Mathematics for Computer Science

Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R07 MIT OpenCourseWare

18.06 Linear Algebra

Matrices, vector spaces, eigenvalues and problem sets.

https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

R08 MIT OpenCourseWare

18.01SC Single Variable Calculus

Calculus lectures, exercises and assessments.

https://ocw.mit.edu/courses/18-01sc-single-variable-calculus-fall-2010/

R09 MIT OpenCourseWare

18.05 Introduction to Probability and Statistics

Probability, inference and quantitative reasoning.

https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

R10 MIT OpenCourseWare

6.003 Signals and Systems

Sampling, convolution, transforms and systems.

https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/

R11 C++ Core Guidelines editors

C++ Core Guidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 Kitware

CMake tutorial

Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 MIT OpenCourseWare

6.006 Introduction to Algorithms

Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 LLVM Project

Kaleidoscope and LLVM tutorials

Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 MIT OpenCourseWare

6.004 Computation Structures

Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 Remzi and Andrea Arpaci-Dusseau

Operating Systems: Three Easy Pieces

Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R17 IETF / RFC Editor

RFC 9293: Transmission Control Protocol

Transport semantics; reference reading after networking foundations.

https://www.rfc-editor.org/rfc/rfc9293.html

R18 PostgreSQL Global Development Group

PostgreSQL tutorial

Relational data, transactions and SQL; pin local server and client versions.

https://www.postgresql.org/docs/current/tutorial.html

R19 Carnegie Mellon University

15-462/662 Computer Graphics

Rendering, geometry and animation course structure and exercises.

https://15462.courses.cs.cmu.edu/fall2021/

R20 Carnegie Mellon University

15-466/666 Computer Game Programming

Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

R21 Khronos Group

Vulkan tutorial

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

R22 SDL project

SDL3 documentation

Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples.

https://wiki.libsdl.org/SDL3/FrontPage

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys

Physically Based Rendering, fourth edition

Rendering mathematics and implementation; use chapter titles in lesson assignments.

https://www.pbr-book.org/4ed/contents

R24 Microsoft

Direct3D 12 graphics

Windows graphics backend reference.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

R25 Apple

Metal developer resources

Apple-platform graphics and compute reference.

https://developer.apple.com/metal/

R26 Epic Games

Programming with C++ in Unreal Engine

Engine comparison after C++ foundations, not a beginner substitute.

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R27 Glenn Fiedler

Fix Your Timestep!

Original practitioner explanation of fixed-step simulation; read critically and implement your own lab.

https://gafferongames.com/post/fix_your_timestep/

R28 Box2D project

Box2D documentation

Collision, constraints and simulation reference.

https://box2d.org/documentation/

R29 Microsoft

Xbox Accessibility Guidelines

Accessibility design and testing; not a legal-compliance certificate.

https://learn.microsoft.com/en-us/xbox/accessibility/guidelines

R30 MIT OpenCourseWare

6.002 Circuits and Electronics

Circuits, transistors and laboratory reasoning.

https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/

R31 Noam Nisan and Shimon Schocken

Nand to Tetris

Computer construction projects; review the authors' license terms.

https://www.nand2tetris.org/

R32 Verilator project

Verilator user guide

RTL simulation and linting; match supported SystemVerilog subset.

https://verilator.org/guide/latest/

R33 YosysHQ

Yosys documentation

Synthesis and technology mapping; stable installed release preferred over a development snapshot.

https://yosyshq.readthedocs.io/projects/yosys/en/latest/

R34 cocotb project

cocotb documentation

Python-based HDL verification; simulator and version compatibility must be checked.

https://docs.cocotb.org/en/stable/

R35 RISC-V International

RISC-V ISA specifications

Instruction-set reference; record the exact ISA version and implemented extensions.

https://docs.riscv.org/reference/isa/

R36 KiCad project

KiCad documentation

Schematic capture, PCB layout and manufacturing outputs.

https://docs.kicad.org/

R37 NVIDIA

CUDA Programming Guide

GPU execution and memory; CUDA is an optional vendor-specific lab route.

https://docs.nvidia.com/cuda/cuda-programming-guide/

R38 gem5 project

gem5 documentation

Architecture simulation; distinguish model outputs from measured hardware results.

https://www.gem5.org/documentation/

R39 OpenROAD project

OpenROAD documentation

RTL-to-physical-design study; educational results are not production signoff.

https://openroad.readthedocs.io/en/latest/

R40 OpenStax / Rice University

Precalculus, second edition

Algebra, functions and trigonometry bridge before calculus.

https://openstax.org/details/books/precalculus-2e

R41 MIT OpenCourseWare

18.02SC Multivariable Calculus

Partial derivatives, gradients, multiple integrals and constrained differentials.

https://ocw.mit.edu/courses/18-02sc-multivariable-calculus-fall-2010/

R42 Stephen Boyd and Lieven Vandenberghe

Convex Optimization

Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

https://web.stanford.edu/~boyd/cvxbook/

R43 Samuel Williams, Andrew Waterman and David Patterson

Roofline performance model, technical report

Original 2008 performance-model reference; use current measured hardware limits for experiments.

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

R44 Benedikt Bitterli and coauthors

Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

R45 LLVM Project

AddressSanitizer documentation

Memory-error instrumentation, limitations and supported toolchains.

https://clang.llvm.org/docs/AddressSanitizer.html

R46 University of California, Berkeley

CS188 Introduction to Artificial Intelligence textbook

Search, games, decision processes, learning and probabilistic reasoning.

https://inst.eecs.berkeley.edu/~cs188/textbook/

R47 Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola

Dive into Deep Learning

Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions.

https://d2l.ai/

R48 MIT OpenCourseWare

6.823 Computer System Architecture

Graduate architecture foundations; historical course concepts supplemented with current specifications.

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

R49 Christopher Batten / Cornell University

ECE5745 Complex Digital ASIC Design

Graduate ASIC methodology, circuits, EDA and design-project expectations.

https://www.csl.cornell.edu/courses/ece5745/

R50 YosysHQ

SymbiYosys documentation

Formal hardware verification, safety, liveness and cover properties.

https://yosyshq.readthedocs.io/projects/sby/en/latest/

R51 Linux kernel contributors

Driver implementer API guide

Device ownership, I/O, DMA and driver interfaces; use disposable VMs or simulated devices for kernel experiments.

https://docs.kernel.org/driver-api/index.html

R52 Khronos Group

OpenXR registry and specifications

XR coordinate spaces, frame timing and runtime interfaces; optional hardware-dependent extension.

https://registry.khronos.org/OpenXR/

R53 Zephyr Project

Zephyr documentation

Embedded kernel, drivers, board support and simulation; select and pin a supported release.

https://docs.zephyrproject.org/latest/introduction/index.html

R54 MIT OpenCourseWare

18.085 Computational Science and Engineering I

Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

https://ocw.mit.edu/courses/18-085-computational-science-and-engineering-i-fall-2008/

R55 Matthias Mueller and collaborators

Author publication archive: position-based simulation and XPBD

Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

https://matthias-research.github.io/pages/publications/publications.html

VOLUME 01 OF 10

Foundations and Your First Game

Beginner / early undergraduate

Learn to reason about programs, run code, debug, test, use Git and complete a tiny game.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 01 Starting as an engineer

01.1 What a program actually does • 01.2 Files, terminals and reproducible execution • 01.3 Trace before running • 01.4 Units, budgets and engineering notebooks

Module 02 Values, decisions and repetition

02.1 Types and expressions • 02.2 Conditions and boundary cases • 02.3 Loops and invariants • 02.4 Functions and contracts

Module 03 Organizing data and small programs

03.1 Sequences, dictionaries and identity • 03.2 Decomposition, modules and interfaces • 03.3 Files, serialization and validation • 03.4 Classes, composition and state machines

Module 04 Debugging, testing and collaboration

04.1 Debugging as an experiment • 04.2 Unit, integration and property tests • 04.3 Git, branches and recoverable changes • 04.4 Review, documentation and responsible AI help

Module 05 A complete first game

05.1 The update loop and command model • 05.2 Collision rules and complete behavior • 05.3 Deterministic replay and simple measurement • 05.4 Finishing, usability and the first portfolio artifact

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 01

Starting as an engineer

Prerequisites None. Basic reading, arithmetic and the ability to save a file.

Four lessons

01.1 What a program actually does

Explain state, instructions, input and output using a tiny game.

01.2 Files, terminals and reproducible execution

Run a saved Python program and distinguish a shell from an interpreter.

01.3 Trace before running

Predict a program by following assignments in order.

01.4 Units, budgets and engineering notebooks

Use units and simple arithmetic to check game calculations.

Module project

Create a learning repository with a working motion program, an environment record, a prediction table and a one-page explanation of the input-to-display path.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R04 Python Software Foundation — Python tutorial and language documentation

R15 MIT OpenCourseWare — 6.004 Computation Structures

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 01.1 • MODULE 01

What a program actually does

Learning goal Explain state, instructions, input and output using a tiny game.

A program is a set of instructions that transforms information. Its state is the information that must persist: a player position, a health value, or a list of enemies. An input is a new observation or command. An output might be a picture, sound, file or network message. A game repeatedly reads inputs, changes state and produces outputs. The picture is a representation of the state, not the state itself.

A processor executes machine instructions; an operating system manages resources and access to devices. A game engine is software that organizes recurring tasks such as rendering and asset loading. A graphics processor is hardware that executes suitable workloads. Keep these layers distinct: changing a game rule is not the same activity as designing a processor.

WORKED EXAMPLE

For a player at x = 2, the command move-right by 3 units produces x = 5. Drawing the character twice must not move it twice. Rendering observes state; the simulation changes it.

PRACTICE + PASS CRITERION

On paper, trace five commands: right 3, left 1, right 0, left 4, right 2, starting at x = 2. Draw boxes for input, simulation state and output. Save the trace. Pass: every state transition is explainable without running code.

ASSIGNED STUDY

R03: introductory functions/variables lecture; R15: Basics of Information.

CHECK YOUR UNDERSTANDING

1. What is the difference between state and a displayed image?

2. What final position results from the five lab commands?

3. Why should drawing a character not update its position?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.2 • MODULE 01

Files, terminals and reproducible execution

Learning goal Run a saved Python program and distinguish a shell from an interpreter.

A source file stores the instructions you write. A terminal presents a command-line interface; its shell starts programs and manages paths. The Python interpreter is a program that runs Python instructions. A shell prompt is not a Python prompt. Commands such as cd belong in the shell; expressions such as print(2 + 3) belong in Python code.

A working directory tells a process where relative paths begin. Many apparent programming errors are actually wrong-directory or wrong-file errors. Reproducibility starts by recording the exact command, file and tool version. Use a dedicated course folder. Do not run copied administrative commands or upload private files merely to make a tutorial work.

WORKED EXAMPLE

With day01_motion.py in the current folder, python3 day01_motion.py runs that file on many macOS/Linux setups. On Windows, py -3 day01_motion.py is a common launcher form. First confirm the available command with its --version option.

PRACTICE + PASS CRITERION

Follow START_HERE. Run the included program twice from the same folder; save the command and output in your journal. Intentionally misspell the filename, read the error, then repair it. Pass: you can explain which program produced the error.

ASSIGNED STUDY

R04: Using the Python Interpreter; installation links in START_HERE.

CHECK YOUR UNDERSTANDING

1. What does a relative filename depend on?

2. Is cd a Python statement in an ordinary script?

3. What three facts help another person reproduce a run?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.3 • MODULE 01

Trace before running

Learning goal Predict a program by following assignments in order.

An assignment evaluates the expression on the right and binds the result to the name on the left. In x = x + 1, the right-hand x means the old value. This is not an algebraic equation claiming that x equals x plus one forever. The order of instructions matters because later instructions can read changed state.

A trace table records values after each instruction. It is a small, executable model of your reasoning. Predict first, run second, and explain any difference. Copying code until it works trains recognition rather than independent reasoning. For an unfamiliar program, choose one tiny input and work through it before testing a large case.

WORKED EXAMPLE

Let x = 4; y = x + 2; x = 10. The final values are x = 10 and y = 6. The earlier calculation does not automatically rerun when x changes.

PRACTICE + PASS CRITERION

Create three assignment sequences with at least four statements each. Predict their outputs, then check with Python. Include one swap attempt that fails because it overwrites a needed value. Pass: fix the swap and explain the lost information.

ASSIGNED STUDY

R03: Functions, Variables; R04: An Informal Introduction to Python.

CHECK YOUR UNDERSTANDING

1. After a = 3; b = a; a = 8, what is b for these integers?

2. Why does x = y; y = x fail to swap two different values?

3. What makes prediction useful when a run disagrees?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 01.4 • MODULE 01

Units, budgets and engineering notebooks

Learning goal Use units and simple arithmetic to check game calculations.

A number without a unit can be misleading. Position may be measured in meters or pixels, time in seconds or milliseconds, and speed in units per second. Multiplying speed by elapsed time yields displacement. Adding a time directly to a position has no coherent physical meaning. Naming variables with units makes mistakes visible.

A performance budget is also arithmetic. At a target of f frames per second, the time available per frame is 1000/f milliseconds. This is a planning limit, not a guarantee that the machine will meet it. Record hypotheses, observations and conclusions separately. A measurement describes a particular workload on a particular system, not every possible game.

WORKED EXAMPLE

A speed of 6 units/second for 0.25 seconds gives 1.5 units of travel. At 60 frames/second, the frame interval is about 16.67 milliseconds. A 20 ms frame exceeds that interval.

PRACTICE + PASS CRITERION

Calculate travel at 2, 6 and 10 units/second for 0.1 and 0.5 seconds. Calculate frame intervals for 30, 60 and 120 frames/second. Pass: show units at every step and identify which assumptions are chosen targets.

ASSIGNED STUDY

R03: Variables and arithmetic; R15: Performance Measures.

CHECK YOUR UNDERSTANDING

1. How far does 8 units/second travel in 0.125 seconds?

2. What is the interval for 120 frames/second?

3. Why record hardware and workload with a timing result?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 02

Values, decisions and repetition

Prerequisites Module 01.

Four lessons

02.1 Types and expressions

Choose integers, real-valued numbers, strings and booleans appropriately.

02.2 Conditions and boundary cases

Write decisions with complete and non-overlapping rules.

02.3 Loops and invariants

Use repetition that terminates and preserves a stated property.

02.4 Functions and contracts

Separate a calculation from interaction and state its valid inputs.

Module project

Build a terminal-based resource-gathering simulation with bounded health, inventory and a fixed number of turns. Include invalid-input handling and a complete trace for a five-turn run.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R04 Python Software Foundation — Python tutorial and language documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 02.1 • MODULE 02

Types and expressions

Learning goal Choose integers, real-valued numbers, strings and booleans appropriately.

A type determines what operations mean and which values are representable. Integers suit counts; floating-point numbers approximate many real-valued measurements; strings store text; booleans represent true or false. Python performs some conversions automatically, but a text value such as "12" is not the same thing as the integer 12. Input from a text prompt begins as text.

Floating-point arithmetic is finite precision. Some simple decimal fractions cannot be represented exactly in a binary format. Do not begin by adding arbitrary rounding everywhere: first decide whether a quantity is a count, a continuous measurement or an exact monetary amount. Record assumptions about ranges and units before choosing the representation.

WORKED EXAMPLE

"3" + "4" produces the string "34" in Python, while 3 + 4 produces 7. An inventory count of 7.5 swords indicates a modeling problem unless partial items are deliberately supported.

PRACTICE + PASS CRITERION

Write a resource-cost calculator that converts two text inputs to integers and reports total cost. Test zero, a positive count and malformed text. Pass: invalid input produces a useful message rather than an unexplained crash.

ASSIGNED STUDY

R03: Functions, Variables and Exceptions; R04: Numbers and Text.

CHECK YOUR UNDERSTANDING

1. Why are "12" and 12 different?

2. What does "ab" + "cd" produce?

3. Would an exact item count normally use floating point? Why?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 02.2 • MODULE 02

Conditions and boundary cases

Learning goal Write decisions with complete and non-overlapping rules.

A condition selects behavior according to a boolean expression. Comparison operators such as < and >= make boundaries explicit. An if/elif/else chain evaluates conditions in order and takes the first matching branch. Independent if statements may execute more than one branch. These are different designs.

Boundary cases reveal missing rules. For a door that opens at level 5, level 4, level 5 and level 6 are more informative initial tests than three arbitrary large values. Compound conditions use and, or and not. Write parentheses when they improve clarity. Name complex conditions so that the code expresses the game rule rather than making readers reconstruct it.

WORKED EXAMPLE

A purchase is allowed when gold >= price and stock > 0. Having enough gold is insufficient when stock is zero. A zero price still requires a policy for availability.

PRACTICE + PASS CRITERION

Implement a door requiring either a key or both level >= 5 and stamina >= 10. Enumerate all key/no-key cases around the two numeric boundaries. Pass: explain why each case succeeds or fails.

ASSIGNED STUDY

R03: Conditionals; R04: if Statements.

CHECK YOUR UNDERSTANDING

1. Does gold > price allow buying an item when gold equals price?

2. Evaluate False or (True and False).

3. Why test immediately below, at and above a threshold?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 02.3 • MODULE 02

Loops and invariants

Learning goal Use repetition that terminates and preserves a stated property.

A loop repeats a body. A for loop commonly processes a known sequence; a while loop repeats while a condition holds. Every loop needs a reason it will terminate or an explicit design for ongoing operation. A changing progress measure, such as remaining turns, is often the simplest termination argument.

An invariant is a statement that remains true at a chosen point in every iteration. For example, gold never becomes negative if every purchase first checks affordability. Distinguish the number of iterations from the final value of a loop variable. Off-by-one errors often arise from forgetting whether an endpoint is included.

WORKED EXAMPLE

In Python, range(5) supplies 0, 1, 2, 3, 4: five values, not six. To process turn numbers 1 through 5, range(1, 6) makes the intended numbering explicit.

PRACTICE + PASS CRITERION

Simulate ten turns, collecting three units each turn and spending five only when affordable. Print a trace. Pass: exactly ten updates occur and the resource count is never negative. Add a test for zero turns.

ASSIGNED STUDY

R03: Loops; R04: for Statements and range.

CHECK YOUR UNDERSTANDING

1. How many values does range(2, 7) produce?

2. What must change in while remaining > 0 to make progress?

3. Give an invariant for an inventory limited to ten items.

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 02.4 • MODULE 02

Functions and contracts

Learning goal Separate a calculation from interaction and state its valid inputs.

A function packages an operation behind a name and parameters. Returning a value is different from printing it: the caller can use a returned value in another calculation or test. A contract describes valid inputs, promised outputs and relevant side effects. A pure function depends only on its inputs and does not change outside state. Such functions are usually easier to test.

Break a program into responsibilities rather than arbitrary lengths. A movement calculation should not also prompt for a username, write a save file and change global audio settings. Handling invalid inputs is part of the interface. Decide whether to reject, clamp or represent failure; do not silently choose inconsistent policies.

WORKED EXAMPLE

advance(position, velocity, dt) returns position + velocity * dt. With inputs 2, 8 and 0.25, it returns 4. A negative dt should be deliberately supported or rejected, not accidentally ignored.

PRACTICE + PASS CRITERION

Implement advance with a documented nonnegative finite dt contract. Test stationary motion, negative velocity, zero dt and invalid dt. Pass: the function returns a value and the caller controls printing.

ASSIGNED STUDY

R03: Functions and Unit Tests; R04: Defining Functions.

CHECK YOUR UNDERSTANDING

1. What is the result of advance(2, 8, 0.25)?

2. Why is returning a value more reusable than only printing it?

3. What belongs in a function contract besides parameter names?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 03

Organizing data and small programs

Prerequisites Module 02.

Four lessons

03.1 Sequences, dictionaries and identity

Choose collections according to the operations the program needs.

03.2 Decomposition, modules and interfaces

Divide a small game into parts that can be tested independently.

03.3 Files, serialization and validation

Save structured data without assuming every file is trustworthy.

03.4 Classes, composition and state machines

Represent behavior without creating an unnecessary inheritance hierarchy.

Module project

Build a data-driven turn simulator with functions, a list of entities, a validated configuration file and save/load round-trip tests.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R04 Python Software Foundation — Python tutorial and language documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 03.1 • MODULE 03

Sequences, dictionaries and identity

Learning goal Choose collections according to the operations the program needs.

A sequence organizes values by position. A dictionary associates keys with values. Use a list when order and iteration matter; use a dictionary when lookup by a meaningful key is central. The container is part of the model, not merely syntax. An entity name may change, so a stable identifier is often safer than using its displayed name as its permanent identity.

Mutable containers can be shared. Assigning a list to another variable does not automatically create an independent deep copy. Mutating a shared object can therefore surprise callers. Before modifying a collection during iteration, understand whether the iterator remains valid and whether elements will be skipped. Building a new collection is often clearer at this stage.

WORKED EXAMPLE

If a = [1, 2] and b = a, then b.append(3) also changes the list seen through a. By contrast, b = list(a) makes a new outer list, but nested mutable objects can still be shared.

PRACTICE + PASS CRITERION

Store five units with stable IDs, names and health. Look up a unit by ID and create a filtered list of living units. Pass: renaming a unit does not break lookup, and the original collection is unchanged by filtering.

ASSIGNED STUDY

R04: Data Structures; R03: Loops and dictionaries.

CHECK YOUR UNDERSTANDING

1. When is a dictionary preferable to a list?

2. Does b = a copy a Python list?

3. Why can a shallow copy still share changes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 03.2 • MODULE 03

Decomposition, modules and interfaces

Learning goal Divide a small game into parts that can be tested independently.

A module groups related definitions. An interface is the set of operations other code is expected to use. Good boundaries hide unnecessary details while exposing the information a caller needs. Start with a small separation: simulation logic, input handling and presentation. This is enough to prevent many tangles without inventing a large framework.

Dependencies should be intentional. Simulation can accept commands as data rather than directly reading a keyboard. Then the same simulation can run from recorded input, a test or a future network client. Avoid circular dependencies: if every module imports every other module, a local change can become a system-wide problem.

WORKED EXAMPLE

A simulate_turn(state, command) function can run in a terminal game, in a unit test and in a replay tool. It should not need to know which window or controller produced command.

PRACTICE + PASS CRITERION

Refactor the resource simulation into simulation.py and a separate command-line entry point. Pass: importing simulation.py does not start a game or prompt the user, and a test can call its logic directly.

ASSIGNED STUDY

R04: Modules; R03: Libraries and Unit Tests.

CHECK YOUR UNDERSTANDING

1. What is an interface?

2. Why avoid starting a game as a side effect of importing simulation.py?

3. How does command-as-data design help replays?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 03.3 • MODULE 03

Files, serialization and validation

Learning goal Save structured data without assuming every file is trustworthy.

Serialization converts an in-memory representation into a storable or transmittable form. Deserialization reconstructs a representation from that form. The file format needs a schema: which fields exist, their types and their valid ranges. A syntactically valid JSON file can still contain impossible health values or missing required fields.

Treat file contents as inputs, even when you normally create the files yourself. Files can be truncated, edited or come from a different program version. Do not use unrestricted evaluation to load a save. A round-trip test checks that saving and loading preserve the intended information, but it must be complemented by malformed-input tests.

WORKED EXAMPLE

A save with {"version": 1, "health": 50} is structurally clearer than an undocumented sequence of numbers. Validation should reject a string health or an unsupported version with an actionable explanation.

PRACTICE + PASS CRITERION

Create JSON save/load functions for a small state. Test a normal round trip, missing field, wrong type, unsupported version and truncated file. Pass: the original working save remains usable after a failed load.

ASSIGNED STUDY

R04: Input and Output, json documentation linked from the standard library.

CHECK YOUR UNDERSTANDING

1. Does valid JSON imply a valid game state?

2. What does a round-trip test check?

3. Why include a format version?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 03.4 • MODULE 03

Classes, composition and state machines

Learning goal Represent behavior without creating an unnecessary inheritance hierarchy.

A class can combine state and operations that maintain its rules. Encapsulation means that other code uses an interface rather than directly violating those rules. Composition builds a larger object from smaller capabilities. Inheritance describes an is-a relationship and can support substitution, but it is not a requirement for every game object.

A finite state machine models a system using named states and transitions. A character might be idle, moving or stunned. Writing allowed transitions explicitly prevents contradictory combinations such as being both stunned and actively attacking, unless the design intentionally allows that. The model should follow actual behavior, not a fashionable pattern.

WORKED EXAMPLE

A door has Closed and Open states. An Open command succeeds only when its lock policy permits the Closed-to-Open transition. Repeated Open commands should have a documented harmless or rejected result.

PRACTICE + PASS CRITERION

Implement a three-state character controller with a transition table. Pass: test every allowed transition and at least three forbidden transitions. Explain why composition would help add inventory independently of movement.

ASSIGNED STUDY

R04: Classes; R03: Object-Oriented Programming.

CHECK YOUR UNDERSTANDING

1. What does encapsulation protect?

2. Why can independent booleans create contradictory states?

3. When is composition useful?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 04

Debugging, testing and collaboration

Prerequisites Modules 01-03.

Four lessons

04.1 Debugging as an experiment

Find a defect by narrowing evidence rather than making random edits.

04.2 Unit, integration and property tests

Choose tests that challenge the contract instead of repeating the implementation.

04.3 Git, branches and recoverable changes

Use small commits to preserve reasoning and collaborate safely.

04.4 Review, documentation and responsible AI help

Explain a change and verify assistance without surrendering understanding.

Module project

Publish a local tagged release of the terminal simulation with tests, a reproducible bug report, a reviewed change and an evidence-based development journal. A remote public upload is not required.

Core references

R03 Harvard University — CS50 Introduction to Programming with Python

R05 Scott Chacon and Ben Straub — Pro Git

R11 C++ Core Guidelines editors — C++ Core Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 04.1 • MODULE 04

Debugging as an experiment

Learning goal Find a defect by narrowing evidence rather than making random edits.

A bug report distinguishes expected behavior from observed behavior and supplies reproducible steps. A debugger lets you pause, inspect variables and follow control flow. Logging records selected events over time. Neither tool replaces a hypothesis: identify what you believe is wrong, choose an observation that could disprove it, and change one relevant thing.

Reduce failing examples. A ten-line reproduction is easier to reason about than a full game. Check the earliest point at which state becomes incorrect, not only where a crash finally happens. Keep the original failure and its fix in a regression test so that future changes do not silently reintroduce it.

WORKED EXAMPLE

If a unit loses health twice, log command receipt and damage application separately. Two damage events could come from duplicate commands or from one command processed twice; the two hypotheses require different fixes.

PRACTICE + PASS CRITERION

Introduce a double-damage bug into a copy of the simulator. Write expected/actual behavior, reduce the reproduction and use a breakpoint or trace to identify the first incorrect update. Pass: fix it with a regression test.

ASSIGNED STUDY

R03: Exceptions and Unit Tests; R11: assertions and error-handling guidance.

CHECK YOUR UNDERSTANDING

1. What information makes a bug report actionable?

2. Why inspect the earliest wrong state?

3. What is a regression test?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 04.2 • MODULE 04

Unit, integration and property tests

Learning goal Choose tests that challenge the contract instead of repeating the implementation.

A unit test checks a small responsibility. An integration test checks that components work together. A property test checks a rule over many inputs, such as inventory never becoming negative after valid purchases. The expected result should come from the specification, a simpler reference implementation or a mathematical property, not a copy of the same potentially wrong algorithm.

Tests are evidence, not proof of all behavior. Passing tests cannot demonstrate untested requirements. Include empty inputs, boundaries, invalid inputs and representative workloads. Keep tests repeatable by controlling randomness and time. A flaky test teaches people to ignore alarms and should be investigated rather than routinely rerun until it passes.

WORKED EXAMPLE

For clamp(x, 0, 100), examples include -1 -> 0, 50 -> 50 and 101 -> 100. A useful property is that every output lies in [0, 100] and clamping an already clamped value changes nothing.

PRACTICE + PASS CRITERION

Write at least twelve tests across motion, resource spending and save/load. Include one property-style loop over many small inputs. Pass: temporarily insert a known bug and demonstrate that a relevant test fails.

ASSIGNED STUDY

R03: Unit Tests; R04: unittest standard-library documentation.

CHECK YOUR UNDERSTANDING

1. How does an integration test differ from a unit test?

2. Why is copying the implementation into the test oracle weak?

3. What does a test suite passing establish?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 04.3 • MODULE 04

Git, branches and recoverable changes

Learning goal Use small commits to preserve reasoning and collaborate safely.

Git records snapshots of tracked files. The working tree is what you edit; the staging area selects the content for the next commit; a commit stores a snapshot and history relationship. A branch is a movable reference to a line of development, not a complete independent copy of every file.

Small commits with meaningful messages make review and recovery easier. Inspect differences before committing. Exclude generated files, credentials and large unnecessary binaries. A remote repository is useful for collaboration and additional copies, but it is not permission to publish someone else's assets or secrets. Learn revert on a practice repository before using destructive history operations on shared work.

WORKED EXAMPLE

A commit message such as "Reject negative purchase quantities; add boundary tests" explains an intent. "Updates" forces a reviewer to reconstruct it from every changed line.

PRACTICE + PASS CRITERION

Initialize the course repository, make three focused commits and implement one feature on a branch. Merge it and practice reverting an intentionally bad commit in a disposable copy. Pass: identify exactly what changed and why.

ASSIGNED STUDY

R05: Getting Started, Git Basics and Basic Branching and Merging.

CHECK YOUR UNDERSTANDING

1. What does the staging area control?

2. Why should secrets not be committed even if later deleted?

3. What makes a useful commit message?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 04.4 • MODULE 04

Review, documentation and responsible AI help

Learning goal Explain a change and verify assistance without surrendering understanding.

A review asks whether a change meets its requirements, preserves contracts and is understandable to the next maintainer. Critique behavior and evidence rather than the person. Documentation should explain setup, decisions and limitations; comments should add intent rather than merely restating each statement.

An AI assistant can explain errors, suggest test cases and compare approaches. Its output is untrusted until checked. Do not submit generated code you cannot explain, and do not expose private data or licensed material without permission. For this course, attempt each quiz unaided, label assisted work and complete a small no-assistance transfer task before marking a skill mastered.

WORKED EXAMPLE

Instead of asking "write the whole engine," ask for three possible causes of a reproducible failure, then test them. Preserve your prediction, the suggestion and the evidence that accepted or rejected it.

PRACTICE + PASS CRITERION

Review a ten-line change using a checklist: requirement, correctness, boundary cases, readability and tests. Write a README that a second person can follow. Pass: explain every changed line and recreate a small related feature without assistance.

ASSIGNED STUDY

R05: Distributed Git; R11: philosophy and interface guidance; course assessment policy.

CHECK YOUR UNDERSTANDING

1. What should a review criticize?

2. Why is compiling AI-generated code insufficient validation?

3. What demonstrates understanding beyond copying a solution?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 05

A complete first game

Prerequisites Modules 01-04.

Four lessons

05.1 The update loop and command model

Separate simulation ticks from input collection and presentation.

05.2 Collision rules and complete behavior

Specify edge cases before implementing a playable rule.

05.3 Deterministic replay and simple measurement

Reproduce a run and measure the work actually performed.

05.4 Finishing, usability and the first portfolio artifact

Deliver a tiny game another person can run and understand.

Module project

Complete Gate A: a deterministic terminal grid game with movement, obstacles, win/loss states, replay, tests and a clean-start README. Use the detailed project brief in the program handbook.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R22 SDL project — SDL3 documentation

R27 Glenn Fiedler — Fix Your Timestep!

R29 Microsoft — Xbox Accessibility Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 05.1 • MODULE 05

The update loop and command model

Learning goal Separate simulation ticks from input collection and presentation.

A simulation tick is one application of the game's update rules. A frame is one produced image. They need not occur at the same rate. Start with a turn-based loop so each command advances exactly one tick. Store commands as values and let update(state, command) apply the rules.

Do not put important state transitions in rendering or sound playback. This separation allows headless testing, where no window is needed. It also makes failures reproducible: the same initial state and command sequence should produce the same state when the simulation is deterministic. Later lessons will address numerical and scheduling obstacles to cross-platform determinism.

WORKED EXAMPLE

For a grid game, command Right can move from (2, 3) to (3, 3) only when the destination is valid. Render reads the resulting grid but performs no additional move.

PRACTICE + PASS CRITERION

Build a 10-by-10 terminal grid with one player and an exit. Pass: one command causes one update, drawing twice changes nothing, and the update function works without printing.

ASSIGNED STUDY

R20: runtime systems course overview; R27: fixed-step concepts, read without copying its implementation.

CHECK YOUR UNDERSTANDING

1. Are a simulation tick and a rendered frame necessarily identical?

2. What should happen when render is called twice without an update?

3. Why is headless simulation useful?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 05.2 • MODULE 05

Collision rules and complete behavior

Learning goal Specify edge cases before implementing a playable rule.

A rule is incomplete when it defines only the happy path. Movement must say what happens at a wall, at the grid boundary, when two commands arrive or after the game is over. A collision system answers questions about overlap or blocked motion, but the game design decides what the answer means.

For this project, use integer grid coordinates and a set of blocked cells. Reject invalid moves without partially changing state. This is a simple transaction: validate, then apply. Separate win/loss detection from input text parsing so a typo does not accidentally consume a game turn unless that is the documented rule.

WORKED EXAMPLE

On a grid with x in 0 through 9, moving right from x = 9 must not yield x = 10. A chosen policy is to keep the old state and report blocked; wrapping is a different valid game rule.

PRACTICE + PASS CRITERION

Add five obstacles, a goal and a terminal win state. Test every edge of the board, blocked movement, reaching the goal and commands after victory. Pass: no partial movement occurs on rejected commands.

ASSIGNED STUDY

R20: game-programming projects; course transition and testing lessons.

CHECK YOUR UNDERSTANDING

1. What is missing from "move right increases x"?

2. Under the reject policy, what happens at x = 9 on a 0-9 grid?

3. Why validate before applying a multi-part state change?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 05.3 • MODULE 05

Deterministic replay and simple measurement

Learning goal Reproduce a run and measure the work actually performed.

A replay stores initial conditions and commands in their intended order. If randomness is used, record the seed and the generator policy as well. Determinism means the relevant outputs match for the same defined inputs and environment. It does not follow merely from setting one random seed when other sources of variability remain.

Measure update cost separately from printing; terminal output can dominate a small program. Use a monotonic timing source for elapsed durations. Repeat a representative workload and report the workload size, repetitions and environment. Avoid claiming that a small terminal benchmark predicts the performance of a future three-dimensional game.

WORKED EXAMPLE

A replay of Right, Right, Down from (0, 0) ends at (2, 1) on an unobstructed grid where Down increases y. A blocked cell at (1, 0) changes that result under the rejection policy.

PRACTICE + PASS CRITERION

Save and replay a twenty-command session. Run the same replay repeatedly and compare final states. Time 10,000 headless updates separately from display. Pass: report exact inputs and no invented performance target.

ASSIGNED STUDY

R27: reproducibility motivation; R04: time and random standard-library references.

CHECK YOUR UNDERSTANDING

1. What information does a replay need besides button names?

2. Why exclude printing from a simulation microbenchmark?

3. Does one matching replay prove determinism for all inputs?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 05.4 • MODULE 05

Finishing, usability and the first portfolio artifact

Learning goal Deliver a tiny game another person can run and understand.

A finished small project teaches integration that disconnected exercises cannot. Define a limited scope and complete it: startup, instructions, gameplay, termination and error messages all matter. Include a way to quit and restart. Avoid relying on color alone to convey important information. Clear controls and readable output are engineering responsibilities even in a terminal game.

A portfolio artifact should include source, tests, run instructions, a short demonstration and a discussion of decisions. Explain what you built yourself and what came from references. Do not inflate a simple project into a claim of production-engine expertise. An honest account of a difficult bug and a measured improvement is more informative than an unsupported adjective.

WORKED EXAMPLE

A useful release note says "Added obstacle rejection and a replay test for the top-right boundary." A useful limitation says "Keyboard commands are turn-based; there is no real-time renderer yet."

PRACTICE + PASS CRITERION

Complete Gate A and ask a tester to start from the README without verbal help. Record three observations and fix at least one usability issue. Pass: the tester can start, play, quit and rerun the tests.

ASSIGNED STUDY

R29: Input, UI navigation and error-message guidelines; R05: tagging releases.

CHECK YOUR UNDERSTANDING

1. What belongs in a clean-start README?

2. Why should color not be the only important cue?

3. What evidence should accompany a claimed improvement?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 02 OF 10

Mathematics for Games and Computing

Undergraduate mathematical core

Build the mathematical language for algorithms, graphics, physics, signals and experiments.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 06 Discrete mathematical reasoning

06.1 Algebra, functions, logarithms and trigonometry • 06.2 Logic, sets and quantified claims • 06.3 Proofs, induction and invariants • 06.4 Counting, graphs and recurrences

Module 07 Linear algebra and game geometry

07.1 Vectors, dot products and projections • 07.2 Matrices, bases and linear systems • 07.3 Homogeneous transforms, cameras and quaternions • 07.4 Eigenvalues, least squares and singular values

Module 08 Calculus, mechanics and numerical simulation

08.1 Limits, derivatives and rates • 08.2 Integrals, accumulation and continuous probability • 08.3 Newtonian mechanics and differential equations • 08.4 Time integration, stability and error

Module 09 Probability, statistics and evidence

09.1 Probability and conditional reasoning • 09.2 Random variables, distributions and sampling • 09.3 Inference, uncertainty and benchmark design • 09.4 Regression, correlation and ethical telemetry

Module 10 Signals, optimization and numerical foundations

10.1 Multivariable calculus and gradients • 10.2 Sampling, aliasing and Fourier ideas • 10.3 Convolution, filters and systems • 10.4 Optimization, conditioning and approximation budgets

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 06

Discrete mathematical reasoning

Prerequisites Volume 01; complete the algebra diagnostic in START_HERE before proceeding.

Four lessons

06.1 Algebra, functions, logarithms and trigonometry

Translate a game relationship into an equation with a defined domain.

06.2 Logic, sets and quantified claims

Express requirements precisely enough to test or prove.

06.3 Proofs, induction and invariants

Justify a program property instead of testing only examples.

06.4 Counting, graphs and recurrences

Model combinations and recursive work without brute-force guessing.

Module project

Write a mathematical notebook containing a truth-table specification, an induction proof, a graph model and a justified operation-count estimate. Include counterexamples to three false claims.

Core references

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

R08 MIT OpenCourseWare — 18.01SC Single Variable Calculus

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 06.1 • MODULE 06

Algebra, functions, logarithms and trigonometry

Learning goal Translate a game relationship into an equation with a defined domain.

A function maps each allowed input to one output. Its domain matters: dividing by zero or taking a real square root of a negative number is not repaired by notation. Rearranging an equation must preserve its conditions. Exponents model repeated multiplication; logarithms invert exponentiation. log2(n) often counts how many times a size can be halved.

Angles connect geometry to periodic motion. In radians, a full revolution is 2*pi. For a unit circle, a direction at angle t has components (cos(t), sin(t)). Check units before calling a trigonometric function. Degrees and radians are different representations of the same angle, not interchangeable numeric inputs.

WORKED EXAMPLE

At 90 degrees, or pi/2 radians, the unit-circle direction is (0, 1). A list of 1024 items can be halved ten times to reach one because 2^10 = 1024.

PRACTICE + PASS CRITERION

Solve distance = speed*time for each variable and state forbidden inputs. Convert 0, 90, 180 and 360 degrees to radians. Pass: solve ten algebra problems and explain logarithms using repeated halving before advancing.

ASSIGNED STUDY

R08: prerequisite algebra review and trigonometric functions; R06: sums and asymptotics.

CHECK YOUR UNDERSTANDING

1. What is log2(64)?

2. Convert 180 degrees to radians.

3. What restriction applies when computing speed = distance/time?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 06.2 • MODULE 06

Logic, sets and quantified claims

Learning goal Express requirements precisely enough to test or prove.

Propositional logic combines true/false statements. Implication P -> Q fails only when P is true and Q is false. It does not claim that P actually happens. A universal claim says a property holds for every allowed input; an existential claim says at least one input has it. Negating a universal claim requires one counterexample.

Sets group distinct elements; relations describe connections; functions are relations with exactly one output for each domain element. These ideas support requirements such as every live entity has one valid transform. Ambiguous language like "usually valid" cannot serve as an invariant until its exceptions are specified.

WORKED EXAMPLE

"Every saved entity has an ID" is disproved by one saved entity without an ID. "Some entity is visible" requires only one witness and does not imply all entities are visible.

PRACTICE + PASS CRITERION

Formalize five requirements for the grid game using sets and quantifiers. Construct truth tables for a door-access rule. Pass: provide a counterexample or witness for each negated requirement.

ASSIGNED STUDY

R06: propositions, predicates, sets and relations.

CHECK YOUR UNDERSTANDING

1. When is P -> Q false?

2. Negate "every unit has positive health."

3. Why must the domain of a quantified statement be explicit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 06.3 • MODULE 06

Proofs, induction and invariants

Learning goal Justify a program property instead of testing only examples.

A proof is a chain of justified steps from assumptions to a conclusion. Direct proof, contradiction and induction solve different kinds of questions. Induction establishes a base case and shows that validity at one step implies validity at the next. Both parts are necessary.

A loop invariant connects mathematical reasoning to implementation: establish it before the loop, preserve it in each iteration, and combine it with the exit condition to prove the result. Testing complements proofs by checking whether code matches the modeled assumptions. A proof of an ideal algorithm does not automatically establish the correctness of its floating-point implementation.

WORKED EXAMPLE

To prove the running sum after k iterations equals 1+...+k, start at k=0 with sum=0. If the claim holds at k, adding k+1 produces the required sum for k+1.

PRACTICE + PASS CRITERION

Prove the formula 1+...+n = n(n+1)/2 by induction. Then write an invariant for counting living units in a prefix of a list. Pass: show initialization, preservation and termination explicitly.

ASSIGNED STUDY

R06: induction, invariants and proof methods.

CHECK YOUR UNDERSTANDING

1. What are the two essential parts of induction?

2. What is 1+...+100?

3. Why does a mathematical proof not eliminate implementation tests?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 06.4 • MODULE 06

Counting, graphs and recurrences

Learning goal Model combinations and recursive work without brute-force guessing.

The product rule counts independent sequential choices; the sum rule counts disjoint alternatives. A graph represents vertices and edges, which may be directed or weighted. A level map, dependency graph and network topology are different applications of the same abstraction. Decide what an edge means before choosing an algorithm.

A recurrence describes a quantity in terms of smaller instances. Repeatedly halving a search space gives logarithmic depth, but doing linear work at every level changes total cost. Do not confuse possible states with states actually explored by a particular algorithm. Counting provides upper bounds and exposes when exhaustive search becomes unrealistic.

WORKED EXAMPLE

Four independent equipment slots with three choices each give 3^4 = 81 combinations. An undirected simple graph with n vertices has at most n(n-1)/2 edges because each unordered pair appears once.

PRACTICE + PASS CRITERION

Build a dependency graph for game startup and identify an invalid cycle. Count loadout combinations with and without a restriction. Pass: explain the recurrence for halving a search range and distinguish depth from total work.

ASSIGNED STUDY

R06: counting, graphs, recurrences and asymptotic notation.

CHECK YOUR UNDERSTANDING

1. How many combinations arise from five independent binary choices?

2. How many edges can a simple undirected graph with five vertices have?

3. Why is a dependency cycle a problem for a simple startup order?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 07

Linear algebra and game geometry

Prerequisites Module 06.1; basic Python functions.

Four lessons

07.1 Vectors, dot products and projections

Use vectors for direction, distance and component measurements.

07.2 Matrices, bases and linear systems

Understand a matrix as a transformation rather than a table to memorize.

07.3 Homogeneous transforms, cameras and quaternions

Compose a scene using explicit coordinate conventions.

07.4 Eigenvalues, least squares and singular values

Recognize stability and approximation problems in game data.

Module project

Build a tested vector/matrix library for a two-dimensional scene, then demonstrate a three-dimensional camera transformation with written coordinate conventions.

Core references

R07 MIT OpenCourseWare — 18.06 Linear Algebra

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 07.1 • MODULE 07

Vectors, dot products and projections

Learning goal Use vectors for direction, distance and component measurements.

A vector represents a magnitude and direction in a chosen coordinate system. A point represents a location; subtracting points yields a displacement vector. The Euclidean length of (x,y,z) is sqrt(x*x+y*y+z*z). Normalization divides by length, so the zero vector needs a deliberate policy.

The dot product measures alignment and produces a scalar. For unit vectors it equals the cosine of the included angle. Projection onto a unit direction u is dot(v,u)*u. In three dimensions the cross product produces a perpendicular vector with orientation determined by the coordinate convention. Do not use a cross-product sign without declaring handedness.

WORKED EXAMPLE

For v=(3,4), length is 5 and its unit direction is (0.6,0.8). Dotting v with u=(1,0) gives 3, the component along the x axis.

PRACTICE + PASS CRITERION

Implement length, dot and safe normalization. Use dot products to determine whether a target is in front of a character. Pass: test zero vectors, perpendicular vectors and opposite directions.

ASSIGNED STUDY

R07: vectors and projections; R23: Geometry and Transformations / Vectors.

CHECK YOUR UNDERSTANDING

1. What is the length of (6,8)?

2. What is the dot product of (1,0) and (0,1)?

3. Why cannot the zero vector be normalized by the ordinary formula?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 07.2 • MODULE 07

Matrices, bases and linear systems

Learning goal Understand a matrix as a transformation rather than a table to memorize.

A matrix describes how basis vectors are mapped. Matrix-vector multiplication combines those images to transform a vector. A basis is a set of independent directions spanning the space. Changing basis changes coordinates without necessarily changing the underlying geometric object.

Matrix multiplication composes transformations, and order generally matters. Solving Ax=b is not the same task as explicitly computing A inverse; elimination or factorization is often preferable. A singular matrix loses information and cannot be uniquely inverted. In graphics, a zero scale along an axis is a concrete example of collapsing multiple points to the same location.

WORKED EXAMPLE

In two dimensions, scaling x by 2 maps (1,1) to (2,1). Rotating and then applying a nonuniform scale generally differs from applying the scale and then rotating.

PRACTICE + PASS CRITERION

Implement 2-by-2 multiplication and solve two small linear systems by elimination on paper. Pass: show a pair of transforms that do not commute and explain a singular example geometrically.

ASSIGNED STUDY

R07: matrix multiplication, elimination, inverses and vector spaces.

CHECK YOUR UNDERSTANDING

1. Do matrices generally satisfy AB = BA?

2. What happens to invertibility when an axis is scaled to zero?

3. Why solve Ax=b without explicitly forming an inverse?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 07.3 • MODULE 07

Homogeneous transforms, cameras and quaternions

Learning goal Compose a scene using explicit coordinate conventions.

Homogeneous coordinates allow translation to be represented with matrix multiplication. In a common three-dimensional convention, points use w=1 and directions use w=0. A model transform maps local coordinates into world space; a view transform maps world coordinates into camera space. Perspective projection introduces a division by w and changes how depth is represented.

Choose row or column vectors, multiplication order, handedness and depth range, then document them. A unit quaternion represents orientation compactly and supports interpolation, but multiplication order still matters and accumulated error may require normalization. Euler angles remain useful for interfaces; their limitations do not make them meaningless.

WORKED EXAMPLE

With column vectors and a local-to-world matrix M, a point is p_world=M*p_local. A parent-child chain is M_parent*M_child*p_local. Reversing it changes the coordinate interpretation.

PRACTICE + PASS CRITERION

Create a parented object and camera using documented conventions. Pass: test identity, inverse round trips within tolerance, a translated direction that remains unchanged, and a parent rotation that moves its child correctly.

ASSIGNED STUDY

R23: Coordinate Systems and Transformations; R19: geometric transformations.

CHECK YOUR UNDERSTANDING

1. Why commonly use w=0 for directions?

2. What does the view transform do?

3. Why document multiplication order even when using a library?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 07.4 • MODULE 07

Eigenvalues, least squares and singular values

Learning goal Recognize stability and approximation problems in game data.

An eigenvector keeps its direction under a linear transformation; its eigenvalue is the scale applied along that direction. These concepts help analyze repeated updates and dynamical systems. Least squares fits an approximate solution when equations are inconsistent or measurements are noisy. It minimizes a chosen squared residual, not necessarily every meaningful error.

Singular values describe how strongly a matrix stretches different directions. Very small singular values can make inversion sensitive to input errors. Condition number measures sensitivity of the problem; algorithmic stability concerns how a method handles arithmetic error. A well-written algorithm cannot remove information absent from a badly conditioned problem.

WORKED EXAMPLE

The diagonal matrix diag(2,0.5) stretches the x direction by 2 and y by 0.5. Repeated application expands one component while shrinking the other.

PRACTICE + PASS CRITERION

Fit a line to five noisy position/time observations and report residuals. Analyze repeated multiplication by a diagonal matrix. Pass: distinguish data noise, poor conditioning and a programming error in your report.

ASSIGNED STUDY

R07: eigenvalues, least squares and singular value decomposition.

CHECK YOUR UNDERSTANDING

1. What are the eigenvalues of diag(3,2)?

2. What objective does ordinary least squares minimize?

3. Can a stable algorithm make an ill-conditioned problem insensitive?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 08

Calculus, mechanics and numerical simulation

Prerequisites Modules 06.1 and 07.1; work through the linked calculus problem sets.

Four lessons

08.1 Limits, derivatives and rates

Connect derivatives to velocity, acceleration and local approximation.

08.2 Integrals, accumulation and continuous probability

Interpret integration as accumulated contribution.

08.3 Newtonian mechanics and differential equations

Model motion from forces with clear simplifying assumptions.

08.4 Time integration, stability and error

Choose and test a numerical update rather than assuming smaller code is correct.

Module project

Build a projectile and spring simulator with analytic comparison cases, timestep refinement, unit checks and a report separating model error from numerical error.

Core references

R08 MIT OpenCourseWare — 18.01SC Single Variable Calculus

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

R28 Box2D project — Box2D documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 08.1 • MODULE 08

Limits, derivatives and rates

Learning goal Connect derivatives to velocity, acceleration and local approximation.

A derivative measures the local rate of change of a function. Velocity is the derivative of position with respect to time; acceleration is the derivative of velocity. The derivative carries units: meters differentiated by seconds gives meters per second. Product and chain rules explain how composed calculations change.

A finite difference approximates a derivative from samples. Making the step smaller reduces some approximation errors but can worsen floating-point cancellation. The derivative of an ideal formula and the difference between two measured samples answer related but not identical questions. Always distinguish the mathematical function from the noisy or discretized data available to your program.

WORKED EXAMPLE

For x(t)=t*t, velocity is 2*t. At t=3, velocity is 6. The forward difference using h=0.1 is ((3.1)^2-3^2)/0.1=6.1, an approximation rather than the exact derivative.

PRACTICE + PASS CRITERION

Compare analytic and finite-difference derivatives of three functions over several step sizes. Pass: include units, an error table and an explanation of why the smallest tested h need not be best.

ASSIGNED STUDY

R08: differentiation, product rule and chain rule.

CHECK YOUR UNDERSTANDING

1. Differentiate x(t)=3*t*t.

2. What units does acceleration have when position is in meters?

3. Why can excessively small finite-difference steps be inaccurate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 08.2 • MODULE 08

Integrals, accumulation and continuous probability

Learning goal Interpret integration as accumulated contribution.

An integral accumulates contributions over a domain. Integrating velocity over time gives displacement; integrating acceleration changes velocity. The fundamental theorem of calculus connects derivatives and integrals. A definite integral is a signed accumulation, so negative contributions can cancel positive ones.

Numerical quadrature approximates an integral with samples and weights. Rectangle, trapezoid and higher-order rules have different accuracy assumptions. More samples do not repair a wrong model or omitted discontinuity. In graphics, integration will later accumulate incoming light over directions; in probability, a density integrates to a probability over an interval.

WORKED EXAMPLE

With constant velocity 4 meters/second over 3 seconds, displacement is 12 meters. For v(t)=2*t from 0 to 3, integrating gives 9 meters, not the final velocity multiplied by the whole duration.

PRACTICE + PASS CRITERION

Implement rectangle and trapezoid integration for v(t)=2*t and a curved function. Pass: compare to an analytic integral, halve the step and report how error changes.

ASSIGNED STUDY

R08: integration and the fundamental theorem; R23: Monte Carlo Integration introduction.

CHECK YOUR UNDERSTANDING

1. Integrate constant velocity 5 over two seconds.

2. What is the integral of 2*t from 0 to 3?

3. Why can an integral be zero despite nonzero motion?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 08.3 • MODULE 08

Newtonian mechanics and differential equations

Learning goal Model motion from forces with clear simplifying assumptions.

Newton's second law relates net force to mass times acceleration. Position and velocity form the state of a basic particle model. Gravity, springs and damping provide forces with different dependencies. A differential equation specifies how state changes; initial conditions select a particular trajectory.

For a simple spring, force is proportional to displacement from rest and opposite its sign. Damping depends on velocity and dissipates energy in the model. Distinguish mass from weight and force from impulse. Games often choose simplified models for control and performance; simplification is legitimate when its behavior and limitations are explicit.

WORKED EXAMPLE

A 2 kg body under a constant 6 N net force has acceleration 3 m/s^2. Over 0.5 seconds, its velocity changes by 1.5 m/s. This says nothing about drag unless drag is included in the net force.

PRACTICE + PASS CRITERION

Implement a particle under constant acceleration and a damped spring. Pass: compare the constant-acceleration trajectory to its analytic solution and state what collision, rotation and air-resistance effects are omitted.

ASSIGNED STUDY

R28: simulation, bodies, forces and impulses; R08: differential-equation applications.

CHECK YOUR UNDERSTANDING

1. What acceleration results from 10 N on a 2 kg mass?

2. How does impulse differ from force?

3. Why must initial conditions accompany a differential equation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 08.4 • MODULE 08

Time integration, stability and error

Learning goal Choose and test a numerical update rather than assuming smaller code is correct.

Explicit Euler advances state using derivatives at the old state. Semi-implicit Euler updates velocity first and then uses the new velocity for position. Other methods trade more work for different accuracy and stability properties. A method can have small local error yet accumulate unacceptable global drift.

Timestep choice depends on the system, method and acceptable error. Stiff springs can require much smaller explicit steps than slow motion. Fixed timesteps aid repeatability but do not guarantee stability or identical results across machines. Compare against analytic cases, inspect energy where meaningful and repeat with a refined step before trusting a visually plausible animation.

WORKED EXAMPLE

For explicit Euler: x_next=x+v*dt; v_next=v+a*dt. For semi-implicit Euler, compute v_next first and use x_next=x+v_next*dt. The methods differ even with identical inputs.

PRACTICE + PASS CRITERION

Compare explicit and semi-implicit Euler on an undamped spring at three timesteps. Pass: plot or tabulate position and energy error, identify an unstable case, and avoid calling visual smoothness proof of physical accuracy.

ASSIGNED STUDY

R23: physically based animation topics via R19; R28: simulation stepping guidance.

CHECK YOUR UNDERSTANDING

1. Which velocity does semi-implicit Euler use for the position update?

2. Does a fixed timestep guarantee a stable simulation?

3. What is a useful check beyond watching the animation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 09

Probability, statistics and evidence

Prerequisites Modules 06 and 08.2.

Four lessons

09.1 Probability and conditional reasoning

Model uncertainty without confusing independent and conditional events.

09.2 Random variables, distributions and sampling

Distinguish an expected value from a guaranteed outcome.

09.3 Inference, uncertainty and benchmark design

Compare implementations without hiding variance or bias.

09.4 Regression, correlation and ethical telemetry

Fit a model while recognizing confounding and data limits.

Module project

Create a reproducible benchmark experiment and a loot-distribution simulation. Report assumptions, uncertainty, representative workloads and one result that did not support the original hypothesis.

Core references

R09 MIT OpenCourseWare — 18.05 Introduction to Probability and Statistics

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 09.1 • MODULE 09

Probability and conditional reasoning

Learning goal Model uncertainty without confusing independent and conditional events.

Probability assigns weights to events under a model. Conditional probability restricts attention to cases where another event occurred. Independence means learning one event does not change the probability of the other; it is an assumption to justify, not a default for every repeated observation.

Bayes' rule reverses conditioning using prior probabilities and evidence likelihoods. Rare-event base rates matter when interpreting an apparently accurate detector. In games, these ideas apply to randomized systems, telemetry and testing. A measured frequency approaches a model probability only under suitable sampling assumptions; a small sample can be very noisy.

WORKED EXAMPLE

For independent fair coin flips, the probability of two heads is 1/4. If the second outcome is deliberately copied from the first, the probability is 1/2, demonstrating why dependence changes the calculation.

PRACTICE + PASS CRITERION

Simulate independent and dependent binary events. Build a small contingency table for a hypothetical bug detector. Pass: calculate both P(alert|bug) and P(bug|alert) and explain why they differ.

ASSIGNED STUDY

R09: probability, conditional probability and Bayes' theorem.

CHECK YOUR UNDERSTANDING

1. What is the probability of two independent events with probabilities 0.5 and 0.2 both occurring?
2. Is P(A|B) generally equal to P(B|A)?
3. Why do rare-event base rates matter for an alert?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 09.2 • MODULE 09

Random variables, distributions and sampling

Learning goal Distinguish an expected value from a guaranteed outcome.

A random variable maps outcomes to numbers. Its expectation is a probability-weighted average; variance measures spread around that average. A distribution describes the possible values and their probabilities. Bernoulli, binomial and normal models have different assumptions and should not be chosen only because their names are familiar.

Pseudorandom generators are deterministic algorithms with internal state. A seed supports repeatable experiments, but a generator suitable for gameplay is not automatically suitable for security. Sampling can be biased by how data is collected. Use separate random streams when independent subsystems should not change each other's results merely by making extra draws.

WORKED EXAMPLE

A reward of 10 with probability 0.2 and 0 otherwise has expected value 2. A player still receives either 10 or 0 on one trial, not necessarily 2.

PRACTICE + PASS CRITERION

Implement a weighted reward sampler and compare observed frequencies to the intended distribution over several sample sizes. Pass: record seed, generator, counts and expected values; explain finite-sample variation.

ASSIGNED STUDY

R09: random variables, expectation and variance; R04: random reference.

CHECK YOUR UNDERSTANDING

1. What is the expectation of 20 with probability 0.25 and 0 otherwise?

2. Does an expectation of 5 guarantee a reward of 5 on a trial?

3. Why separate random streams for unrelated systems?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 09.3 • MODULE 09

Inference, uncertainty and benchmark design

Learning goal Compare implementations without hiding variance or bias.

A sample statistic estimates a property of a larger process. Confidence intervals express uncertainty under a sampling model; they are not a probability that a fixed frequentist parameter lies in a specific realized interval. Hypothesis tests require defined assumptions and do not measure practical importance by themselves.

For performance work, control workload, build mode, warmup and environmental changes. Use repeated independent runs where possible; adjacent frames are often correlated. Report distributions and tail latency rather than only an average. A statistically detectable difference may be too small to matter, while a rare long pause can be important despite little change in the mean.

WORKED EXAMPLE

Implementation A might average 5 ms but occasionally take 100 ms; B might average 6 ms with no large pauses in the tested sample. The preferred design depends on throughput and latency requirements, not the mean alone.

PRACTICE + PASS CRITERION

Compare two implementations using the same inputs, randomized test order and repeated runs. Pass: report median, a tail percentile, sample count and limitations; distinguish observations from causal claims.

ASSIGNED STUDY

R09: confidence intervals, hypothesis testing and experimental reasoning.

CHECK YOUR UNDERSTANDING

1. Why can adjacent frame times violate an independence assumption?

2. Does a small p-value establish a large useful improvement?

3. Why report a tail percentile alongside a mean?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 09.4 • MODULE 09

Regression, correlation and ethical telemetry

Learning goal Fit a model while recognizing confounding and data limits.

Regression relates an outcome to predictors under a chosen model. Correlation is association, not proof of causation. A confounder can influence both the apparent cause and the outcome. Models should be evaluated on held-out observations or a suitable validation scheme, not only on the data used to fit them.

Telemetry collection should have a defined purpose and minimize unnecessary personal data. For course projects, use synthetic or consented data rather than harvesting real players. Do not optimize only the easiest-to-measure metric. Session length, for example, is not equivalent to enjoyment, accessibility or wellbeing.

WORKED EXAMPLE

Players with better hardware may both buy more games and achieve higher frame rates. Observing a purchase/frame-rate correlation does not establish that purchases caused faster rendering.

PRACTICE + PASS CRITERION

Fit a simple performance model using entity count and measured update time. Hold out some workloads. Pass: report prediction error and one confounder; write a short data-minimization plan for the experiment.

ASSIGNED STUDY

R09: regression and inference; R01: ethics and data-management breadth.

CHECK YOUR UNDERSTANDING

1. Why is training-set fit alone insufficient?

2. What is a confounder?

3. Why should telemetry collect only what the question requires?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 10

Signals, optimization and numerical foundations

Prerequisites Modules 07-09.

Four lessons

10.1 Multivariable calculus and gradients

Use partial derivatives to reason about spatial fields and optimization.

10.2 Sampling, aliasing and Fourier ideas

Explain why discrete samples can misrepresent a continuous signal.

10.3 Convolution, filters and systems

Predict a filter's effect on data and latency.

10.4 Optimization, conditioning and approximation budgets

Choose an objective and distinguish convergence from correctness.

Module project

Complete Gate B: a mathematical engineering notebook plus tested transforms, numerical integration, signal filtering and an uncertainty-aware experiment.

Core references

R07 MIT OpenCourseWare — 18.06 Linear Algebra

R10 MIT OpenCourseWare — 6.003 Signals and Systems

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 10.1 • MODULE 10

Multivariable calculus and gradients

Learning goal Use partial derivatives to reason about spatial fields and optimization.

A multivariable function depends on several inputs. A partial derivative varies one input while holding the others fixed. The gradient collects partial derivatives and points toward steepest local increase under the usual Euclidean metric. The Jacobian describes the local linear behavior of a vector-valued function.

Gradients support terrain normals, inverse kinematics and parameter fitting. A gradient is local information, not a guarantee of the global best direction over a large step. Chain-rule reasoning must follow every transformation. Numerical derivatives are useful checks for analytic code, but choose perturbations that account for scale and rounding.

WORKED EXAMPLE

For f(x,y)=x*x+y*y, the gradient is (2*x,2*y). At (3,4), it is (6,8). Taking a small step in the negative gradient direction reduces f locally.

PRACTICE + PASS CRITERION

Derive gradients for three functions and check them with centered differences. Pass: test nonzero and stationary points, vary perturbation size and report errors rather than claiming exact equality.

ASSIGNED STUDY

R07: linearization foundations; R23: geometric differential quantities; use the program's multivariable extension reading.

CHECK YOUR UNDERSTANDING

1. What is the gradient of x*x + 3*y*y?

2. What does a Jacobian describe?

3. Does a zero gradient always identify a minimum?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 10.2 • MODULE 10

Sampling, aliasing and Fourier ideas

Learning goal Explain why discrete samples can misrepresent a continuous signal.

Sampling records a signal at discrete positions or times. Different high-frequency signals can produce the same samples, creating aliasing. Reconstruction needs assumptions about bandwidth and a suitable filter. For a strictly band-limited signal, the familiar sampling criterion requires a rate greater than twice its highest frequency; practical systems need transition bands and margin.

The Fourier viewpoint decomposes signals into frequency components. It connects audio, textures, animation and display artifacts. Increasing resolution helps only relative to the signal content. Filtering before downsampling removes frequencies that the lower rate cannot represent; filtering afterward cannot generally recover information already aliased.

WORKED EXAMPLE

A 900 Hz tone sampled at 1000 samples/second can appear as a 100 Hz tone. The samples do not uniquely preserve the original high-frequency oscillation.

PRACTICE + PASS CRITERION

Generate low- and high-frequency sine samples, then downsample a checker pattern with and without prefiltering. Pass: explain the artifacts using frequency content rather than calling them random noise.

ASSIGNED STUDY

R10: sampling and Fourier representation; R23: Sampling and Reconstruction.

CHECK YOUR UNDERSTANDING

1. What is the Nyquist frequency at 48,000 samples/second?

2. Why filter before downsampling?

3. Can post-filtering always undo aliasing?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 10.3 • MODULE 10

Convolution, filters and systems

Learning goal Predict a filter's effect on data and latency.

Convolution combines a signal with a kernel by weighted summation of shifted values. A moving average smooths high-frequency variation, but it also changes edges and can introduce delay depending on implementation. A linear time-invariant system is characterized by its impulse response under the model assumptions.

Boundary handling is part of a filter definition. Zero padding, wrapping and edge replication produce different results. Filters also have numeric and performance costs. In audio, a stable and bounded processing path is essential; in images, color-space assumptions matter when averaging values. Test an impulse and a constant signal before trusting a complex input.

WORKED EXAMPLE

A three-sample average of [3,6,9] is 6. Sliding this kernel across a sequence produces a smoothed sequence; values near either end require an explicit boundary policy.

PRACTICE + PASS CRITERION

Implement a one-dimensional convolution with documented boundary handling. Test an impulse, a constant and a step. Pass: predict the output before running and identify the filter's delay convention.

ASSIGNED STUDY

R10: convolution, impulse response and linear time-invariant systems.

CHECK YOUR UNDERSTANDING

1. What does convolving with an impulse reveal?

2. What is the average of 2, 5 and 8?

3. Why is boundary handling not an implementation footnote?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 10.4 • MODULE 10

Optimization, conditioning and approximation budgets

Learning goal Choose an objective and distinguish convergence from correctness.

Optimization minimizes or maximizes a specified objective under constraints. Gradient descent takes steps opposite the gradient; its behavior depends on step size, scaling and the shape of the objective. A local optimum need not be global. Constraints can be as important as the objective, such as keeping error below a visible threshold while reducing work.

Numerical engineering combines error sources: model approximations, finite precision, sampling and iterative termination. Define an error budget and a reference case. More iterations can reduce one error while leaving others unchanged. Stop according to a justified criterion and report failure to converge rather than silently returning a plausible number.

WORKED EXAMPLE

For f(x)=(x-3)^2, gradient descent uses x_next=x-alpha*2*(x-3). An excessively large alpha can oscillate or diverge even though the function has a simple minimum.

PRACTICE + PASS CRITERION

Implement gradient descent on a quadratic and a two-variable problem. Pass: compare step sizes, record residuals and state a stopping rule; complete Gate B with no unsupported claim of global optimality.

ASSIGNED STUDY

R07: least squares and conditioning; R23: numerical utilities; graduate optimization reading in the handbook.

CHECK YOUR UNDERSTANDING

1. Where is the minimum of (x-3)^2?

2. Why can reducing step size help an unstable iteration?

3. What should happen when an iterative method reaches its iteration limit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 03 OF 10

C++, Algorithms and Language Implementation

Undergraduate CS core

Write native software safely, select data structures, analyze algorithms and build a tiny language.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 11 Native programming with C++

11.1 Compilation, linking and build systems • 11.2 Integer, floating-point and object representations • 11.3 Values, references, pointers and lifetimes • 11.4 Standard containers, iterators and error handling

Module 12 Ownership, abstractions and C++ design

12.1 RAII and resource ownership • 12.2 Copy, move and exception safety • 12.3 Interfaces, composition and polymorphism • 12.4 Templates, concepts and data-oriented choices

Module 13 Data structures and their costs

13.1 Complexity, arrays and linked structures • 13.2 Stacks, queues and ring buffers • 13.3 Hash tables and collision policies • 13.4 Trees, heaps and priority queues

Module 14 Algorithms for maps, simulation and tools

14.1 Searching, sorting and algorithm selection • 14.2 Graph traversal and dependency ordering • 14.3 Dijkstra, A* and admissible heuristics • 14.4 Dynamic programming, complexity limits and approximation

Module 15 Programming languages, compilers and computability

15.1 Language paradigms and type systems • 15.2 Lexing, parsing and syntax trees • 15.3 Bytecode, virtual machines and bounded execution • 15.4 Automata, computability and compiler optimization

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 11

Native programming with C++

Prerequisites Volume 01, Modules 06-07; do not skip the ownership module that follows.

Four lessons

11.1 Compilation, linking and build systems

Explain how source code becomes a runnable native program.

11.2 Integer, floating-point and object representations

Choose native types with explicit range and precision assumptions.

11.3 Values, references, pointers and lifetimes

Track where an object lives and how long access remains valid.

11.4 Standard containers, iterators and error handling

Use library abstractions while respecting invalidation and failure contracts.

Module project

Port the deterministic grid simulation to C++20, with a command-line build, automated tests, warnings enabled and an explicit account of representation differences from Python.

Core references

R11 C++ Core Guidelines editors — C++ Core Guidelines

R12 Kitware — CMake tutorial

R15 MIT OpenCourseWare — 6.004 Computation Structures

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 11.1 • MODULE 11

Compilation, linking and build systems

Learning goal Explain how source code becomes a runnable native program.

A C++ translation unit is compiled into object code; linking resolves references and combines the required pieces into a program or library. Headers usually provide declarations shared across translation units. A declaration describes an entity; a definition supplies its implementation or storage. Build errors therefore occur at different stages and require different evidence.

A build system records dependencies and compiler settings. Use a small CMake target rather than hardcoding paths throughout source files. Keep build products outside the source tree. Debug and optimized builds can differ substantially, so record which configuration produced a measurement. For this course, C++20 is a selected teaching baseline, not a claim that every feature is supported by every compiler.

WORKED EXAMPLE

A function can compile at a call site when its declaration is visible yet fail to link when no matching definition is supplied. A missing header usually fails earlier during compilation.

PRACTICE + PASS CRITERION

Build the included C++ starter, then split one function into a header and source file. Pass: reproduce and explain one compile error and one link error, then restore a warning-clean build.

ASSIGNED STUDY

R12: Getting Started and libraries; R11: Source Files.

CHECK YOUR UNDERSTANDING

1. What task does the linker perform?

2. Can valid declarations alone supply every function implementation?

3. Why record build configuration with a benchmark?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 11.2 • MODULE 11

Integer, floating-point and object representations

Learning goal Choose native types with explicit range and precision assumptions.

Native representations have finite range and precision. Do not assume int is a particular width in every environment. Fixed-width integer types are available when the implementation provides them; use them where a format or interface requires that width. Unsigned arithmetic wraps modulo its range, while signed integer overflow in ordinary C++ arithmetic is undefined behavior.

Floating-point operations approximate real arithmetic and have special values such as infinity and NaN on common IEEE-style implementations. Object layout can contain padding, so raw struct bytes are not a portable save or network format. Byte order, alignment and field encoding require explicit decisions. An apparently correct result on one machine does not establish portability.

WORKED EXAMPLE

An unsigned 8-bit value has 256 distinct values, 0 through 255. A 32-bit byte count can be too small for a large asset. Converting a negative signed value to an unsigned type may produce a very large number.

PRACTICE + PASS CRITERION

Print relevant sizes and numeric limits on the available compiler. Write checked conversions for a file length and tests for boundary values. Pass: do not trigger signed overflow as a test oracle; reject out-of-range input before converting.

ASSIGNED STUDY

R11: arithmetic and conversion guidance; R15: information encoding and arithmetic.

CHECK YOUR UNDERSTANDING

1. How many distinct bit patterns fit in eight bits?

2. Is ordinary signed integer overflow defined to wrap in C++?

3. Why is writing raw struct bytes not a portable format?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 11.3 • MODULE 11

Values, references, pointers and lifetimes

Learning goal Track where an object lives and how long access remains valid.

A value is an object's stored information. A reference aliases an existing object; a pointer stores an address-like value and can represent no object with nullptr. Neither a raw pointer nor a reference automatically owns what it refers to. Ownership determines who must release a resource; lifetime determines when the object exists.

Automatic storage duration often corresponds to stack allocation, but storage-duration rules are the reliable language concept. Returning a pointer or reference to a local object leaves a dangling access after that object's lifetime ends. Const expresses permitted modification through an interface, not universal thread safety or deep immutability. Draw ownership and lifetime diagrams before clever pointer code.

WORKED EXAMPLE

Passing const State& can avoid copying while allowing observation of a state owned by the caller. Returning a State by value can be safe and efficient; do not assume every value return causes an expensive extra copy.

PRACTICE + PASS CRITERION

Implement functions taking values, const references and pointers with a documented null policy. Pass: explain each lifetime and identify a deliberately supplied dangling-reference example without executing undefined behavior.

ASSIGNED STUDY

R11: Interfaces, Resource Management and lifetime safety guidance.

CHECK YOUR UNDERSTANDING

1. Does a raw pointer necessarily own its target?

2. Why is returning a reference to a local variable invalid?

3. Does const by itself make shared access thread-safe?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 11.4 • MODULE 11

Standard containers, iterators and error handling

Learning goal Use library abstractions while respecting invalidation and failure contracts.

Standard containers provide reusable storage and algorithms. std::vector offers contiguous elements and amortized growth behavior, but reallocation can invalidate pointers, references and iterators into its storage. Size is the number of live elements; capacity is storage reserved for possible growth. Reserving capacity does not construct that many elements.

Use standard algorithms when they make intent clearer. Check container preconditions and distinguish exceptional failure from ordinary absence. An optional value, status type or exception may each be appropriate under a documented policy. Do not catch every error and pretend success; preserve useful diagnostic context and avoid leaving partially updated state.

WORKED EXAMPLE

For a vector with size 3 and capacity 8, valid element indices are still 0, 1 and 2. Accessing index 7 is not allowed merely because storage has been reserved.

PRACTICE + PASS CRITERION

Port the unit list to std::vector and implement lookup with an explicit not-found result. Pass: test empty input, missing ID and growth; document which operations invalidate stored references.

ASSIGNED STUDY

R11: standard-library use and error handling; R12: testing integration.

CHECK YOUR UNDERSTANDING

1. Does reserve(100) make a vector's size 100?

2. Why can a pointer to a vector element become invalid after growth?

3. What is wrong with ignoring every exception?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 12

Ownership, abstractions and C++ design

Prerequisites Module 11.

Four lessons

12.1 RAII and resource ownership

Make cleanup follow object lifetime on success and failure paths.

12.2 Copy, move and exception safety

Define what transferring a resource means without duplicating ownership.

12.3 Interfaces, composition and polymorphism

Choose abstractions according to substitutability and measured cost.

12.4 Templates, concepts and data-oriented choices

Use generic code without hiding costs or exploding complexity.

Module project

Build a small resource manager with explicit ownership, safe move behavior, bounded interfaces, fault-injection tests and a written comparison of composition and inheritance.

Core references

R11 C++ Core Guidelines editors — C++ Core Guidelines

R12 Kitware — CMake tutorial

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 12.1 • MODULE 12

RAII and resource ownership

Learning goal Make cleanup follow object lifetime on success and failure paths.

Resource acquisition is initialization, or RAII, ties a resource to an owning object whose destructor releases it. The resource can be memory, a file, a mutex guard or a graphics handle. This makes cleanup occur on normal scope exit and during supported exception unwinding. Ownership should be visible in types rather than scattered comments.

Prefer unique ownership when one component controls lifetime. Shared ownership is useful for genuinely shared lifetimes but adds costs and can form cycles. Non-owning views must not outlive their owner. Wrapping an operating-system or GPU handle requires the correct release operation and context; a destructor is not magic if it runs after the device is gone.

WORKED EXAMPLE

A unique_ptr owns one object and releases it automatically. Passing a raw pointer to inspect that object can be fine when the caller guarantees the owner remains alive throughout the call.

PRACTICE + PASS CRITERION

Wrap a mock file/device resource whose open/close counters can be inspected. Inject failure after acquisition. Pass: each acquired resource is released exactly once and no view outlives its owner.

ASSIGNED STUDY

R11: Resource Management and smart-pointer ownership rules.

CHECK YOUR UNDERSTANDING

1. What does RAII connect?

2. Can shared ownership create a leak through cycles?

3. Why must device-resource destruction order be planned?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 12.2 • MODULE 12

Copy, move and exception safety

Learning goal Define what transferring a resource means without duplicating ownership.

Copying creates an independent value according to a type's contract. Moving transfers or reuses resources where appropriate; it is not merely a guaranteed byte copy. A moved-from standard-library object remains valid but may have an unspecified value unless its specific contract says more. std::move is a cast enabling move-aware overload resolution, not an operation that itself moves bytes.

Exception safety asks what remains true if an operation fails. A basic guarantee preserves validity and prevents leaks; a strong guarantee leaves observable state unchanged on failure. Build a replacement first and commit only after success when that guarantee is important. Prefer the rule of zero: compose well-behaved members rather than writing custom special-member functions unnecessarily.

WORKED EXAMPLE

To replace a loaded asset, validate a new asset before swapping it into the live slot. A failed parse then leaves the old working asset available instead of an empty or half-filled replacement.

PRACTICE + PASS CRITERION

Implement a move-only mock resource and transactional asset replacement. Pass: copying is prohibited, moving does not double-release, and injected load failure preserves the old asset.

ASSIGNED STUDY

R11: class special members, rule of zero and exception safety.

CHECK YOUR UNDERSTANDING

1. Does std::move itself guarantee that an object is moved?

2. What is the strong exception guarantee?

3. Why prefer the rule of zero?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 12.3 • MODULE 12

Interfaces, composition and polymorphism

Learning goal Choose abstractions according to substitutability and measured cost.

An interface should expose a coherent capability with a clear contract. Runtime polymorphism chooses behavior through a dynamic type, often via virtual functions. Static polymorphism resolves behavior through compile-time types. Neither is always faster or better; ownership, extensibility, binary boundaries and workload matter.

Composition keeps independent capabilities separate. An inheritance hierarchy is appropriate when derived objects can honor the base contract. A base class deleted polymorphically normally needs a virtual destructor. Avoid embedding every engine subsystem in one universal object. Small explicit interfaces make tests, replacement implementations and dependency analysis easier.

WORKED EXAMPLE

A Renderer interface might accept a scene description and produce a frame. A test renderer can record calls without drawing. It should not need to own gameplay rules or the network session.

PRACTICE + PASS CRITERION

Create interchangeable console and recording renderers behind one small interface. Pass: both satisfy the same contract, tests run without display output, and ownership/destructor behavior is explicit.

ASSIGNED STUDY

R11: Interfaces, Class Hierarchies and composition guidance.

CHECK YOUR UNDERSTANDING

1. What does substitutability require?

2. Why can a polymorphic base need a virtual destructor?

3. Why not put every responsibility into one engine class?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 12.4 • MODULE 12

Templates, concepts and data-oriented choices

Learning goal Use generic code without hiding costs or exploding complexity.

Templates express algorithms and types parameterized by compile-time information. Concepts constrain acceptable operations and improve interface meaning. Generic code still needs tests for relevant instantiations and clear compilation errors. Excessive instantiation can increase build times and binary size, so abstraction costs include developer time as well as runtime.

Data-oriented design starts from access patterns: which fields are processed together, how often, and in what order? Array-of-structures and structure-of-arrays layouts trade convenience, locality and vectorization opportunities. Do not adopt either as doctrine. Measure a representative loop and preserve correctness while comparing designs.

WORKED EXAMPLE

If a movement pass reads only positions and velocities, a structure-of-arrays layout can avoid fetching unrelated names and inventory fields. If each operation needs the complete object, that advantage may shrink or reverse.

PRACTICE + PASS CRITERION

Write a constrained generic clamp or interpolation function and benchmark two simple entity layouts. Pass: test contracts, record compiler/build mode and explain why the result is workload-specific.

ASSIGNED STUDY

R11: Templates and Generic Programming; R15: memory hierarchy motivation.

CHECK YOUR UNDERSTANDING

1. What does a concept constrain?

2. Is structure-of-arrays universally superior?

3. Name a cost of templates beyond runtime.

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 13

Data structures and their costs

Prerequisites Modules 06 and 11-12.

Four lessons

13.1 Complexity, arrays and linked structures

Predict asymptotic work while recognizing locality and constants.

13.2 Stacks, queues and ring buffers

Match ordering semantics to the intended workflow.

13.3 Hash tables and collision policies

Explain expected lookup behavior and its assumptions.

13.4 Trees, heaps and priority queues

Preserve structural invariants and choose the right query behavior.

Module project

Implement educational versions of a dynamic array, queue, hash table and heap, each with invariants and tests. Compare them to standard containers without replacing standard code in production merely for practice.

Core references

R13 MIT OpenCourseWare — 6.006 Introduction to Algorithms

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 13.1 • MODULE 13

Complexity, arrays and linked structures

Learning goal Predict asymptotic work while recognizing locality and constants.

Asymptotic notation describes how cost grows with input size under a stated model. Big-O is an upper bound, not an exact time or a synonym for worst case in every context. Arrays provide indexed access and contiguous storage. Linked structures connect separately located nodes and can support local insertion when the location is already known.

Searching for that location still costs work. Pointer chasing and allocation affect real performance beyond operation counts. A dynamic array can grow geometrically to achieve amortized constant-time append, even though some individual appends require linear relocation. Amortized analysis concerns a sequence of operations, not an average over random inputs.

WORKED EXAMPLE

Doubling capacity after each expansion makes total copied elements across n appends proportional to n. Growing capacity by one each time can copy approximately 1+2+...+(n-1), which is quadratic.

PRACTICE + PASS CRITERION

Implement a growable integer buffer for study. Count copied elements under doubling and one-at-a-time growth. Pass: derive the growth behavior and separately measure elapsed time without equating it to the proof.

ASSIGNED STUDY

R13: arrays, cost models and asymptotic analysis; R06: sums.

CHECK YOUR UNDERSTANDING

1. Does O(n) specify an exact runtime?

2. What is the total 1+...+(n-1) proportional to?

3. How does amortized analysis differ from random-input average-case analysis?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 13.2 • MODULE 13

Stacks, queues and ring buffers

Learning goal Match ordering semantics to the intended workflow.

A stack is last-in, first-out; a queue is first-in, first-out. Stacks support nested calls and undo-like history; queues support pending work and messages. A ring buffer reuses a fixed storage region by wrapping indices. It needs an unambiguous policy to distinguish empty from full.

Bounded structures force an overload decision: reject, overwrite, block or drop according to requirements. Silently overwriting input commands may be unacceptable even if overwriting old diagnostic samples is permitted. A single-threaded ring buffer is not automatically safe for concurrent producers and consumers; synchronization is a separate lesson.

WORKED EXAMPLE

Enqueue A, B and C into a FIFO queue, then dequeue twice: A and B leave first. Push the same values onto a stack, then pop twice: C and B leave first.

PRACTICE + PASS CRITERION

Implement a bounded queue with explicit full/empty results and wraparound tests. Pass: preserve FIFO order through more than three times its capacity in operations and never read an uninitialized slot.

ASSIGNED STUDY

R13: elementary data structures; R16: later concurrency comparison.

CHECK YOUR UNDERSTANDING

1. Which item leaves first after pushing A then B onto a stack?

2. Why does a ring buffer need a full/empty convention?

3. Does a correct single-threaded queue imply a thread-safe queue?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 13.3 • MODULE 13

Hash tables and collision policies

Learning goal Explain expected lookup behavior and its assumptions.

A hash table maps keys to buckets using a hash function. Different keys can share a bucket; collision handling is therefore required, not optional. Equality decides whether keys are actually the same. Load factor measures occupancy and affects performance according to the collision strategy.

Expected constant-time lookup relies on assumptions about hashing, load and inputs. Worst-case behavior can be much worse. Rehashing can invalidate references according to a container's contract. For untrusted inputs, adversarial collisions may matter. Hash values are not unique identifiers, encryption or proof that two objects are equal.

WORKED EXAMPLE

Keys 12 and 22 both map to bucket 2 under key modulo 10. A correct table retains both and compares actual keys during lookup.

PRACTICE + PASS CRITERION

Implement chaining or open addressing with collision tests and resizing. Pass: deliberately force many collisions, test missing keys and verify that distinct equal-hash keys remain distinct.

ASSIGNED STUDY

R13: hashing and dictionary data structures.

CHECK YOUR UNDERSTANDING

1. Does a matching hash prove two keys are equal?

2. What is the load factor for 30 entries in 10 buckets?

3. Why is expected O(1) not an unconditional guarantee?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 13.4 • MODULE 13

Trees, heaps and priority queues

Learning goal Preserve structural invariants and choose the right query behavior.

A binary search tree orders keys so that search can follow comparisons down a path. Its height determines many operation costs; an unbalanced tree can degrade to a chain. A heap maintains a different invariant: each parent has priority no worse than its children. It efficiently exposes an extreme element but does not globally sort all positions.

Priority queues are central to scheduling and pathfinding. Updating priorities requires a supported policy, such as handles, indexed heaps or inserting a new entry and ignoring stale ones. State the ordering and tie-break rules so tests and replays behave consistently.

WORKED EXAMPLE

A min-heap containing 2, 5 and 9 must expose 2 at the root, but its internal array is not promised to be a sorted sequence. A sorted insertion stream can make a naive search tree height n.

PRACTICE + PASS CRITERION

Implement a binary min-heap with push/pop and randomized comparison to a sorted reference. Pass: validate the heap invariant after every operation and test duplicate priorities.

ASSIGNED STUDY

R13: binary search trees, balanced trees and heaps.

CHECK YOUR UNDERSTANDING

1. Does a heap provide a fully sorted array?

2. What can the height of an unbalanced n-node search tree become?

3. Why define a tie-break rule in a priority queue?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 14

Algorithms for maps, simulation and tools

Prerequisites Modules 06, 09 and 13.

Four lessons

14.1 Searching, sorting and algorithm selection

Choose an algorithm from its preconditions and the data distribution.

14.2 Graph traversal and dependency ordering

Use graph algorithms with explicit edge semantics.

14.3 Dijkstra, A* and admissible heuristics

Build a correct pathfinder before optimizing its queue.

14.4 Dynamic programming, complexity limits and approximation

Recognize repeated subproblems and justify a practical solution.

Module project

Build a pathfinding visualizer and asset-dependency planner. Provide correctness arguments, unreachable cases, complexity analysis and measured comparisons on multiple map families.

Core references

R13 MIT OpenCourseWare — 6.006 Introduction to Algorithms

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 14.1 • MODULE 14

Searching, sorting and algorithm selection

Learning goal Choose an algorithm from its preconditions and the data distribution.

Binary search discards part of an ordered range after each comparison. Its invariant must preserve where a possible answer can remain. Sorting establishes order but incurs a cost; whether sorting first is worthwhile depends on how many later queries will reuse it. Stable sorting preserves relative order of equivalent keys.

Comparison sorting has well-known lower bounds under its model, while specialized methods can exploit bounded integer keys or other structure. Algorithm selection also considers memory, input size, nearly sorted data and worst-case behavior. Never apply a performance claim without its assumptions.

WORKED EXAMPLE

For 1024 ordered elements, binary search takes roughly ten halving decisions to narrow to one candidate, with exact comparisons depending on the implementation. An unsorted list does not satisfy the required invariant.

PRACTICE + PASS CRITERION

Implement binary search with a documented interval convention and test empty, one-element, duplicate and missing cases. Compare two sorts on random and nearly sorted inputs. Pass: verify output order and element preservation.

ASSIGNED STUDY

R13: binary search, sorting and comparison models.

CHECK YOUR UNDERSTANDING

1. What precondition does ordinary binary search require?

2. What does stable sorting preserve?

3. Why test element preservation as well as sortedness?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 14.2 • MODULE 14

Graph traversal and dependency ordering

Learning goal Use graph algorithms with explicit edge semantics.

Breadth-first search explores an unweighted graph by distance in edges; depth-first search follows a path before backtracking. Both need visited-state handling to avoid repeated traversal in cyclic graphs. A topological ordering exists only for a directed acyclic graph and places prerequisites before their dependents under the chosen edge convention.

Graph representation affects cost. Adjacency lists suit many sparse graphs; adjacency matrices can simplify dense or constant-time edge-existence queries at greater storage cost. A navigation graph is a model of movement, not the world itself: missing edges, one-way passages and dynamic obstacles change the problem.

WORKED EXAMPLE

With edges A->B and B->C meaning "must precede," A,B,C is a valid topological order. Adding C->A creates a cycle, so no such order exists.

PRACTICE + PASS CRITERION

Implement BFS and topological sorting. Pass: recover a shortest unweighted path, report unreachable goals, reject a cyclic dependency graph and justify the O(V+E) traversal bound for adjacency lists.

ASSIGNED STUDY

R13: graph representations, BFS, DFS and topological sorting.

CHECK YOUR UNDERSTANDING

1. When does BFS find a shortest path by edge count?

2. Can a directed graph with a cycle have a topological order?

3. Why mark visited nodes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 14.3 • MODULE 14

Dijkstra, A* and admissible heuristics

Learning goal Build a correct pathfinder before optimizing its queue.

Dijkstra's algorithm finds shortest paths with nonnegative edge costs. A* adds a heuristic estimate of remaining cost, prioritizing f=g+h. An admissible heuristic never overestimates the true remaining cost. A consistent heuristic also respects a triangle-like condition across edges.

Implementation details matter: closed-set policies, stale queue entries and reopening nodes affect correctness when a heuristic is not consistent. Negative edge costs invalidate ordinary Dijkstra assumptions. For a four-neighbor unit-cost grid, Manhattan distance is a useful lower bound; diagonal motion or terrain costs require reconsidering the model. Faster search that returns an invalid route is not an improvement.

WORKED EXAMPLE

From (1,2) to (4,6), Manhattan distance is |4-1|+|6-2|=7 on a four-neighbor unit-cost grid. Allowing inexpensive diagonals changes the appropriate estimate.

PRACTICE + PASS CRITERION

Implement Dijkstra and A* against the same grid model. Pass: compare path costs on many small maps, include no-path cases and demonstrate a deliberately overestimating heuristic that can sacrifice optimality.

ASSIGNED STUDY

R13: shortest paths; R20: game navigation context.

CHECK YOUR UNDERSTANDING

1. What edge-cost condition does ordinary Dijkstra require?

2. What is Manhattan distance from (0,0) to (3,4)?

3. Why can inconsistent heuristics require reopening nodes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 14.4 • MODULE 14

Dynamic programming, complexity limits and approximation

Learning goal Recognize repeated subproblems and justify a practical solution.

Dynamic programming stores solutions to overlapping subproblems under a recurrence. The difficult step is defining sufficient state and a valid dependency order. Memoization evaluates on demand; tabulation fills an ordered table. More dimensions can make a state space explode even when each transition is simple.

Complexity classes describe families of problems and reductions under formal models. NP-complete does not mean impossible on every instance, and a polynomial algorithm is not automatically fast enough in practice. Approximation, heuristics and bounded search can be appropriate when their guarantees and limits are explicit. Do not label an algorithm optimal merely because it found a good result once.

WORKED EXAMPLE

A budgeted loadout problem can use state (item_prefix, remaining_budget). Remembering only remaining_budget loses which items remain available and can produce an invalid recurrence.

PRACTICE + PASS CRITERION

Solve a small 0/1 knapsack instance by exhaustive search and dynamic programming. Pass: compare answers, derive state count, and describe how a heuristic would be evaluated without claiming a guarantee it lacks.

ASSIGNED STUDY

R13: dynamic programming; R06: reductions and asymptotic reasoning.

CHECK YOUR UNDERSTANDING

1. What two features motivate dynamic programming?

2. Does NP-complete mean no instance can be solved quickly?

3. Why keep a brute-force solver for tiny inputs?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 15

Programming languages, compilers and computability

Prerequisites Modules 11-14 and discrete proofs.

Four lessons

15.1 Language paradigms and type systems

Separate language semantics from syntax and implementation habits.

15.2 Lexing, parsing and syntax trees

Convert source text into a structured representation with diagnostics.

15.3 Bytecode, virtual machines and bounded execution

Implement a small execution model with explicit limits.

15.4 Automata, computability and compiler optimization

Know what analysis can establish and when transformation is legal.

Module project

Complete Gate C: a tested C++ simulation plus a tiny expression language with lexer, parser, evaluator, bytecode execution and resource limits.

Core references

R14 LLVM Project — Kaleidoscope and LLVM tutorials

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

R11 C++ Core Guidelines editors — C++ Core Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 15.1 • MODULE 15

Language paradigms and type systems

Learning goal Separate language semantics from syntax and implementation habits.

Imperative code describes updates; functional style emphasizes expression evaluation and controlled side effects; object-oriented design organizes behavior through objects and interfaces. Real languages combine paradigms. A type system rejects certain invalid combinations according to its rules, but no ordinary type system proves all game requirements.

Static versus dynamic typing concerns when and how types are checked, not a simple ranking of language quality. Ownership-aware languages offer useful comparisons for systems design, while C remains important at binary interfaces and embedded boundaries. Learn enough C syntax and layout rules to read those interfaces, then preserve explicit ownership when calling them from C++.

WORKED EXAMPLE

A pure damage calculation can be tested independently of an imperative combat loop. The two styles cooperate: the loop manages time and effects, while the function computes a value.

PRACTICE + PASS CRITERION

Write one rule in imperative and pure-functional styles, then define a small typed C-compatible interface for a mock subsystem. Pass: state allocation ownership and reject type-invalid expressions in a toy checker.

ASSIGNED STUDY

R14: language frontend concepts; R11: interfaces and C-style boundary guidance.

CHECK YOUR UNDERSTANDING

1. Does static typing prove all logical requirements?

2. What advantage does a pure calculation offer tests?

3. What must a C-compatible ownership boundary specify?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 15.2 • MODULE 15

Lexing, parsing and syntax trees

Learning goal Convert source text into a structured representation with diagnostics.

A lexer groups characters into tokens such as numbers and operators. A parser applies a grammar to produce a syntax tree. Precedence and associativity determine structure: multiplication normally binds more tightly than addition, and subtraction usually groups left-to-right. An evaluator should operate on that structure rather than repeatedly guessing meaning from raw text.

Parsing failures need locations and useful messages. An input can be syntactically valid but semantically invalid, such as referring to an unknown variable. Keep these phases separate enough to test. Never implement a game-data parser by unrestricted execution of input text.

WORKED EXAMPLE

For 2+3*4, the tree has addition at the root and multiplication on the right, yielding 14. Parsing it as (2+3)*4 would yield 20 and violate the chosen precedence.

PRACTICE + PASS CRITERION

Implement tokens, recursive-descent parsing and evaluation for numbers, parentheses and four arithmetic operators. Pass: test precedence, associativity, malformed input, unknown names and division-by-zero policy.

ASSIGNED STUDY

R14: Kaleidoscope lexer, parser and abstract syntax tree chapters.

CHECK YOUR UNDERSTANDING

1. What separates lexing from parsing?

2. Under ordinary precedence, what is 2+3*4?

3. Why preserve source locations in a syntax tree?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 15.3 • MODULE 15

Bytecode, virtual machines and bounded execution

Learning goal Implement a small execution model with explicit limits.

A virtual machine executes a defined instruction set in software. A stack machine can use instructions such as push, add and multiply. Compiling an expression tree to bytecode separates source syntax from execution. A VM still needs a specification: stack effects, value types, branch behavior and error conditions.

Scripts and mods must be treated as potentially faulty input. Bound instruction count, memory use and access to host capabilities. A VM is not automatically a secure sandbox; unsafe host functions can defeat isolation. Keep the educational VM offline and limited, and do not equate a few checks with production security review.

WORKED EXAMPLE

The expression 2+3*4 can become PUSH 2, PUSH 3, PUSH 4, MUL, ADD. MUL consumes two values and pushes one, so the stack height drops by one.

PRACTICE + PASS CRITERION

Compile the expression language into stack bytecode and compare results to the tree evaluator. Pass: reject stack underflow, invalid opcodes and over-budget execution; preserve source context for errors.

ASSIGNED STUDY

R14: code generation and execution concepts; course security requirements.

CHECK YOUR UNDERSTANDING

1. What is the stack-height change of a binary add that pushes one result?

2. Why compare VM output to the tree evaluator?

3. Does using bytecode automatically create a secure sandbox?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 15.4 • MODULE 15

Automata, computability and compiler optimization

Learning goal Know what analysis can establish and when transformation is legal.

Finite automata recognize patterns using bounded state. More expressive computation models explain parsing and general programs. Some questions about arbitrary programs, including universal termination, are undecidable. This does not prevent proving termination for restricted programs or enforcing execution budgets.

Compiler optimization must preserve the language's observable semantics under its assumptions. Constant folding, dead-code elimination and inlining are examples, but floating-point reassociation can change results and undefined behavior can invalidate a programmer's intuition. Inspect generated code to learn, not to assume one assembly listing predicts every target. Correctness precedes instruction-count reduction.

WORKED EXAMPLE

Replacing 2+3 by 5 is straightforward for the defined integer literals. Reassociating (a+b)+c into a+(b+c) can change finite-precision floating-point results when magnitudes differ greatly.

PRACTICE + PASS CRITERION

Add constant folding to the toy compiler and test it against the unoptimized evaluator. Pass: include a finite-state recognizer, explain a termination restriction and document transformations deliberately excluded for semantic reasons.

ASSIGNED STUDY

R14: optimizer support; R06: computation models and proof limitations.

CHECK YOUR UNDERSTANDING

1. Does undecidability mean no individual program can be proven to terminate?

2. Why can floating-point reassociation change a result?

3. What is the primary correctness condition for an optimization?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 04 OF 10

Computer Systems and Software Engineering

Intermediate undergraduate

Connect programs to processors, operating systems, concurrency, networks, storage and professional practice.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 16 Processor architecture and assembly

16.1 Instruction sets, registers and calling conventions • 16.2 Pipelines, dependencies and branch prediction • 16.3 Caches, locality and memory hierarchy • 16.4 Virtual memory, SIMD and architectural limits

Module 17 Operating systems and resource management

17.1 Processes, threads, system calls and protection • 17.2 Allocation, paging and memory-mapped data • 17.3 Scheduling, responsiveness and real-time constraints • 17.4 Filesystems, persistence and asynchronous I/O

Module 18 Concurrency and parallel programming

18.1 Races, mutexes and happens-before • 18.2 Condition variables, deadlocks and cancellation • 18.3 Atomics, memory ordering and false sharing • 18.4 Task graphs, work stealing and scaling limits

Module 19 Networks, storage and data services

19.1 Network layers, latency and transport semantics • 19.2 Protocol design, serialization and timeouts • 19.3 Relational modeling, SQL and indexes • 19.4 Transactions, consistency and distributed failure

Module 20 Engineering practice, security and maintainability

20.1 Requirements, architecture and design records • 20.2 Defensive programming and threat modeling • 20.3 Build automation, dependencies and release discipline • 20.4 Ethics, accessibility, teamwork and sustainable delivery

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 16

Processor architecture and assembly

Prerequisites Modules 11-15.

Four lessons

16.1 Instruction sets, registers and calling conventions

Read a small machine-level routine without confusing ISA and implementation.

16.2 Pipelines, dependencies and branch prediction

Explain why instruction count alone does not predict latency.

16.3 Caches, locality and memory hierarchy

Relate access patterns to cache lines and working sets.

16.4 Virtual memory, SIMD and architectural limits

Recognize address translation and vector execution as separate mechanisms.

Module project

Trace a compiled C++ loop through a small assembly example and a cache model. Deliver an instruction/data-flow explanation and a controlled locality experiment.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R35 RISC-V International — RISC-V ISA specifications

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 16.1 • MODULE 16

Instruction sets, registers and calling conventions

Learning goal Read a small machine-level routine without confusing ISA and implementation.

An instruction-set architecture defines visible operations, registers and relevant behavior. A microarchitecture implements that contract using particular pipelines, caches and execution units. Different implementations can run the same machine code while achieving different performance. A calling convention specifies how functions pass arguments, preserve registers and return values.

Begin with a small load/store ISA such as the selected RISC-V subset, then compare your host's generated assembly. Do not assume an educational subset implements every extension or privileged feature. Assembly understanding helps explain costs and interoperability, but handwritten assembly is rarely the first optimization step.

WORKED EXAMPLE

A load reads a value from memory into a register; an add combines register values; a store writes a result back. A C++ array loop may contain all three, plus loop-control instructions.

PRACTICE + PASS CRITERION

Annotate a ten-to-twenty-instruction routine with register meanings and memory accesses. Pass: predict its output for two inputs and identify caller/callee responsibilities from the chosen ABI reference.

ASSIGNED STUDY

R15: Assembly Language and Procedures and Stacks; R35: selected base integer ISA.

CHECK YOUR UNDERSTANDING

1. How does an ISA differ from a microarchitecture?

2. What does a calling convention specify?

3. Why identify the exact ISA subset in a simulator?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 16.2 • MODULE 16

Pipelines, dependencies and branch prediction

Learning goal Explain why instruction count alone does not predict latency.

A pipeline overlaps stages of multiple instructions. Dependencies can prevent later work from proceeding until earlier results are ready. Modern processors may issue or complete operations out of order while preserving required architectural behavior. Branch prediction speculates about control flow, and mispredictions can discard work.

Throughput measures sustained work per time; latency measures delay for a particular result. A long dependency chain can have poor latency even when the processor can execute many independent operations per cycle. Do not infer performance from clock frequency alone. Instruction mix, memory stalls and available parallelism matter.

WORKED EXAMPLE

Summing one running accumulator creates a dependency chain. Several independent accumulators may expose more instruction-level parallelism, but final combination and numeric-order effects must be considered.

PRACTICE + PASS CRITERION

Compare dependent and independent arithmetic loops while preventing dead-code elimination. Pass: preserve observable results, inspect generated code and explain measured differences as evidence rather than universal CPU facts.

ASSIGNED STUDY

R15: Pipelining and Performance Measures.

CHECK YOUR UNDERSTANDING

1. What is a data dependency?

2. Does higher clock frequency guarantee a faster program?

3. Why can extra independent accumulators help?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 16.3 • MODULE 16

Caches, locality and memory hierarchy

Learning goal Relate access patterns to cache lines and working sets.

Processors use a hierarchy of storage with different capacities and access costs. Caches transfer data in blocks called cache lines. Spatial locality reuses nearby data; temporal locality reuses recently accessed data. A working set is the data actively needed during a phase. Once it exceeds a cache level, behavior can change sharply.

Cache associativity and replacement influence conflicts. A cache miss is not always a main-memory access, because another cache level may satisfy it. Prefetching can help predictable patterns but can also waste bandwidth. Measure several working-set sizes and avoid attributing every slow access to one guessed cache characteristic.

WORKED EXAMPLE

If a hypothetical cache line is 64 bytes and each element is a 4-byte integer, one line can contain 16 consecutive elements when aligned appropriately. A stride of 16 elements can use only one integer from each line.

PRACTICE + PASS CRITERION

Benchmark sequential, strided and randomized access over multiple array sizes. Pass: record assumed versus measured hardware information and explain spatial locality without claiming exact cache behavior from one timing curve.

ASSIGNED STUDY

R15: Caches and the Memory Hierarchy.

CHECK YOUR UNDERSTANDING

1. What is spatial locality?

2. How many 4-byte values fit in a hypothetical 64-byte line?

3. Does every first-level cache miss access DRAM?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 16.4 • MODULE 16

Virtual memory, SIMD and architectural limits

Learning goal Recognize address translation and vector execution as separate mechanisms.

Virtual addresses belong to a process view; address translation maps them to physical storage subject to permissions. A translation lookaside buffer caches recent translations. Large working sets and access patterns can stress translation independently of ordinary data caches.

Single-instruction multiple-data execution applies operations across several lanes. It benefits suitable regular work but needs attention to alignment, tails, masks and numerical semantics. Vector width is not a guaranteed speedup factor. Some workloads are limited by memory bandwidth or serial dependencies instead. Later co-design lessons will connect these observations to roofline and Amdahl-style limits.

WORKED EXAMPLE

Processing four independent additions in vector lanes can reduce instruction overhead, but loading and storing the same bytes may still dominate. The last three elements of a non-multiple-sized array still require correct handling.

PRACTICE + PASS CRITERION

Inspect compiler vectorization reports for a simple loop and compare aligned/misaligned or tail cases safely. Pass: test lengths 0, 1, lane_width-1 and lane_width+1 and explain translation versus caching.

ASSIGNED STUDY

R15: Virtual Memory and Parallel Processing; R11: bounds and arithmetic contracts.

CHECK YOUR UNDERSTANDING

1. What does a TLB cache?

2. Does four-lane SIMD guarantee a fourfold speedup?

3. Why test array lengths not divisible by vector width?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 17

Operating systems and resource management

Prerequisites Module 16.

Four lessons

17.1 Processes, threads, system calls and protection

Explain which resources are isolated and which are shared.

17.2 Allocation, paging and memory-mapped data

Distinguish reserved address space, committed storage and live objects.

17.3 Scheduling, responsiveness and real-time constraints

Distinguish average completion from meeting a deadline.

17.4 Filesystems, persistence and asynchronous I/O

Design data operations that survive partial failure.

Module project

Create a systems lab report covering processes, memory, scheduling and file I/O. Use unprivileged programs and disposable data; do not alter kernel or device settings on a primary machine.

Core references

R16 Remzi and Andrea Arpaci-Dusseau — Operating Systems: Three Easy Pieces

R15 MIT OpenCourseWare — 6.004 Computation Structures

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 17.1 • MODULE 17

Processes, threads, system calls and protection

Learning goal Explain which resources are isolated and which are shared.

A process is an execution environment with an address space and operating-system-managed resources. Threads within a process share memory but have their own execution state. A system call requests a kernel service through a controlled boundary. User/kernel separation protects privileged operations; it does not make application code automatically safe.

Process isolation helps contain faults, while threads allow lower-overhead sharing at the cost of synchronization complexity. Starting a worker process, a thread and an asynchronous task are not interchangeable choices. Identify ownership, failure propagation and communication needs before choosing a mechanism.

WORKED EXAMPLE

Two threads can see the same game-state object, creating race risk. Two isolated processes normally need an explicit communication mechanism to exchange equivalent information.

PRACTICE + PASS CRITERION

Write a program that reports its process identity and runs independent worker tasks. Pass: explain what memory is shared in the chosen approach and how worker failure is reported without privileged operations.

ASSIGNED STUDY

R16: Processes, Process API and Threads.

CHECK YOUR UNDERSTANDING

1. What do threads in one process typically share?

2. What is a system call?

3. Why might processes be preferable to threads for a risky tool?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 17.2 • MODULE 17

Allocation, paging and memory-mapped data

Learning goal Distinguish reserved address space, committed storage and live objects.

An allocator manages storage for application objects; the operating system manages virtual memory and backing resources. Reserving an address range is not the same as having every page physically resident. Page faults can occur when access requires additional mapping or data movement. Application allocation behavior and operating-system paging interact but are not identical.

Memory mapping can present file data through an address range, but it still requires lifetime, bounds and error handling. It does not guarantee zero I/O latency. Avoid assuming that freeing an object immediately reduces the process's visible memory usage: an allocator may retain storage for reuse.

WORKED EXAMPLE

A large mapped file can occupy a large virtual range while only accessed pages become resident. Reading a new region can still incur page faults and storage activity.

PRACTICE + PASS CRITERION

Compare sequential file reads with a read-only mapped-file approach on a small disposable file. Pass: verify identical content and report virtual/resident concepts separately without promising one method is faster.

ASSIGNED STUDY

R16: Address Spaces, Address Translation, Paging and Memory API.

CHECK YOUR UNDERSTANDING

1. Does reserving virtual address space imply all pages are resident?

2. Why may process memory usage stay high after freeing objects?

3. Does memory mapping eliminate storage latency?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 17.3 • MODULE 17

Scheduling, responsiveness and real-time constraints

Learning goal Distinguish average completion from meeting a deadline.

A scheduler chooses runnable work according to its policy. Context switches have costs, and oversubscription can create contention rather than useful parallelism. Throughput-oriented scheduling is not necessarily suitable for a time-critical audio or input path. Real-time reasoning concerns deadlines and worst-case behavior under stated assumptions.

General desktop operating systems do not make arbitrary user programs hard-real-time merely because a thread has high priority. Priority inversion occurs when high-priority work waits indirectly on lower-priority work holding a needed resource. Design short critical sections and bounded work before experimenting with priorities.

WORKED EXAMPLE

An audio callback needed every few milliseconds can glitch if it waits for a worker holding a mutex during a long file load, even when average CPU usage is low.

PRACTICE + PASS CRITERION

Simulate a scheduler or instrument a bounded producer/consumer workload without changing privileged priorities. Pass: show one contention-induced latency spike and propose a structural fix rather than only raising priority.

ASSIGNED STUDY

R16: Scheduling, Multi-Level Feedback Queue and Concurrency Bugs.

CHECK YOUR UNDERSTANDING

1. What is priority inversion?

2. Does low average CPU use guarantee deadlines are met?

3. Why avoid blocking I/O in an audio callback?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 17.4 • MODULE 17

Filesystems, persistence and asynchronous I/O

Learning goal Design data operations that survive partial failure.

A filesystem organizes persistent data and metadata. A successful write call does not always mean data has reached durable storage. Buffering, ordering and crash consistency require explicit treatment when durability matters. A robust save process avoids overwriting the only valid copy with partially generated data.

Asynchronous I/O lets work proceed without blocking the caller, but completion can occur later and fail. Buffers and request objects must remain valid until completion according to the API. Cancellation is a request with a contract, not necessarily immediate erasure of all work. Separate correctness, durability and performance goals.

WORKED EXAMPLE

A common save strategy writes a new temporary file, validates it and performs an appropriate replacement operation. The exact durability guarantee still depends on filesystem/API synchronization and failure assumptions.

PRACTICE + PASS CRITERION

Build a save pipeline with injected failures before, during and after serialization. Pass: recover a valid prior or new save under your documented model and never reuse an asynchronous buffer before completion.

ASSIGNED STUDY

R16: Files and Directories, Crash Consistency and Journaling.

CHECK YOUR UNDERSTANDING

1. Does every successful write imply durable storage?

2. Why retain an asynchronous I/O buffer until completion?

3. What should a save-recovery test inject?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 18

Concurrency and parallel programming

Prerequisites Modules 12 and 16-17.

Four lessons

18.1 Races, mutexes and happens-before

Protect shared invariants using a clear synchronization model.

18.2 Condition variables, deadlocks and cancellation

Make waiting and shutdown correct under adverse timing.

18.3 Atomics, memory ordering and false sharing

Use low-level synchronization only with a complete correctness argument.

18.4 Task graphs, work stealing and scaling limits

Parallelize independent work and measure end-to-end benefit.

Module project

Implement a bounded job system with dependency handling, shutdown, failure propagation and stress tests. Compare one-thread and multi-thread execution on a real workload.

Core references

R11 C++ Core Guidelines editors — C++ Core Guidelines

R16 Remzi and Andrea Arpaci-Dusseau — Operating Systems: Three Easy Pieces

R37 NVIDIA — CUDA Programming Guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 18.1 • MODULE 18

Races, mutexes and happens-before

Learning goal Protect shared invariants using a clear synchronization model.

Concurrent execution interleaves operations. A C++ data race occurs when conflicting accesses are not properly ordered and at least one is a write, with the relevant accesses not atomic; this leads to undefined behavior. A race condition is a broader logical timing defect and can exist even when individual accesses are atomic.

A mutex protects a shared invariant only when all relevant participants follow the same locking protocol. The critical section must cover the complete operation that needs to be atomic at the application level. Ownership and message passing can reduce shared mutable state. Document the happens-before relationships rather than relying on "it usually runs first."

WORKED EXAMPLE

Checking balance >= price and then subtracting price in separate unlocked steps can allow two purchases to spend the same funds. Making each field read atomic does not automatically make the whole purchase transaction atomic.

PRACTICE + PASS CRITERION

Implement a shared counter and a compound inventory transaction with proper synchronization. Pass: stress test, inspect with an available race detector and explain why detector silence alone is not proof.

ASSIGNED STUDY

R11: Concurrency and Parallelism; R16: Locks and Concurrency Bugs.

CHECK YOUR UNDERSTANDING

1. Can individually atomic reads/writes still form a logical race?

2. What must a mutex protocol protect?

3. Why is "this thread usually runs first" insufficient?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 18.2 • MODULE 18

Condition variables, deadlocks and cancellation

Learning goal Make waiting and shutdown correct under adverse timing.

A condition variable allows a thread to wait for a predicate to become true while coordinating with a mutex. Wait in a loop because wakeups do not by themselves prove the predicate. Notifications and state changes must follow a consistent protocol to avoid missed progress.

Deadlock can arise from circular resource dependencies. A lock-order rule or reduced lock scope can prevent common cases. Shutdown deserves the same rigor as steady-state execution: wake waiters, stop accepting work, define what happens to queued jobs and join threads safely. Cancellation must specify whether work is abandoned, completed or rolled back.

WORKED EXAMPLE

A consumer waits while queue_empty and not stopping. On shutdown, the producer changes stopping under the agreed synchronization and notifies waiters so they can exit even when no work arrives.

PRACTICE + PASS CRITERION

Create a producer/consumer queue with graceful shutdown and failure propagation. Pass: test stopping with an empty queue, a full queue and active consumers; no thread remains blocked indefinitely.

ASSIGNED STUDY

R16: Condition Variables, Semaphores and Concurrency Bugs; R11: locking guidance.

CHECK YOUR UNDERSTANDING

1. Why test a condition-variable predicate in a loop?

2. What does a consistent lock order prevent?

3. Why must shutdown notify blocked workers?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 18.3 • MODULE 18

Atomics, memory ordering and false sharing

Learning goal Use low-level synchronization only with a complete correctness argument.

Atomic operations provide indivisible access according to their type and memory-order rules. Acquire/release synchronization can publish earlier writes when the specified read observes the release sequence appropriately. Relaxed ordering does not provide the same cross-object publication guarantees. Volatile is not a substitute for C++ thread synchronization.

False sharing occurs when independent frequently written data occupy a shared cache line, causing coherence traffic. Padding may help a measured case but increases footprint. Lock-free algorithms also face reclamation and progress issues; an atomic pointer alone does not make referenced storage safe to free. Prefer a proven library design unless low-level implementation is the explicit learning goal.

WORKED EXAMPLE

A producer writes payload, then publishes a ready flag with release ordering; a consumer that observes the corresponding flag with acquire ordering can safely observe the published payload under the complete protocol.

PRACTICE + PASS CRITERION

Write a publication litmus test with a documented happens-before diagram. Measure a padded versus unpadded counter layout. Pass: do not claim stress tests prove memory-model correctness or assume a fixed cache-line size without labeling it.

ASSIGNED STUDY

R11: atomics and memory model; R16: concurrency; R37: later GPU memory-model comparison.

CHECK YOUR UNDERSTANDING

1. Is volatile a C++ inter-thread synchronization mechanism?

2. What is false sharing?

3. Why is freeing a node difficult in lock-free structures?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 18.4 • MODULE 18

Task graphs, work stealing and scaling limits

Learning goal Parallelize independent work and measure end-to-end benefit.

A task graph expresses jobs and dependencies. Parallel execution is possible among ready tasks that do not conflict. Work stealing distributes queued tasks dynamically, but scheduling overhead can exceed the benefit of tiny jobs. Batch work at a granularity that matches the workload and hardware.

Amdahl's law models the limit from a serial fraction: speedup = 1/(s+(1-s)/p) under its simplifying assumptions. Communication, contention and load imbalance can reduce real speedup further. A job system must handle exceptions, ownership, completion and shutdown, not only start threads. Benchmark the actual simulation rather than an empty task loop.

WORKED EXAMPLE

If 20% of a workload remains serial, an infinite number of ideal workers cannot exceed a 5x speedup. Four ideal workers give 1/(0.2+0.8/4)=2.5x.

PRACTICE + PASS CRITERION

Implement a small task graph for independent entity updates followed by aggregation. Pass: compare 1, 2 and available worker counts, verify results and explain a case where more workers hurt.

ASSIGNED STUDY

R15: Parallel Processing; R16: concurrency; R11: task-oriented concurrency guidance.

CHECK YOUR UNDERSTANDING

1. What is the ideal maximum speedup with a 25% serial fraction?

2. Why can very small tasks reduce performance?

3. What does a task dependency express?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 19

Networks, storage and data services

Prerequisites Modules 15 and 17-18.

Four lessons

19.1 Network layers, latency and transport semantics

Choose transport behavior from application requirements.

19.2 Protocol design, serialization and timeouts

Treat every received byte as untrusted structured input.

19.3 Relational modeling, SQL and indexes

Represent game data with constraints that survive application mistakes.

19.4 Transactions, consistency and distributed failure

Preserve invariants when several operations must succeed together.

Module project

Build a local-only client/server message exercise and a small relational game-data service. Test malformed framing, transaction rollback and duplicate requests without exposing a public service.

Core references

R17 IETF / RFC Editor — RFC 9293: Transmission Control Protocol

R18 PostgreSQL Global Development Group — PostgreSQL tutorial

R16 Remzi and Andrea Arpaci-Dusseau — Operating Systems: Three Easy Pieces

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 19.1 • MODULE 19

Network layers, latency and transport semantics

Learning goal Choose transport behavior from application requirements.

Networks exchange data through layered protocols. IP provides packet delivery without guaranteeing arrival or order. TCP presents a reliable ordered byte stream, not a stream of application messages. UDP preserves datagram boundaries but does not itself supply delivery, ordering or congestion-control behavior. Applications still need explicit protocols.

Latency, jitter, loss and bandwidth describe different constraints. A high-bandwidth link can still have long round-trip delay. Reliable delivery can involve waiting for missing information, which affects real-time design. Use localhost or an authorized lab network for exercises and keep authentication and encryption decisions separate from transport assumptions.

WORKED EXAMPLE

Two send calls over TCP do not guarantee two matching receive calls. The receiver may get part of one message or several messages together and must parse framing correctly.

PRACTICE + PASS CRITERION

Create a localhost client/server exchange with a length-prefixed message. Pass: split one message across several writes and combine several messages in one write; the receiver reconstructs them correctly.

ASSIGNED STUDY

R17: TCP service and interface semantics; R16: networking context.

CHECK YOUR UNDERSTANDING

1. Does TCP preserve application message boundaries?

2. Does high bandwidth imply low latency?

3. What behavior must an application add when using UDP?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 19.2 • MODULE 19

Protocol design, serialization and timeouts

Learning goal Treat every received byte as untrusted structured input.

A protocol defines message types, fields, ranges, versioning and permitted sequences. Serialization must specify byte order and numeric representation. Validate lengths before allocation and cap total resource use. A checksum can detect some corruption but is not equivalent to cryptographic authentication.

Timeouts handle missing progress, not certainty about remote state. A timed-out operation may have executed remotely even if its response was lost. Request IDs and idempotent operations help manage retries. Compatibility is a design decision: unknown fields and versions should be accepted, ignored or rejected according to a documented rule.

WORKED EXAMPLE

A repeated "add 10 coins" request can duplicate a reward. A request with a unique operation ID and server-side deduplication can avoid applying the same accepted operation twice under the defined retention policy.

PRACTICE + PASS CRITERION

Add versioned framing, maximum sizes, timeout handling and request IDs to the lab protocol. Pass: reject oversized and truncated messages before dangerous allocation and test duplicate delivery.

ASSIGNED STUDY

R17: transport failure context; R18: transaction and data-integrity foundations.

CHECK YOUR UNDERSTANDING

1. Does a timeout prove the remote operation never happened?

2. Why validate a length before allocating its buffer?

3. What does idempotence mean?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 19.3 • MODULE 19

Relational modeling, SQL and indexes

Learning goal Represent game data with constraints that survive application mistakes.

A relational schema defines tables, keys and relationships. A primary key identifies a row; a foreign key constrains references between tables. Normalization reduces certain redundancy and update anomalies, while deliberate denormalization trades simpler reads for additional consistency work.

SQL queries express desired results, but their execution plans determine physical work. An index can accelerate suitable lookups while consuming storage and adding write cost. Parameterized queries separate data from executable query structure. Do not build SQL by concatenating untrusted text, and do not assume an index helps every query.

WORKED EXAMPLE

A players table and an inventory table linked by player_id avoid embedding an unbounded list in a single ambiguous text field. A uniqueness constraint on (player_id,item_id) can enforce one aggregate row per item type.

PRACTICE + PASS CRITERION

Create a small schema with primary, foreign and uniqueness constraints. Use parameterized queries and inspect one indexed query plan. Pass: demonstrate rejected invalid references and compare a query before/after indexing.

ASSIGNED STUDY

R18: tables, queries, joins, foreign keys and indexes via the linked manual.

CHECK YOUR UNDERSTANDING

1. What does a foreign key protect?

2. Why can indexes slow writes?

3. Why use parameterized SQL?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 19.4 • MODULE 19

Transactions, consistency and distributed failure

Learning goal Preserve invariants when several operations must succeed together.

A transaction groups related database work under defined atomicity, consistency, isolation and durability guarantees. Isolation levels differ; applications must understand which anomalies remain possible. A transfer should not debit one account without the corresponding credit. Concurrency can require retries even when each statement is individually correct.

Across multiple services, a local transaction does not automatically create a global one. Partial failure, duplicate messages and stale reads require explicit coordination. Separate durable authoritative data from caches. A cache miss is recoverable; losing the only authoritative record is a different failure. Test the system's invariants under interruption rather than only normal response codes.

WORKED EXAMPLE

Transferring five items between players requires both inventories to change together and total quantity to remain constant. A failure between updates must not destroy or create items.

PRACTICE + PASS CRITERION

Implement an inventory transfer with transactions and forced failure between updates. Pass: verify conservation, rollback and duplicate-request handling; document what is not guaranteed across processes or services.

ASSIGNED STUDY

R18: Transactions; R16: persistence and failure reasoning.

CHECK YOUR UNDERSTANDING

1. What does transaction atomicity mean?

2. Does a transaction in one database automatically include another service?

3. What invariant should an item-transfer test preserve?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 20

Engineering practice, security and maintainability

Prerequisites Volumes 01-04.

Four lessons

20.1 Requirements, architecture and design records

Make tradeoffs explicit and trace implementation back to needs.

20.2 Defensive programming and threat modeling

Protect assets and trust boundaries before selecting security mechanisms.

20.3 Build automation, dependencies and release discipline

Produce a verifiable artifact from a known source revision.

20.4 Ethics, accessibility, teamwork and sustainable delivery

Evaluate engineering decisions beyond code correctness.

Module project

Complete Gate D: a reviewed systems application with a design record, threat model, CI-style local test pipeline, dependency inventory and an incident/recovery exercise.

Core references

R01 ACM / IEEE-CS / AAAI — CS2023 knowledge areas

R02 ACM / IEEE Computer Society — Computer Engineering Curricula 2016

R05 Scott Chacon and Ben Straub — Pro Git

R11 C++ Core Guidelines editors — C++ Core Guidelines

R29 Microsoft — Xbox Accessibility Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 20.1 • MODULE 20

Requirements, architecture and design records

Learning goal Make tradeoffs explicit and trace implementation back to needs.

A requirement states an observable need or constraint. Functional requirements describe behavior; quality requirements describe properties such as latency, reliability or accessibility. Architecture allocates responsibilities and dependencies to meet those requirements. A diagram without contracts and failure behavior is not enough.

An architecture decision record states the problem, constraints, alternatives, chosen approach and consequences. Avoid premature distributed systems or custom frameworks when a smaller design meets the need. Revisit decisions when evidence changes. Traceability links requirements to tests and avoids declaring success merely because all planned code was written.

WORKED EXAMPLE

"Loads quickly" is vague. "Loads the defined test level within a chosen threshold on the declared baseline machine, measured from action to usable state" is testable once the threshold and procedure are filled in.

PRACTICE + PASS CRITERION

Write six measurable requirements and two decision records for the simulation. Pass: every requirement has an acceptance test and each decision includes a rejected alternative and a cost.

ASSIGNED STUDY

R01: Software Engineering; R02: engineering design and professional practice.

CHECK YOUR UNDERSTANDING

1. What makes a requirement testable?

2. Why record rejected alternatives?

3. Is completing planned tasks the same as satisfying requirements?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 20.2 • MODULE 20

Defensive programming and threat modeling

Learning goal Protect assets and trust boundaries before selecting security mechanisms.

A threat model identifies assets, actors, entry points, trust boundaries and plausible failures. Game clients, mods, save files and asset importers are not automatically trusted. Validate input, minimize privileges and isolate risky processing. Memory-safe interfaces, bounds checks, fuzzing and sanitizers help address different defect classes.

Authentication establishes an identity or credential; authorization decides permitted actions. Encryption alone does not enforce game rules or protect a compromised endpoint. Use established security libraries rather than inventing cryptography. Perform testing only against your own code or authorized lab targets; the purpose here is defensive engineering.

WORKED EXAMPLE

A client may request a move, but an authoritative server should validate speed and permissions rather than accepting the client's claimed final position as truth.

PRACTICE + PASS CRITERION

Threat-model the save parser and local service. Add bounds checks and a small malformed-input fuzz harness. Pass: cap work/memory, preserve useful errors and record residual risks instead of claiming complete security.

ASSIGNED STUDY

R01: Security; R11: bounds/lifetime safety; R02: information-security breadth.

CHECK YOUR UNDERSTANDING

1. How does authorization differ from authentication?

2. Why is a game client a trust boundary?

3. Does encryption replace input validation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 20.3 • MODULE 20

Build automation, dependencies and release discipline

Learning goal Produce a verifiable artifact from a known source revision.

Continuous integration runs repeatable checks when changes are made. A useful pipeline builds, tests and reports failures from a clean environment. Pin dependencies sufficiently to reproduce the result and record toolchain versions. A dependency inventory includes origin, version and license obligations; it is not enough to know that a package downloaded successfully.

Separate secrets from source and limit automation permissions. Preserve symbols or diagnostics needed to investigate crashes, while controlling access to sensitive information. A release artifact should be traceable to a commit and build configuration. Automated success is evidence only for the checks actually run.

WORKED EXAMPLE

A release manifest can record source commit, compiler, build type, dependency versions and asset bundle hash. This helps determine whether a reported defect came from the same artifact that was tested.

PRACTICE + PASS CRITERION

Create a local clean-build script that runs tests and produces a versioned archive plus manifest. Pass: run it from a fresh build directory and deliberately break a test to confirm the pipeline stops.

ASSIGNED STUDY

R05: workflows and tagging; R12: builds and testing; R01: software engineering.

CHECK YOUR UNDERSTANDING

1. What is the purpose of dependency pinning?

2. What should identify a release artifact's origin?

3. Why test a pipeline with an intentional failure?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 20.4 • MODULE 20

Ethics, accessibility, teamwork and sustainable delivery

Learning goal Evaluate engineering decisions beyond code correctness.

Engineering work affects players, colleagues and the public. Accessibility, privacy, intellectual property, honest reporting and sustainable maintenance belong in technical decisions. Obtain permission for assets and code and respect the licenses under which they are supplied. Do not assume publicly visible material is free to redistribute.

Teams need clear responsibilities, respectful review and realistic scope. Crunch is not a substitute for planning and can hide systemic delivery problems. A postmortem should identify contributing conditions and improvements, not scapegoats. For hardware, safety and environmental constraints become additional design requirements, not optional polish.

WORKED EXAMPLE

A performance optimization that removes readable subtitles or makes controls non-remappable may violate the product's requirements despite increasing frame rate.

PRACTICE + PASS CRITERION

Run an accessibility review and a blameless failure retrospective on the project. Pass: identify at least three actionable changes, assign acceptance evidence and document third-party attribution honestly.

ASSIGNED STUDY

R29: input, text, captions and UI guidance; R01/R02: professional and ethical practice.

CHECK YOUR UNDERSTANDING

1. Does public availability imply permission to redistribute?

2. Why include accessibility in acceptance criteria?

3. What should a blameless retrospective seek?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 05 OF 10

Graphics and Game-Engine Construction

Upper undergraduate

Build a renderer and a small engine instead of only learning an existing editor.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 21 From geometry to a software renderer

21.1 Images, color and coordinate conventions • 21.2 Projection and clipping before perspective division • 21.3 Triangle coverage, depth and perspective-correct interpolation • 21.4 Rays, intersections and acceleration structures

Module 22 Explicit GPU programming and graphics APIs

22.1 GPU work submission, resources and pipeline state • 22.2 Buffers, shaders, layouts and resource binding • 22.3 Synchronization, hazards and frames in flight • 22.4 Presentation, resizing and device capability policy

Module 23 Lighting, materials and real-time image quality

23.1 Radiometry, reflectance and physically based materials • 23.2 Textures, mipmaps, filtering and tangent-space detail • 23.3 Shadows, light selection and rendering-path tradeoffs • 23.4 HDR, tone mapping, temporal reconstruction and validation

Module 24 Engine architecture, simulation and resource ownership

24.1 Fixed updates, interpolation and overload policy • 24.2 Entities, components, handles and data-oriented storage • 24.3 Allocators, task graphs and frame-resource lifetime • 24.4 Subsystem boundaries, save formats and reloadability

Module 25 Asset pipelines, editors and an integrated engine

25.1 Importing, cooking and content-addressed asset builds • 25.2 Streaming, residency and memory budgets • 25.3 Editors, inspection and undoable commands • 25.4 Engine integration and reproducible demonstration

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 21

From geometry to a software renderer

Prerequisites Modules 07-08, 11-16; complete the mathematics checkpoint.

Four lessons

21.1 Images, color and coordinate conventions

Define the numerical and geometric meaning of an image buffer.

21.2 Projection and clipping before perspective division

Transform geometry without confusing visibility with division by depth.

21.3 Triangle coverage, depth and perspective-correct interpolation

Produce crack-resistant filled triangles with correct varying attributes.

21.4 Rays, intersections and acceleration structures

Build a second visibility method and compare its costs with rasterization.

Module project

Build a CPU renderer that writes a portable image file. It must clip geometry, rasterize triangles, resolve visibility and interpolate attributes correctly. Keep it as a reference implementation for future GPU work.

Core references

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 21.1 • MODULE 21

Images, color and coordinate conventions

Learning goal Define the numerical and geometric meaning of an image buffer.

An image buffer stores samples on a two-dimensional grid. Its width, height, channel order, stride and origin must be explicit. A row can include padding, so assuming that every row occupies exactly width times bytes-per-pixel can be wrong. A coordinate convention is a contract shared by geometry, cameras, shaders and tools. Record handedness, axis directions, angle units and matrix multiplication order.

Color values also have meaning. Many image assets use a nonlinear transfer function such as sRGB, while lighting and blending generally need appropriate linear-light values. A normalized channel value is not automatically a physical light measurement. Distinguish color data from non-color textures such as normals or roughness. Treating every texture with the same conversion creates subtle rendering errors.

WORKED EXAMPLE

A 320 by 180 image with four 8-bit channels needs 230,400 bytes of tightly packed pixel storage. A normal-map channel is data; decoding it with a color transfer function changes the represented direction.

PRACTICE + PASS CRITERION

Write a PPM image containing a gradient and coordinate markers. Pass: the dimensions, orientation and channel values are verified by tests, and a README states all coordinate and color assumptions.

ASSIGNED STUDY

R19: image representation and transformations; R23: Radiometry, Spectra, and Color.

CHECK YOUR UNDERSTANDING

1. Why must image stride be recorded?

2. How many bytes are in the tightly packed example image?

3. Should a normal map automatically receive sRGB decoding?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 21.2 • MODULE 21

Projection and clipping before perspective division

Learning goal Transform geometry without confusing visibility with division by depth.

A vertex moves through object, world, view and clip spaces according to the chosen convention. Homogeneous clip coordinates retain information needed to clip primitives before perspective division. Dividing by w maps into normalized device coordinates, but blindly dividing vertices behind or across the eye plane can produce enormous or inverted triangles.

Clipping computes intersections with the view volume and retains the visible portion of a primitive. After clipping, perform perspective division and viewport mapping. Different graphics APIs use different conventions for depth ranges and related details; do not copy a projection matrix without checking its contract. Test points on planes, beyond planes and near degeneracies. Back-face culling is a separate decision based on winding and orientation.

WORKED EXAMPLE

For a clip-space component x=2 and w=4, x/w=0.5. A triangle with vertices on both sides of a clipping plane must be clipped as a primitive; dropping one outside vertex alone destroys its shape.

PRACTICE + PASS CRITERION

Implement clipping against one plane, then the complete chosen clip volume. Pass: triangles crossing the near plane remain finite and correctly shaped, and tests cover points exactly on the boundary.

ASSIGNED STUDY

R19: perspective transformations, clipping and the graphics pipeline; R23: Cameras.

CHECK YOUR UNDERSTANDING

1. Why clip before perspective division?

2. What normalized component follows from x=2 and w=4?

3. Is deleting every outside vertex a valid triangle-clipping algorithm?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 21.3 • MODULE 21

Triangle coverage, depth and perspective-correct interpolation

Learning goal Produce crack-resistant filled triangles with correct varying attributes.

Edge functions determine on which side of a triangle edge a sample lies. Their normalized values yield barycentric coordinates, which sum to one for a nondegenerate triangle. A consistent shared-edge rule prevents neighboring triangles from both missing or inconsistently owning a boundary sample. Degenerate triangles need an explicit rejection policy.

A depth buffer resolves visibility according to the chosen depth representation and comparison rule. Screen-space barycentric interpolation of a general surface attribute is not perspective correct. Interpolate attribute divided by clip w and reciprocal w, then divide the interpolated quantities. Depth interpolation has its own pipeline convention; do not substitute an arbitrary world-space distance. Anti-aliasing estimates pixel coverage rather than pretending a pixel is an infinitesimal point.

WORKED EXAMPLE

For screen-space weights lambda_i, an attribute is a=(sum lambda_i*a_i/w_i)/(sum lambda_i/w_i). When all w values are equal, this reduces to ordinary barycentric interpolation.

PRACTICE + PASS CRITERION

Render two triangles sharing an edge and a receding checkerboard. Pass: no cracks under the documented fill rule, correct overlap ordering, and a visible comparison between affine and perspective-correct texture coordinates.

ASSIGNED STUDY

R19: rasterization, barycentric coordinates and texture mapping; R23: Sampling and Reconstruction.

CHECK YOUR UNDERSTANDING

1. What is the sum of barycentric coordinates?

2. When does perspective-correct interpolation reduce to affine interpolation?

3. Why specify a shared-edge coverage rule?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 21.4 • MODULE 21

Rays, intersections and acceleration structures

Learning goal Build a second visibility method and compare its costs with rasterization.

A ray has origin o and direction d, with points r(t)=o+t*d over a stated interval. The nearest valid positive intersection usually determines a primary visible surface. Directions need not be normalized mathematically, but t is a distance only under suitable normalization and units. Robust intersection tests handle parallel directions, boundary cases and floating-point error.

A bounding-volume hierarchy groups primitives under conservative bounds. Traversal rejects entire groups whose bounds the ray misses. Construction cost, traversal cost, memory and update behavior all matter. An acceleration structure is not a promise of logarithmic worst-case work; difficult overlap can degrade it. Use the brute-force renderer as a correctness oracle before optimizing traversal.

WORKED EXAMPLE

If o=(0,0,0), d=(0,0,2) and t=3, the ray point is (0,0,6); t is not the traveled distance because d has length 2.

PRACTICE + PASS CRITERION

Implement sphere or triangle intersections and a small BVH. Pass: accelerated and brute-force results agree within a defined tolerance over randomized rays, including misses and grazing cases; report build and query costs separately.

ASSIGNED STUDY

R23: Shapes and Primitives and Intersection Acceleration.

CHECK YOUR UNDERSTANDING

1. When is ray parameter t directly a distance?

2. What point occurs in the worked example at t=3?

3. Why retain the brute-force implementation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 22

Explicit GPU programming and graphics APIs

Prerequisites Module 21 plus Modules 18 and 20. Choose one compatible backend, not all at once.

Four lessons

22.1 GPU work submission, resources and pipeline state

Separate CPU recording from asynchronous GPU execution.

22.2 Buffers, shaders, layouts and resource binding

Match data representations across CPU and GPU boundaries.

22.3 Synchronization, hazards and frames in flight

Express dependencies without serializing the entire renderer.

22.4 Presentation, resizing and device capability policy

Treat the window system as a changing environment rather than a fixed image.

Module project

Implement a GPU backend for the reference scene. Query device capabilities, enable available validation, document the toolchain and handle resize and shutdown safely. A CPU backend remains a valid fallback while hardware access is arranged.

Core references

R21 Khronos Group — Vulkan tutorial

R22 SDL project — SDL3 documentation

R24 Microsoft — Direct3D 12 graphics

R25 Apple — Metal developer resources

R37 NVIDIA — CUDA Programming Guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 22.1 • MODULE 22

GPU work submission, resources and pipeline state

Learning goal Separate CPU recording from asynchronous GPU execution.

A graphics API lets software describe resources, pipeline state and work for a GPU. The CPU records or encodes commands, then submits them to queues. Submission does not imply immediate completion. The GPU can still be reading data after the CPU has moved to the next frame. This separation is the central source of lifetime and synchronization mistakes.

A pipeline combines programmable stages with fixed-function configuration and resource interfaces. Vulkan, Direct3D 12 and Metal expose related concepts through different contracts; learning one does not make their details interchangeable. Use the API documentation for the installed version and the selected device. Current tutorial feature requirements must be checked, especially on compatibility layers. Start with a minimal scene before adopting a large abstraction framework.

WORKED EXAMPLE

Recording a draw that references a vertex buffer does not copy the entire buffer into the command stream. Destroying that buffer immediately after submission can invalidate in-flight work.

PRACTICE + PASS CRITERION

Create a window or offscreen target and report supported capabilities. Pass: render a clear color, capture validation output and explain which objects belong to CPU recording versus GPU execution.

ASSIGNED STUDY

R21: Introduction and Drawing a Triangle; alternatively R24 or R25 for one native platform; R22: SDL3 platform setup.

CHECK YOUR UNDERSTANDING

1. Does queue submission mean the GPU has finished?

2. Why can immediate buffer destruction be unsafe?

3. Must a beginner implement three graphics APIs simultaneously?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 22.2 • MODULE 22

Buffers, shaders, layouts and resource binding

Learning goal Match data representations across CPU and GPU boundaries.

Buffers contain bytes; shaders interpret those bytes according to a declared layout. Alignment, padding, matrix convention and element stride must agree between host code and shader code. A C++ structure that looks similar to a shader structure is not proof of binary compatibility. Use explicit offsets or verified layout rules and test small diagnostic values.

Resource binding tells a shader which buffers, textures and samplers it can access. Binding indices and lifetime form an interface contract. Vertex and fragment stages execute with different inputs and responsibilities. Compute shaders are a more general work-dispatch path but still need defined indexing and bounds. Out-of-range threads often occur when dispatch dimensions are rounded upward; guard them rather than assuming exact divisibility.

WORKED EXAMPLE

A workload of 1,000 elements dispatched in groups of 64 needs 16 groups, creating 1,024 logical invocations. The last 24 must not access elements beyond index 999.

PRACTICE + PASS CRITERION

Upload a transform and vertex colors, then add a bounded compute operation. Pass: host/shader layouts are documented, a readback test checks results, and non-multiple work sizes remain correct.

ASSIGNED STUDY

R21: vertex buffers, uniform buffers and compute; R37: execution configuration and memory organization, only for the optional CUDA route.

CHECK YOUR UNDERSTANDING

1. Why can matching field names still produce a layout bug?

2. How many 64-thread groups cover 1,000 elements?

3. What does a resource binding establish?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 22.3 • MODULE 22

Synchronization, hazards and frames in flight

Learning goal Express dependencies without serializing the entire renderer.

A hazard occurs when operations access the same resource in an incompatible order, such as reading before a prior write becomes available and visible. Execution ordering and memory visibility are related but distinct requirements. Barriers, events, semaphores or fences serve different roles depending on the API. Learn their exact scope rather than treating every synchronization primitive as a universal wait.

Frames in flight improve overlap but increase the number of live resources and potentially latency. Per-frame allocations cannot be recycled until the relevant work is complete. A CPU-wide wait after every command may hide a bug while destroying concurrency. First write down the producer, consumer, resource state and required ordering; then choose the narrowest correct mechanism.

WORKED EXAMPLE

A compute pass writes positions and a later draw reads them as vertex data. Correctness requires the appropriate dependency and visibility for that resource, not merely placing source-code statements next to each other.

PRACTICE + PASS CRITERION

Implement a compute-to-draw dependency and two reusable frame contexts. Pass: validation reports no relevant hazards and a stress test never resets a context still used by the GPU.

ASSIGNED STUDY

R21: synchronization, frames in flight and resource transitions; use R24 or R25 equivalents for the selected backend.

CHECK YOUR UNDERSTANDING

1. Are execution ordering and memory visibility identical?

2. When may a frame context be reused?

3. Why is waiting for the whole GPU after each command a poor final design?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 22.4 • MODULE 22

Presentation, resizing and device capability policy

Learning goal Treat the window system as a changing environment rather than a fixed image.

Presentation connects rendered images to a display surface. The set of presentation images, formats and modes is constrained by the platform and device. A window can resize, become minimized or lose its surface. Rebuilding size-dependent resources requires coordination with in-flight work. A zero-sized drawable should not trigger invalid allocations or divide-by-zero camera calculations.

Define a feature policy: required capabilities, optional enhancements and a fallback. Record why a device is unsupported instead of failing with an obscure assertion. Device errors must preserve useful diagnostics; recovery strategy can range from a clear restart request to a tested rebuild. Keep platform services behind a small interface, while acknowledging that platform-specific behavior cannot always be abstracted away.

WORKED EXAMPLE

A minimized window can report a drawable size of zero. The application should suspend or skip size-dependent rendering until a usable size returns, rather than allocating a zero-height depth target.

PRACTICE + PASS CRITERION

Stress resize, minimize, restore and orderly shutdown. Pass: no stale size-dependent resources, no division by zero, and a readable capability report explains enabled features and fallbacks.

ASSIGNED STUDY

R21: swapchain recreation and device selection; R22: window events; R24 or R25: presentation and resource lifetime.

CHECK YOUR UNDERSTANDING

1. Why can resizing require more than changing the viewport?

2. What should happen for a zero-height drawable?

3. What is the purpose of an explicit feature policy?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 23

Lighting, materials and real-time image quality

Prerequisites Modules 07, 09-10, 21-22.

Four lessons

23.1 Radiometry, reflectance and physically based materials

Explain how lighting models approximate light transport.

23.2 Textures, mipmaps, filtering and tangent-space detail

Control aliasing while preserving the meaning of texture data.

23.3 Shadows, light selection and rendering-path tradeoffs

Choose a lighting architecture from scene and bandwidth requirements.

23.4 HDR, tone mapping, temporal reconstruction and validation

Separate display mapping from reconstruction and simulation correctness.

Module project

Create a material-and-lighting demonstration with a documented color pipeline. Compare image quality and frame cost at identical resolution and scene content; include reference images, not only attractive screenshots.

Core references

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

R21 Khronos Group — Vulkan tutorial

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 23.1 • MODULE 23

Radiometry, reflectance and physically based materials

Learning goal Explain how lighting models approximate light transport.

Radiometry gives precise meanings to quantities such as radiant flux, irradiance and radiance. A bidirectional reflectance distribution function describes how incoming light contributes to outgoing light at a surface. Physically motivated models respect appropriate energy and reciprocity constraints, although real-time implementations often introduce approximations.

A material parameter is meaningful only within its model and encoding. Roughness controls a distribution of microscopic surface orientations in common microfacet models; it is not simply a blur slider. Metallic and dielectric materials have different reflectance behavior. Normals, tangent frames and units must agree. A plausible image can still be numerically or physically inconsistent, so test simple controlled scenes before judging a complex one.

WORKED EXAMPLE

For an ideal diffuse surface, the Lambertian BRDF is reflectance divided by pi. Increasing reflectance beyond one would violate the intended passive energy-conservation interpretation.

PRACTICE + PASS CRITERION

Implement diffuse lighting and one documented microfacet model. Pass: test a constant environment, grazing angles and roughness limits, and state approximations rather than labeling any shiny shader physically exact.

ASSIGNED STUDY

R23: Radiometry, Reflection Models and Light Transport; R19: shading.

CHECK YOUR UNDERSTANDING

1. What does a BRDF relate?

2. What is the Lambertian BRDF for reflectance rho?

3. Why document roughness encoding?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 23.2 • MODULE 23

Textures, mipmaps, filtering and tangent-space detail

Learning goal Control aliasing while preserving the meaning of texture data.

Textures are sampled signals attached to geometry. When many source texels cover one output pixel, selecting a single texel aliases. Mipmaps prefilter a hierarchy of resolutions; sampling chooses suitable levels and combines samples according to the filtering method. Anisotropic footprints require more than a single isotropic level to preserve oblique detail.

Generate filtered color in an appropriate linear representation, and treat normal or data maps according to their semantics. Tangent-space normal mapping depends on a consistent tangent basis, handedness and mesh UVs. Seams, mirrored UVs and degenerate triangles are valuable tests. Compression trades storage and bandwidth for error; evaluate error in use, not only as a file-size ratio.

WORKED EXAMPLE

A 256 by 256 base texture contains 65,536 texels. A complete power-of-two mip chain adds about one third as many texels beyond the base, approaching four thirds of the base total.

PRACTICE + PASS CRITERION

Render a receding textured floor with and without mipmaps and inspect a normal-mapped seam. Pass: report texture storage including mip levels and explain why non-color data needs its own filtering policy.

ASSIGNED STUDY

R23: Textures and Sampling and Reconstruction; R21: texture images, samplers and mipmaps.

CHECK YOUR UNDERSTANDING

1. What problem do mipmaps primarily address?

2. What is the approximate total mip-chain storage factor?

3. Why can mirrored UVs break normal mapping?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 23.3 • MODULE 23

Shadows, light selection and rendering-path tradeoffs

Learning goal Choose a lighting architecture from scene and bandwidth requirements.

A shadow map stores depth from a light viewpoint, allowing a later surface sample to test visibility. Resolution, bias and filtering introduce tradeoffs: too little bias causes self-shadow artifacts, while too much separates shadows from contact. A ray-traced visibility query has different costs and artifacts, not automatic perfection.

Forward rendering evaluates material shading while drawing geometry. Deferred approaches store intermediate surface information and light it later, trading bandwidth and storage for different lighting behavior. Tiled or clustered light selection limits which lights a region considers. Transparency, multisampling, material diversity and platform bandwidth affect the choice. Count bytes and work rather than selecting a path because a large studio uses it.

WORKED EXAMPLE

A hypothetical 1,920 by 1,080 G-buffer with 16 bytes per pixel occupies about 33.2 MB for one set of attachments, before depth, extra buffers or repeated reads and writes.

PRACTICE + PASS CRITERION

Implement shadow mapping and compare a simple forward path with tiled light selection or a small deferred prototype. Pass: disclose buffer sizes, light counts, transparency limitations and measured frame costs.

ASSIGNED STUDY

R19: real-time rendering and shadows; R23: Lights and Light Transport.

CHECK YOUR UNDERSTANDING

1. Why can excessive shadow bias be visible?

2. Does deferred rendering remove bandwidth cost?

3. What determines whether clustered lighting helps?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 23.4 • MODULE 23

HDR, tone mapping, temporal reconstruction and validation

Learning goal Separate display mapping from reconstruction and simulation correctness.

High-dynamic-range rendering retains scene values beyond a narrow display range. Tone mapping converts those values for a target presentation, while exposure controls the chosen scale. These are not substitutes for correct lighting units or color conversion. Display capabilities and output formats must be handled explicitly.

Temporal techniques reuse earlier samples, often after motion-based reprojection. History can reduce noise or aliasing but can also create ghosting when visibility changes or motion vectors are wrong. Detect invalid history, including disocclusion and camera cuts. A single still screenshot misses temporal artifacts. Evaluate moving sequences, latency, stability and error against a suitable reference. An image-quality metric is evidence, not a complete description of perceptual quality.

WORKED EXAMPLE

A newly revealed background pixel has no valid previous foreground history. Reusing that history without rejection can leave a trailing silhouette even when each individual frame is numerically finite.

PRACTICE + PASS CRITERION

Add exposure, a documented tone mapper and a basic temporal accumulation experiment. Pass: capture a camera cut and disocclusion test, show failure modes, and compare both still-image error and motion behavior.

ASSIGNED STUDY

R23: Film and Imaging and Sampling; R19: anti-aliasing and image processing.

CHECK YOUR UNDERSTANDING

1. Are tone mapping and correct light transport the same task?

2. Why reject history at a camera cut?

3. Why are still screenshots insufficient for temporal methods?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 24

Engine architecture, simulation and resource ownership

Prerequisites Modules 12, 17-18, 21-23.

Four lessons

24.1 Fixed updates, interpolation and overload policy

Separate simulation time from rendering time and real elapsed time.

24.2 Entities, components, handles and data-oriented storage

Design identity separately from memory location.

24.3 Allocators, task graphs and frame-resource lifetime

Make temporary storage and parallel dependencies observable.

24.4 Subsystem boundaries, save formats and reloadability

Choose interfaces that make change and failure manageable.

Module project

Build a small engine with a fixed-step world, generational entity handles, explicit resource ownership and an inspectable task graph. Keep the game rules separate enough to run automated headless tests.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R27 Glenn Fiedler — Fix Your Timestep!

R11 C++ Core Guidelines editors — C++ Core Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 24.1 • MODULE 24

Fixed updates, interpolation and overload policy

Learning goal Separate simulation time from rendering time and real elapsed time.

A fixed-step simulation advances by a chosen interval, while rendering may occur at a different rate. An accumulator collects elapsed time and runs enough simulation steps to catch up. Rendering can interpolate between stored simulation states using the leftover fraction. This improves presentation smoothness without changing the simulation step itself.

The system needs an overload policy. An unbounded catch-up loop can consume more time than it recovers. Capping work, dropping accumulated time or reducing quality have different consequences and must be documented. Fixed stepping alone does not guarantee deterministic results: input timing, floating-point behavior, iteration order and parallel execution can still differ. Keep wall-clock measurement outside the deterministic state transition where possible.

WORKED EXAMPLE

With a 10 ms simulation step and 25 ms accumulated, two updates consume 20 ms and leave 5 ms. The interpolation fraction is 0.5 between the relevant stored states.

PRACTICE + PASS CRITERION

Implement an accumulator loop and inject long frames. Pass: simulation advances consistently under normal load, interpolation is bounded, and the overload behavior is tested rather than silently spiraling.

ASSIGNED STUDY

R27: Fix Your Timestep!; R20: game loops and timing. Use the article as a starting model, not a guarantee of determinism.

CHECK YOUR UNDERSTANDING

1. What fraction remains after two 10 ms steps from 25 ms?

2. Does a fixed timestep alone guarantee determinism?

3. Why must catch-up work be bounded or otherwise controlled?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 24.2 • MODULE 24

Entities, components, handles and data-oriented storage

Learning goal Design identity separately from memory location.

An entity is an identity to which state can be attached. A component represents a category of data or behavior according to the chosen architecture. An entity-component-system design is useful when it improves iteration, separation and tooling; it is not a universal requirement for every object.

A handle can contain a slot index and a generation counter. Reusing a slot changes its generation, allowing stale handles to be rejected. Raw pointers into moving arrays cannot provide the same safety automatically. Sparse sets, archetype chunks and ordinary arrays have different iteration and mutation costs. Structural changes during iteration require a policy, such as deferred commands. Evaluate representative access patterns rather than maximizing architectural complexity.

WORKED EXAMPLE

Handle (slot 7, generation 3) must not refer to a new entity placed in slot 7 with generation 4. Matching the index alone would mistakenly resurrect a destroyed identity.

PRACTICE + PASS CRITERION

Implement a small generational entity store with two components and deferred destruction. Pass: stale handles fail safely, iteration survives queued changes, and benchmarks include creation, deletion and common queries.

ASSIGNED STUDY

R20: game-state organization; R26: gameplay architecture for comparison; R11: lifetime and interface guidance.

CHECK YOUR UNDERSTANDING

1. Why separate identity from an object address?

2. Is handle (7,3) valid for a slot now at generation 4?

3. Why defer structural changes during a component iteration?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 24.3 • MODULE 24

Allocators, task graphs and frame-resource lifetime

Learning goal Make temporary storage and parallel dependencies observable.

Arena allocation groups objects with a shared lifetime and can make allocation and reset inexpensive. It is safe only when no surviving references point into reset storage. A frame arena may outlive the CPU frame when asynchronous jobs or the GPU still reference it. Allocation strategy follows lifetime, not the other way around.

A task graph declares dependencies among units of work. A frame graph can additionally describe resource reads and writes so ordering, transient allocation and synchronization can be derived. Hidden global state defeats this model. Start with a correct serial execution of the graph, then parallelize independent work. Instrument queue delay, execution duration and critical-path length rather than reporting only total task count.

WORKED EXAMPLE

Tasks A and B can run concurrently, but C reading both outputs waits for both. Adding more workers cannot shorten C or remove that dependency.

PRACTICE + PASS CRITERION

Create a serial-then-parallel task graph and a checked temporary arena. Pass: resetting live storage is detected in tests, dependency order is verified, and the critical path is identified from a trace.

ASSIGNED STUDY

R11: ownership and concurrency; R20: engine architecture; R26: engine systems as a comparative implementation.

CHECK YOUR UNDERSTANDING

1. When is resetting an arena safe?

2. Can task C consume A before A has completed its required output?

3. Why implement a serial graph executor first?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 24.4 • MODULE 24

Subsystem boundaries, save formats and reloadability

Learning goal Choose interfaces that make change and failure manageable.

An engine subsystem should expose a small contract for initialization, updates, resource ownership and shutdown. Avoid a global manager that silently depends on every other manager. Explicit dependency order makes partial initialization failure and test construction easier to reason about. A plugin boundary adds binary and versioning constraints beyond ordinary source-level interfaces.

Save data should represent durable game meaning rather than raw memory addresses or accidental object layout. Version formats, validate input and define migration or rejection behavior. Hot reload is especially difficult when code, layouts or resources change while old references remain. A safe educational implementation can stop the world, unload controlled state and rebuild it rather than pretending arbitrary live code replacement is simple.

WORKED EXAMPLE

A saved entity reference uses a stable serialized identifier and a reconstruction table, not a pointer value from the previous process. A missing referenced asset needs a fallback or explicit error.

PRACTICE + PASS CRITERION

Add versioned saves and one reloadable asset type. Pass: old supported data migrates, malformed data is rejected without partial world corruption, and subsystem shutdown works after injected initialization failure.

ASSIGNED STUDY

R20: resource management and tools; R26: serialization and gameplay architecture; R11: interfaces and error handling.

CHECK YOUR UNDERSTANDING

1. Why is a raw memory pointer unsuitable for a durable save?

2. What must a format version enable?

3. Why test partial initialization failure?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 25

Asset pipelines, editors and an integrated engine

Prerequisites Module 24 and the software-engineering practices of Module 20.

Four lessons

25.1 Importing, cooking and content-addressed asset builds

Treat assets as compiled inputs with dependency tracking.

25.2 Streaming, residency and memory budgets

Load content without allowing asynchronous work to violate ownership.

25.3 Editors, inspection and undoable commands

Make engine state understandable and safely editable.

25.4 Engine integration and reproducible demonstration

Deliver a small complete system whose limitations are measurable.

Module project

Gate E: deliver a small 3D engine demonstration with an asset cooker, reproducible scene, inspector, save/load, diagnostics and a measured frame/memory budget. A second person must build and run it from the README.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R22 SDL project — SDL3 documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 25.1 • MODULE 25

Importing, cooking and content-addressed asset builds

Learning goal Treat assets as compiled inputs with dependency tracking.

Authoring assets prioritize editability; runtime assets prioritize loading and execution. An importer reads a source representation, while a cooker transforms it into a platform-appropriate runtime form. The build result depends on source bytes, settings, tool version and dependencies. Caching only by filename can return stale or incorrect output.

A content-addressed key can combine these inputs into a digest. Dependency graphs determine what must be rebuilt after a change. Reproducibility requires stable ordering and controlled metadata, not only hashing the final file. Validate imported geometry, dimensions and resource limits because assets are untrusted inputs. Keep source provenance and license information with the pipeline rather than rediscovering it before release.

WORKED EXAMPLE

Changing a texture compression setting must invalidate the cooked texture even when the original PNG is unchanged. Otherwise the cache key is missing a meaningful input.

PRACTICE + PASS CRITERION

Build a cooker for one mesh and one image representation. Pass: identical declared inputs reproduce the same result, changed settings invalidate the cache, and corrupted sources produce readable failures.

ASSIGNED STUDY

R20: asset pipelines; R26: asset management and content tools. Apply Module 20 dependency and provenance practices.

CHECK YOUR UNDERSTANDING

1. Why distinguish source assets from runtime assets?

2. Must a compression-setting change invalidate a cached build?

3. Why include tool version in a cache key?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 25.2 • MODULE 25

Streaming, residency and memory budgets

Learning goal Load content without allowing asynchronous work to violate ownership.

Streaming moves needed content through storage, CPU memory and device memory under a budget. An asset can be requested, loading, decoded, uploaded, resident or failed. Model these states explicitly. A cancellation request must not leave callbacks writing into destroyed owners. Residency policy chooses what remains available when total demand exceeds the budget.

Peak memory matters: compressed input, decoded staging data and final GPU resources can temporarily coexist. Loading one asset at a time is not always sufficient if each individual asset is too large. Level of detail and fallback resources can preserve responsiveness while content is incomplete. Measure misses, stalls and quality changes under controlled constrained budgets.

WORKED EXAMPLE

A 20 MB compressed asset may expand to 100 MB and require another staging copy. Budgeting only the on-disk size substantially understates the transient memory requirement.

PRACTICE + PASS CRITERION

Implement a bounded streaming queue with placeholder assets and cancellation. Pass: a forced small budget produces graceful fallback, no use-after-free, and a report of peak rather than only steady-state memory.

ASSIGNED STUDY

R20: resource loading; R26: asset management, asynchronous loading and streaming concepts.

CHECK YOUR UNDERSTANDING

1. Why is compressed file size not the complete memory budget?

2. What should rendering do while an optional asset is unavailable?

3. Why model loading as explicit states?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 25.3 • MODULE 25

Editors, inspection and undoable commands

Learning goal Make engine state understandable and safely editable.

An editor is a program for manipulating structured state, not merely a collection of buttons. Separate the document or scene model from its views. Commands can capture reversible changes for undo and redo. Some operations are not trivially reversible, so record enough prior state or use a well-defined transaction.

Inspection tools expose entity identity, transforms, resources, timings and errors. Good tools shorten debugging loops and let non-programmers test assumptions. Input focus matters when an editor and game view share keys. Changes made during a running simulation need a policy: transient preview, committed authoring change or explicit rejection. Keep data validation active in tools; editor-generated data can still be invalid.

WORKED EXAMPLE

A move command can store the entity identifier, old transform and new transform. Undo restores the old transform only if the target identity remains valid under the editor contract.

PRACTICE + PASS CRITERION

Create a scene inspector with transform editing and a ten-command undo/redo test. Pass: invalid inputs are rejected, focus does not trigger unintended gameplay, and save/reload preserves committed changes.

ASSIGNED STUDY

R20: game tools; R26: editor and reflection concepts. Use a minimal UI toolkit appropriate to the selected platform.

CHECK YOUR UNDERSTANDING

1. What is the model/view distinction useful for?

2. What data can make a transform move reversible?

3. Why distinguish preview state from committed authoring state?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 25.4 • MODULE 25

Engine integration and reproducible demonstration

Learning goal Deliver a small complete system whose limitations are measurable.

Integration exposes mismatched assumptions that isolated components hide: units, coordinate systems, thread ownership, asset versions and shutdown order. A useful milestone freezes a small scene and tests the complete path from source assets to presentation. It is better to finish a constrained engine than to accumulate disconnected advanced features.

Define a target machine and operating conditions before claiming a frame budget. Report CPU and GPU timings separately where available, plus resolution, scene counts, build mode and peak memory. Use percentiles or time-series evidence to reveal stutter. A successful demonstration also needs clear controls, error messages and a clean build process. Reviewers should be able to reproduce the result without undocumented files on the author's computer.

WORKED EXAMPLE

A claimed 60 frames per second implies a 16.67 ms frame interval, but an average of 16 ms can still hide repeated 60 ms stalls. The distribution matters to the declared experience.

PRACTICE + PASS CRITERION

Complete Gate E using the project specification in the handbook. Pass: another person follows the README, a headless test scene passes, all assets have provenance, and a limitations section identifies remaining correctness or performance risks.

ASSIGNED STUDY

R20: project completion; R26: profiling and engine architecture comparison; revisit Module 20 release and review practices.

CHECK YOUR UNDERSTANDING

1. What frame interval corresponds to 60 frames per second?

2. Why report more than average frame time?

3. What does a clean-machine build test reveal?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 06 OF 10

Game Systems and Production

Upper undergraduate / engineering capstone

Add simulation, animation, audio, AI, multiplayer, accessibility and release discipline.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 26 Collision detection and physical simulation

26.1 Broad phase, narrow phase and geometric robustness • 26.2 Rigid bodies, angular motion and collision impulses • 26.3 Constraints, iterative solvers and continuous collision detection • 26.4 Physics testing, determinism and engine integration

Module 27 Animation, audio and interactive timing

27.1 Skeletons, transforms and skinning • 27.2 Animation blending, state machines and inverse kinematics • 27.3 Digital audio, mixing and real-time callback discipline • 27.4 Spatial audio, animation events and multimodal accessibility

Module 28 Game AI, decision systems and machine learning

28.1 Navigation graphs, steering and dynamic obstacles • 28.2 Behavior trees, utility systems and explainable decisions • 28.3 Adversarial search, planning and simulation budgets • 28.4 Learning systems, neural inference and responsible evaluation

Module 29 Multiplayer simulation and online services

29.1 Authority, prediction and reconciliation • 29.2 Snapshots, interpolation, relevance and lag compensation • 29.3 Lockstep, rollback and deterministic replay • 29.4 Match services, identity, persistence and failure recovery

Module 30 Production quality and the undergraduate engine capstone

30.1 Profiling, frame budgets and performance regression tests • 30.2 Compatibility, accessibility, localization and human testing • 30.3 Packaging, patching, modding and supply-chain boundaries • 30.4 Capstone review, release and technical postmortem

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 26

Collision detection and physical simulation

Prerequisites Modules 07-10, 14 and 24.

Four lessons

26.1 Broad phase, narrow phase and geometric robustness

Separate possible collisions from confirmed contacts.

26.2 Rigid bodies, angular motion and collision impulses

Use conservation laws with explicit approximations.

26.3 Constraints, iterative solvers and continuous collision detection

Explain where discrete simulation fails and how solvers control error.

26.4 Physics testing, determinism and engine integration

Validate simulation with invariants and adversarial scenes.

Module project

Build a deterministic test harness for a small physics scene, compare your simple implementation with a maintained physics library, and record precisely which physical and numerical assumptions differ.

Core references

R28 Box2D project — Box2D documentation

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 26.1 • MODULE 26

Broad phase, narrow phase and geometric robustness

Learning goal Separate possible collisions from confirmed contacts.

A collision system first identifies potentially interacting pairs, then performs more expensive geometric tests. The broad phase must be conservative: extra candidate pairs cost time, but missing a real collision breaks correctness. Spatial grids, sweep-and-prune and bounding-volume hierarchies suit different distributions and movement patterns.

The narrow phase determines actual overlap or separation and, when needed, contact positions and normals. Contact normal orientation is an interface contract. Touching, overlapping and nearly parallel configurations need deliberate tolerance policies. Floating-point tolerances should relate to scale and purpose, not an unexplained universal constant. Keep a brute-force pair generator as an oracle for small tests before optimizing.

WORKED EXAMPLE

For 100 objects, checking every unordered pair requires 100 x 99 / 2 = 4,950 pair tests. A broad phase helps only if its maintenance cost and candidate count are favorable for the scene.

PRACTICE + PASS CRITERION

Implement a grid broad phase and circle or box narrow phase. Pass: no true pairs are omitted compared with brute force over randomized scenes, and boundary-touching behavior is documented.

ASSIGNED STUDY

R28: collision module, shapes and broad-phase concepts; R19: geometry and collision.

CHECK YOUR UNDERSTANDING

1. May a correct broad phase return false positives?

2. How many unordered pairs exist for 100 objects?

3. Why document contact-normal orientation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 26.2 • MODULE 26

Rigid bodies, angular motion and collision impulses

Learning goal Use conservation laws with explicit approximations.

A rigid-body model represents position, orientation, linear velocity and angular velocity without modeling deformation. Mass relates force to linear acceleration, while an inertia tensor relates torque and angular acceleration under the chosen frame conventions. Off-center forces and impulses can change rotation as well as translation.

Collision impulses change velocity over an idealized short event. For a frictionless one-dimensional collision, restitution controls the ratio of separating to approaching relative speed under the model. Friction constrains tangential impulses and requires care around resting contact. Infinite-mass or static bodies are commonly represented through zero inverse mass rather than numerical infinity. Conservation tests are powerful when applied to scenarios that actually satisfy the assumptions.

WORKED EXAMPLE

A 2 kg body receiving a 6 N*s impulse through its center changes velocity by 3 m/s. The same impulse away from its center also changes angular momentum.

PRACTICE + PASS CRITERION

Implement center-of-mass impulses and a simple rotational case. Pass: momentum is conserved in isolated tests within tolerance, static bodies remain fixed, and energy behavior matches the chosen restitution model.

ASSIGNED STUDY

R28: rigid bodies, contacts and simulation; R19: physical simulation.

CHECK YOUR UNDERSTANDING

1. What velocity change follows from 6 N*s applied to 2 kg?

2. Why can an off-center impulse cause rotation?

3. Why use zero inverse mass for a static body?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 26.3 • MODULE 26

Constraints, iterative solvers and continuous collision detection

Learning goal Explain where discrete simulation fails and how solvers control error.

A constraint restricts allowed motion, such as maintaining a joint distance or preventing interpenetration. Iterative solvers repeatedly reduce constraint error using local updates. The number of iterations, ordering, timestep and warm starting influence convergence and apparent stiffness. A stable-looking scene is not proof of exact constraint satisfaction.

Discrete collision detection samples configurations at steps and can miss an object passing completely through a thin obstacle. Continuous methods examine motion over an interval or estimate a time of impact. They require motion assumptions and add cost. Use them selectively where speed and geometry justify them. Position correction should not silently create unbounded energy or conceal a broken velocity solver.

WORKED EXAMPLE

An object moving 30 m/s travels 0.5 m during a 1/60 s step. A wall much thinner than that can be crossed between sampled positions without either endpoint overlapping.

PRACTICE + PASS CRITERION

Create a fast-projectile tunneling test and a short chain of constraints. Pass: a continuous or swept test catches the projectile, and a plot or table shows constraint error versus iteration count and timestep.

ASSIGNED STUDY

R28: joints, solver settings and continuous collision; R19: numerical simulation.

CHECK YOUR UNDERSTANDING

1. How far does 30 m/s travel in 1/60 second?

2. Why can increasing solver iterations help?

3. Does CCD remove every numerical simulation problem?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 26.4 • MODULE 26

Physics testing, determinism and engine integration

Learning goal Validate simulation with invariants and adversarial scenes.

Physics tests should include resting stacks, extreme but supported mass ratios, repeated contacts, long runtimes and objects entering or leaving the world. Define units and supported scales before tuning. NaNs or exploding positions should produce a diagnostic near their origin, not only a later renderer crash.

Deterministic replay requires control of ordering, arithmetic assumptions, random sources and inputs. Bitwise determinism across platforms is a stronger requirement than visually similar behavior on one machine. A library can have explicit determinism limits; record them. Compare analytic solutions where available and use convergence studies elsewhere. Integration tests must verify that rendering interpolation, collision events and entity destruction do not violate the physics system's lifetime rules.

WORKED EXAMPLE

A freely moving body with no force should maintain velocity in the ideal model. A falling body before collision can be checked against an analytic constant-acceleration solution, separating integration error from collision behavior.

PRACTICE + PASS CRITERION

Build a regression suite with replay hashes on one pinned environment, finite-value checks and analytic tests. Pass: failures identify the first divergent tick and reports distinguish same-machine repeatability from cross-platform determinism.

ASSIGNED STUDY

R28: simulation, determinism and limitations; revisit Modules 08 and 24.

CHECK YOUR UNDERSTANDING

1. Is same-machine repeatability proof of cross-platform determinism?

2. Why identify the first divergent tick?

3. What should happen when a simulation first creates a NaN?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 27

Animation, audio and interactive timing

Prerequisites Modules 07, 10, 18 and 24.

Four lessons

27.1 Skeletons, transforms and skinning

Compute hierarchical animation without mixing coordinate spaces.

27.2 Animation blending, state machines and inverse kinematics

Coordinate authored motion with runtime goals.

27.3 Digital audio, mixing and real-time callback discipline

Generate sound without missing the output deadline.

27.4 Spatial audio, animation events and multimodal accessibility

Synchronize feedback while preserving user control.

Module project

Integrate an animated character or articulated object with event-driven audio. Demonstrate smooth interpolation, bounded audio work, adjustable sound categories and correct behavior when animation rate differs from simulation rate.

Core references

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R22 SDL project — SDL3 documentation

R26 Epic Games — Programming with C++ in Unreal Engine

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 27.1 • MODULE 27

Skeletons, transforms and skinning

Learning goal Compute hierarchical animation without mixing coordinate spaces.

A skeleton is a hierarchy of local transforms. A joint's world transform depends on its ancestors, so evaluation must respect the hierarchy. Skinning transforms mesh vertices using joint transforms and stored bind information. Inverse bind matrices relate the original mesh pose to the current skeleton; applying a transform in the wrong space creates characteristic stretching or displacement.

Linear blend skinning combines transformed positions using weights that normally sum to one. It is efficient but has deformation limitations. Rotations should be interpolated with a representation appropriate to orientation, such as normalized quaternions with a shortest-path convention. Interpolating arbitrary matrix entries can introduce unintended scale or shear. Test a two-joint example before importing a complex character.

WORKED EXAMPLE

A vertex influenced equally by two joints has weights 0.5 and 0.5. Omitting the bind-pose correction can move it even when the skeleton is supposedly in its original pose.

PRACTICE + PASS CRITERION

Animate a two-joint mesh or line skeleton. Pass: the bind pose reproduces the source geometry, weights are validated and a parent rotation moves descendants consistently.

ASSIGNED STUDY

R19: transformations and animation; R26: skeletal animation concepts.

CHECK YOUR UNDERSTANDING

1. Why evaluate parent transforms before dependent children?

2. What should ordinary normalized skinning weights sum to?

3. Why avoid directly interpolating arbitrary rotation-matrix entries?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 27.2 • MODULE 27

Animation blending, state machines and inverse kinematics

Learning goal Coordinate authored motion with runtime goals.

Animation state machines choose motion states and transitions, while blend systems combine clips using parameters such as speed or direction. Transition conditions, interruption rules and event timing are part of gameplay behavior. A clip completing is not always equivalent to an action becoming interruptible.

Inverse kinematics finds joint configurations that satisfy target constraints, such as placing a foot on a surface. It may have no solution, multiple solutions or a solution outside joint limits. Iterative methods need stopping criteria and stable behavior near unreachable targets. Root motion introduces a policy decision about whether animation or simulation controls displacement. Keep ownership clear to prevent animation and physics from both moving the same body inconsistently.

WORKED EXAMPLE

A two-link arm with lengths 1 and 1 cannot reach a target 3 units from its base. A solver should clamp or report the unreachable condition rather than generating invalid angles.

PRACTICE + PASS CRITERION

Build idle/walk/action transitions and a two-link IK demonstration. Pass: interruptions behave as specified, unreachable targets are handled, and root-motion ownership is documented.

ASSIGNED STUDY

R19: animation and inverse kinematics; R26: animation blueprints and blending concepts, compared with your own small implementation.

CHECK YOUR UNDERSTANDING

1. Can every IK target be reached?

2. What is the maximum reach of two collinear links of length 1 each?

3. Why define root-motion ownership?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 27.3 • MODULE 27

Digital audio, mixing and real-time callback discipline

Learning goal Generate sound without missing the output deadline.

Digital audio represents sampled signals. Sample rate, channel layout and sample format describe the buffer contract. Mixing combines signals with controlled gain; clipping occurs when output exceeds the representable or intended range. Resampling is a signal-processing operation requiring filtering, not merely dropping arbitrary samples.

An audio callback usually has a deadline tied to the buffer duration. Avoid unbounded allocation, blocking locks, disk access or unpredictable work on that path. Prepare data elsewhere and communicate through a carefully designed bounded mechanism. Smaller buffers can reduce one source of latency but leave less scheduling margin. Measure underruns and end-to-end response rather than assuming the smallest available buffer is best.

WORKED EXAMPLE

A 256-frame buffer at 48,000 frames per second spans 256/48,000 seconds, about 5.33 ms. A callback that blocks for 20 ms can miss that timing requirement.

PRACTICE + PASS CRITERION

Mix two generated tones with adjustable gains using a platform audio API. Pass: no clipping under the documented gain policy, bounded callback work and a recorded underrun or deadline-stress test.

ASSIGNED STUDY

R10 from Volume 2: sampling and filtering; R22: SDL3 audio; R20: audio systems.

CHECK YOUR UNDERSTANDING

1. What duration does 256 frames represent at 48 kHz?

2. Why avoid disk reads inside an audio callback?

3. Does halving the buffer automatically improve every aspect of audio?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 27.4 • MODULE 27

Spatial audio, animation events and multimodal accessibility

Learning goal Synchronize feedback while preserving user control.

Spatial audio approximates cues such as level, direction and environmental filtering. A simple stereo pan is not a full head-related acoustic model. Distance attenuation and occlusion are design choices that should be consistent with the game's spatial scale and readability needs. Avoid treating physically plausible sound as automatically useful feedback.

Animation and gameplay events should reference a deliberate clock and deduplication policy. Rollback or repeated evaluation must not replay every sound unintentionally. Provide category volume controls and alternatives for information communicated only through sound or color. Subtitle timing, speaker identification and visual event indicators are engineering requirements, not a final decoration. Evaluate accessibility with actual task completion where possible.

WORKED EXAMPLE

A predicted footstep may be encountered again during replay. Assigning an event identity and a presentation policy prevents an unintended duplicate sound while preserving the corrected simulation.

PRACTICE + PASS CRITERION

Add spatial panning, separate music/effects controls and a visual equivalent for a critical audio cue. Pass: replay does not duplicate one-shot events and a muted-audio test still allows the required task.

ASSIGNED STUDY

R20: audio and event systems; R22: audio interfaces; R29 from the resource directory: audio and subtitle accessibility.

CHECK YOUR UNDERSTANDING

1. Is stereo panning a complete spatial-hearing model?

2. Why give presentation events stable identities during replay?

3. Why provide a non-audio equivalent for a critical cue?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 28

Game AI, decision systems and machine learning

Prerequisites Modules 09, 14, 24 and 26.

Four lessons

28.1 Navigation graphs, steering and dynamic obstacles

Separate finding a route from executing movement.

28.2 Behavior trees, utility systems and explainable decisions

Represent behavior so designers can understand its failures.

28.3 Adversarial search, planning and simulation budgets

Trade decision quality against bounded computation.

28.4 Learning systems, neural inference and responsible evaluation

Understand learning fundamentals without replacing every subsystem with a model.

Module project

Build an inspectable agent that navigates, chooses actions and recovers from blocked plans. Compare a rules-based baseline with one small learned or search-based component; evaluate behavior and runtime cost, not only win rate.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R46 University of California, Berkeley — CS188 Introduction to Artificial Intelligence textbook

R47 Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola — Dive into Deep Learning

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 28.1 • MODULE 28

Navigation graphs, steering and dynamic obstacles

Learning goal Separate finding a route from executing movement.

Pathfinding chooses a route through an abstract representation; steering controls local motion along or toward that route. A navigation mesh represents traversable regions, while grids and waypoint graphs offer other tradeoffs. The abstraction must include the agent's dimensions and movement constraints. A path valid for a point may be impossible for a wide character.

Dynamic obstacles can invalidate a route or require local avoidance. Replanning every frame for every agent may waste time; never replanning may leave agents stuck. Use bounded work, event-driven invalidation and observable failure states. Local avoidance does not guarantee a globally feasible route. Test narrow passages, opposing traffic and unreachable destinations rather than only empty rooms.

WORKED EXAMPLE

A doorway 0.8 units wide is not traversable by a circular agent with diameter 1.0 under a no-overlap model, even if the navigation graph contains an edge through its center.

PRACTICE + PASS CRITERION

Combine A* with a steering controller and a blocked-path detector. Pass: agent size is respected, unreachable goals terminate or report failure, and repeated replanning has a bounded budget.

ASSIGNED STUDY

R46: Search; R20: game AI and navigation concepts.

CHECK YOUR UNDERSTANDING

1. How do pathfinding and steering differ?

2. Can the example 1.0-wide agent pass through the 0.8-wide doorway?

3. Why bound replanning work?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 28.2 • MODULE 28

Behavior trees, utility systems and explainable decisions

Learning goal Represent behavior so designers can understand its failures.

A finite-state machine encodes states and transitions. A behavior tree organizes conditional execution and action status. A utility system scores alternatives and selects according to a policy. These are alternative or complementary organizational tools, not a ranking from obsolete to modern. Choose based on behavior complexity, authoring needs and debugging.

An action should expose progress, success, failure and cancellation semantics. Utility scores need compatible scales and often hysteresis to prevent rapid switching between nearly equal options. Behavior traces should record why an action was selected and why it failed. A black-box agent that occasionally behaves well is harder to improve than a simpler agent whose decisions can be inspected.

WORKED EXAMPLE

An agent choosing between attack score 0.51 and retreat score 0.50 can oscillate as small observations change. A switching threshold or minimum commitment interval can stabilize the policy, with explicit tradeoffs.

PRACTICE + PASS CRITERION

Implement either a small behavior tree or utility selector with a trace viewer. Pass: action cancellation releases resources and three induced failure cases produce understandable explanations.

ASSIGNED STUDY

R20: game AI; R46: agents, utilities and decision-making foundations.

CHECK YOUR UNDERSTANDING

1. What statuses should a multi-frame action expose?

2. Why can nearly tied utility scores cause oscillation?

3. Why is a decision trace valuable?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 28.3 • MODULE 28

Adversarial search, planning and simulation budgets

Learning goal Trade decision quality against bounded computation.

Minimax reasons about alternating choices under a specified game model. Alpha-beta pruning can avoid exploring branches that cannot affect the result while preserving the minimax value under the same assumptions. Move ordering changes efficiency, not the objective. Evaluation functions approximate the value of states that cannot be searched to completion.

Monte Carlo tree search uses repeated sampled exploration to allocate work. Its behavior depends on sampling, exploration policy and rollout or value estimates. Real-time agents need deadlines and a valid fallback action. Hidden information requires reasoning about beliefs rather than granting the agent access to facts a player could not observe. Compare policies on diverse scenarios, not only against one weak opponent.

WORKED EXAMPLE

A branching factor of 10 searched to depth 6 can produce roughly one million leaf paths before pruning. A 2 ms decision budget cannot be assumed sufficient without measurement and a bounded policy.

PRACTICE + PASS CRITERION

Implement minimax with alpha-beta or a small MCTS agent for a tiny game. Pass: legal moves are always returned within a defined budget or fallback, and evaluation includes multiple seeds and opponents.

ASSIGNED STUDY

R46: Games, Minimax, Expectimax and Monte Carlo Tree Search.

CHECK YOUR UNDERSTANDING

1. Does alpha-beta change the minimax objective when applied correctly?

2. How many depth-6 paths arise from branching factor 10 without pruning?

3. Why use a fallback action?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 28.4 • MODULE 28

Learning systems, neural inference and responsible evaluation

Learning goal Understand learning fundamentals without replacing every subsystem with a model.

Supervised learning fits parameters from examples; reinforcement learning optimizes behavior through rewards and interaction. A training objective is not the same as the desired player experience. Split data by independent episodes or sources when appropriate so nearly duplicate examples do not leak between training and testing. Establish a simple non-learned baseline first.

A neural network composes parameterized transformations and nonlinearities; gradients update parameters through the chain rule. Inference cost includes preprocessing, memory movement and scheduling, not only arithmetic. Distribution shift can cause failure when gameplay differs from training. Record data rights, privacy and failure policies. Learned components should remain bounded and observable, especially when their output affects authoritative game state.

WORKED EXAMPLE

A model trained and tested on adjacent frames from the same recorded match may appear accurate by exploiting near-duplicates. Holding out complete matches provides a stronger test of generalization.

PRACTICE + PASS CRITERION

Train a small classifier or value estimator on synthetic game data and compare it with a rules baseline. Pass: independent splits, error analysis, inference timing, reproducible seeds and a fallback behavior are reported.

ASSIGNED STUDY

R46: Machine Learning and Reinforcement Learning; R47: linear models, multilayer perceptrons, backpropagation and generalization.

CHECK YOUR UNDERSTANDING

1. Why separate complete episodes across data splits?

2. What mathematical rule underlies backpropagation?

3. Why retain a rules-based baseline?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 29

Multiplayer simulation and online services

Prerequisites Modules 18-20 and 24; Module 26 for physical interaction.

Four lessons

29.1 Authority, prediction and reconciliation

Provide responsive local control without trusting arbitrary client outcomes.

29.2 Snapshots, interpolation, relevance and lag compensation

Manage bandwidth and latency while making fairness assumptions explicit.

29.3 Lockstep, rollback and deterministic replay

Choose synchronization around the game model rather than genre slogans.

29.4 Match services, identity, persistence and failure recovery

Design a small online service without treating it as part of the render loop.

Module project

Create a local two-client multiplayer demonstration with a network simulator for delay, jitter, loss, duplication and reordering. No public server or paid cloud account is required. Document authority and recovery rules.

Core references

R17 IETF / RFC Editor — RFC 9293: Transmission Control Protocol

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 29.1 • MODULE 29

Authority, prediction and reconciliation

Learning goal Provide responsive local control without trusting arbitrary client outcomes.

An authoritative simulation decides which state is accepted as game truth. Clients commonly send input intentions rather than unrestricted final positions or rewards. Local prediction improves immediate response by simulating the client's own input before server confirmation. Reconciliation compares authoritative state with the corresponding predicted history and reapplies unacknowledged inputs when necessary.

Input sequence numbers or simulation ticks make this correspondence possible. Validation needs resource and rate limits as well as semantic checks. Authority does not eliminate all cheating or fairness problems, but it defines the trust boundary. Presentation smoothing should not rewrite the authoritative rules. Keep correction behavior visible during development so errors are not merely hidden behind interpolation.

WORKED EXAMPLE

The server acknowledges input 40. The client corrects to the returned state for that point and replays locally stored inputs 41 through 44, rather than applying those inputs twice to the wrong starting state.

PRACTICE + PASS CRITERION

Implement a one-dimensional or grid movement server with client prediction. Pass: invalid movement is rejected and induced delay produces documented corrections without permanently diverging state.

ASSIGNED STUDY

R17: transport semantics; R26: networking and multiplayer concepts; R20: networking in games.

CHECK YOUR UNDERSTANDING

1. What is authoritative state?

2. Which inputs are replayed after acknowledgment of input 40 in the example?

3. Why send intentions rather than unrestricted reward totals?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 29.2 • MODULE 29

Snapshots, interpolation, relevance and lag compensation

Learning goal Manage bandwidth and latency while making fairness assumptions explicit.

Remote entities can be rendered between buffered snapshots, trading some added presentation delay for smoother motion under jitter. Extrapolation predicts beyond available data and can be wrong when motion changes. Snapshot sequence numbers, timestamps and baselines must be validated. Delta compression requires agreement about the baseline being referenced.

Interest management sends relevant state rather than every entity to every client. Relevance is also a security concern: sending hidden information can expose it even when the interface does not display it. Lag compensation may evaluate an action against historical state, but history bounds and fairness policy must be explicit. There is no universal policy that makes every latency condition equally fair.

WORKED EXAMPLE

Buffering two snapshots 50 ms apart can interpolate motion during a delay variation, but it intentionally displays a slightly older world. This is a latency-quality tradeoff, not free smoothing.

PRACTICE + PASS CRITERION

Add remote interpolation and relevance filtering. Pass: out-of-order snapshots are handled, missing baselines trigger recovery, and the report explains both visual delay and hidden-state exposure policy.

ASSIGNED STUDY

R20: networked game state; R26: replication and network smoothing; revisit Module 19 message validation.

CHECK YOUR UNDERSTANDING

1. What cost does snapshot interpolation commonly add?

2. Why must delta-compression baselines be identified?

3. Why avoid transmitting irrelevant hidden enemy state?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 29.3 • MODULE 29

Lockstep, rollback and deterministic replay

Learning goal Choose synchronization around the game model rather than genre slogans.

Lockstep peers or participants advance a deterministic simulation using agreed inputs. It can reduce state bandwidth, but progress depends on input availability and determinism. A single divergent arithmetic or ordering decision can amplify into different worlds. Periodic state hashes help detect disagreement but do not repair it by themselves.

Rollback predicts missing inputs, then restores a past state and resimulates when the prediction differs. It needs fast state capture, bounded history and careful handling of irreversible presentation or external side effects. The rollback horizon controls memory and correction cost. Neither rollback nor lockstep is automatically appropriate for every RTS or action game; evaluate state size, simulation cost, latency and player experience.

WORKED EXAMPLE

At 60 simulation ticks per second, storing 120 snapshots covers two seconds of history. If each snapshot is 2 MB, raw snapshot storage alone reaches 240 MB before metadata or compression.

PRACTICE + PASS CRITERION

Implement rollback or input lockstep for the tiny deterministic game. Pass: a delayed-input test converges, state hashes detect a deliberately introduced divergence, and audio or reward side effects are not duplicated.

ASSIGNED STUDY

R20 and R26: multiplayer concepts; derive the implementation using Modules 18, 24 and 26 rather than assuming engine-specific guarantees.

CHECK YOUR UNDERSTANDING

1. How much time do 120 ticks cover at 60 Hz?

2. How much raw storage do 120 snapshots of 2 MB require?

3. Why are external side effects difficult during rollback?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 29.4 • MODULE 29

Match services, identity, persistence and failure recovery

Learning goal Design a small online service without treating it as part of the render loop.

Matchmaking, authentication, session allocation and persistent progression are service responsibilities distinct from frame simulation. A match can continue or fail under a defined policy when a nonessential service is unavailable. Identity and authorization must be checked at the relevant boundary; a valid account is not permission to modify another account's data.

Retries, timeouts and idempotency are essential for reward and progression operations. Use established authentication and cryptographic libraries rather than inventing protocols. Store secrets outside repositories and logs. Operational readiness includes health checks, structured diagnostics, graceful shutdown and recovery tests. For learning, local mock services and synthetic accounts are sufficient and avoid unnecessary privacy and spending risks.

WORKED EXAMPLE

A match-result submission is retried after a timeout. A stable operation identifier lets the progression service recognize the duplicate and avoid granting the same reward twice.

PRACTICE + PASS CRITERION

Build a local match registry and idempotent result store. Pass: duplicate submissions do not duplicate rewards, unauthorized edits fail, and simulated service loss produces a documented client outcome.

ASSIGNED STUDY

R18 from the directory: transactions; R17: transport; revisit Modules 19-20 for distributed failure and threat modeling.

CHECK YOUR UNDERSTANDING

1. Is authentication the same as authorization?

2. Why identify a match-result operation uniquely?

3. Must this lab use a paid public cloud?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 30

Production quality and the undergraduate engine capstone

Prerequisites Modules 21-29 plus Gate D software-engineering evidence.

Four lessons

30.1 Profiling, frame budgets and performance regression tests

Optimize measured bottlenecks without sacrificing correctness.

30.2 Compatibility, accessibility, localization and human testing

Test whether people can actually use the complete product.

30.3 Packaging, patching, modding and supply-chain boundaries

Distribute a game without creating an uncontrolled execution path.

30.4 Capstone review, release and technical postmortem

Demonstrate ownership of a complete engineering result.

Module project

Gate F: release a small original game using your engine, with one complete gameplay loop, settings, save/load, diagnostics, accessibility checks, a tested distribution and a technical postmortem. Scope is judged by quality of evidence, not visual size.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R29 Microsoft — Xbox Accessibility Guidelines

R45 LLVM Project — AddressSanitizer documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 30.1 • MODULE 30

Profiling, frame budgets and performance regression tests

Learning goal Optimize measured bottlenecks without sacrificing correctness.

Profiling identifies where time and resources are spent under a specific workload. Instrumentation records explicit events; sampling estimates where execution tends to be observed. Both have overhead and interpretation limits. A CPU trace alone cannot fully explain GPU work, and a GPU timing alone cannot describe input-to-display latency.

Create repeatable scenes with fixed settings and a warmup policy. Record build type, hardware, drivers, resolution and background conditions. Measure distributions and peak memory, then change one hypothesis-driven factor at a time. A microbenchmark speedup is useful only if it improves an important application path without unacceptable regressions. Keep correctness tests alongside performance tests and tolerate normal measurement noise appropriately.

WORKED EXAMPLE

A subsystem taking 2 ms of a 20 ms frame can save at most 2 ms if removed entirely. A 10x speedup in that subsystem alone yields an 18.2 ms frame under the simplified additive model.

PRACTICE + PASS CRITERION

Profile the capstone scene, optimize one verified bottleneck and add a regression benchmark. Pass: repeated before/after results, unchanged correctness and disclosed measurement uncertainty are included.

ASSIGNED STUDY

R26: profiling and optimization; R45: sanitizer diagnostics; revisit Modules 09 and 18.

CHECK YOUR UNDERSTANDING

1. What is the ideal frame time after a 10x speedup of the example 2 ms subsystem?

2. Why record build type?

3. Why can a microbenchmark improvement fail to help the game?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 30.2 • MODULE 30

Compatibility, accessibility, localization and human testing

Learning goal Test whether people can actually use the complete product.

A compatibility matrix lists supported environments and input/display configurations. Test cases should include different resolutions, aspect ratios, controllers, keyboard layouts and missing optional devices where relevant. Unsupported cases need clear behavior rather than accidental crashes. Localization affects text length, fonts, layout, number formatting and sometimes text direction.

Accessibility begins with requirements such as remappable controls, readable interface scale, contrast, alternative cues and adjustable timing where the game permits. A guideline checklist supports design but does not replace user testing or establish legal compliance. Use informed, voluntary test participation and collect only necessary data. Observe task completion and confusion rather than asking only whether the game is enjoyable.

WORKED EXAMPLE

A button sized for a short English label can clip a longer translated label. A control prompt that hard-codes a keyboard letter can become wrong after rebinding.

PRACTICE + PASS CRITERION

Test the game at two window sizes, with remapped input and an expanded pseudo-localized text set. Pass: no critical clipped controls, a muted-audio task remains solvable and observed usability issues are tracked.

ASSIGNED STUDY

R29: input, text, audio, contrast and UI guidelines; R20: playtesting and production.

CHECK YOUR UNDERSTANDING

1. Why use pseudo-localization before full translation?

2. What should a control prompt show after rebinding?

3. Does checking every accessibility guideline prove legal compliance?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 30.3 • MODULE 30

Packaging, patching, modding and supply-chain boundaries

Learning goal Distribute a game without creating an uncontrolled execution path.

A release package includes the executable, required runtime assets, dependency information, notices and configuration defaults. Test installation and startup outside the development tree. Patch systems need integrity checks, version compatibility and a rollback or recovery plan; interrupted updates must not destroy the only runnable installation.

Modding expands the trust boundary. Data-only mods still require parsing and resource limits, while executable plugins can run with substantial authority unless isolated. Define supported APIs, compatibility versions and permissions. Do not download and execute arbitrary community code automatically. A script sandbox needs explicit capability and resource limits, and its security depends on implementation, not the word sandbox. Preserve user saves separately from replaceable program files.

WORKED EXAMPLE

A malformed mod declares a texture with dimensions large enough to overflow a byte-count calculation. Validation must check multiplication and total resource limits before allocation, even though the file is nominally data.

PRACTICE + PASS CRITERION

Package the game and implement a data-only mod with a versioned schema. Pass: a clean-folder launch works, oversized or malformed mods are rejected and an interrupted-update simulation preserves a valid version.

ASSIGNED STUDY

R20: production; R26: packaging and plugins; R45 and Module 20: defensive memory and dependency practices.

CHECK YOUR UNDERSTANDING

1. Can a data-only mod still be dangerous to robustness?

2. Why separate saves from replaceable installation files?

3. Does naming a system a sandbox establish isolation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 30.4 • MODULE 30

Capstone review, release and technical postmortem

Learning goal Demonstrate ownership of a complete engineering result.

A capstone combines requirements, implementation, tests, performance evidence and communication. Freeze a small scope: one complete loop with a clear beginning, success/failure and restart is enough. Review the architecture against actual use, not only the original diagram. List dependencies and distinguish original code from libraries or assistance.

A technical postmortem explains what worked, what failed, evidence behind decisions and what would change next. Avoid rewriting the project history as an uninterrupted success story. An oral review should include a live bug investigation, a design tradeoff and a small code modification. These reveal understanding beyond a polished video. Graduation to advanced work requires the practical artifact and reasoning, not only lesson attendance.

WORKED EXAMPLE

A small game with reproducible builds, bounded memory, clear controls and an honest bug list is stronger evidence of engineering ownership than an unfinished large world with no tests.

PRACTICE + PASS CRITERION

Complete Gate F and the oral questions in the handbook. Pass: another tester finishes a full loop, the release is reproducible, critical defects are resolved or clearly blocked from release, and the postmortem cites measured evidence.

ASSIGNED STUDY

R20: course projects and critique; revisit the project rubric and professional-practice lessons.

CHECK YOUR UNDERSTANDING

1. What proves completion beyond a recorded demo?

2. Why disclose library and AI assistance?

3. Why include a live modification in an oral review?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 07 OF 10

Computer Engineering and Digital Design

Undergraduate CE core

Move from circuits and gates to RTL, a processor and an FPGA-based game subsystem.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 31 Circuits, measurement and electronic foundations

31.1 Voltage, current, resistance and safe measurement • 31.2 Capacitors, inductors, transients and frequency response • 31.3 Diodes, transistors, CMOS and analog interfaces • 31.4 Sampling, ADCs, DACs and noisy controller inputs

Module 32 Digital logic, state and timing

32.1 Boolean logic and combinational building blocks • 32.2 Registers, finite-state machines and synchronous timing • 32.3 Binary arithmetic, fixed point and overflow policies • 32.4 Clock domains, reset and metastability

Module 33 SystemVerilog and verification-driven RTL design

33.1 RTL semantics and synthesizable SystemVerilog • 33.2 Testbenches, scoreboards and constrained stimulus • 33.3 Assertions, coverage and formal reasoning • 33.4 Synthesis, constraints and FPGA resource mapping

Module 34 Designing a small processor

34.1 Instruction sets, datapaths and single-cycle execution • 34.2 Pipelining, forwarding, stalls and branch recovery • 34.3 Memory interfaces, caches and protection boundaries • 34.4 Processor verification, toolchains and architectural evidence

Module 35 FPGA-based game subsystems and integration

35.1 Memory-mapped I/O, timers, interrupts and DMA • 35.2 Framebuffers, scanout and pixel pipelines • 35.3 A small accelerator and its software contract • 35.4 Integrated digital-design review and optional board bring-up

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 31

Circuits, measurement and electronic foundations

Prerequisites Modules 06-10; basic algebra and calculus competence required. Simulation is the default laboratory.

Four lessons

31.1 Voltage, current, resistance and safe measurement

Analyze a simple circuit using units and conservation laws.

31.2 Capacitors, inductors, transients and frequency response

Relate stored energy to timing and filtering behavior.

31.3 Diodes, transistors, CMOS and analog interfaces

Understand the physical devices beneath digital abstractions.

31.4 Sampling, ADCs, DACs and noisy controller inputs

Convert physical signals into stable digital measurements.

Module project

Create a circuit laboratory notebook using simulation first: voltage/current prediction, an RC transient, a transistor switch and a sampled input. Physical extensions require a qualified mentor and a documented low-voltage safety review.

Core references

R30 MIT OpenCourseWare — 6.002 Circuits and Electronics

R40 OpenStax / Rice University — Precalculus, second edition

R10 MIT OpenCourseWare — 6.003 Signals and Systems

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 31.1 • MODULE 31

Voltage, current, resistance and safe measurement

Learning goal Analyze a simple circuit using units and conservation laws.

Voltage is potential difference between two points; current is charge flow. Resistance relates voltage and current for an ideal resistor through V=IR. Kirchhoff's current law expresses charge conservation at a node, while the voltage law applies around a lumped-circuit loop under its modeling assumptions. Always choose reference directions and keep signs consistent.

Power is P=VI, so a circuit can be logically correct but thermally unsuitable. Real components have ratings and tolerances. A voltmeter is connected across the relevant points; measuring current generally changes the measurement arrangement. Incorrect meter configuration can create a short. Begin in simulation and use only reviewed low-voltage educational hardware for physical work. Do not open mains-powered supplies, displays or battery packs.

WORKED EXAMPLE

An ideal 1,000 ohm resistor across 5 V carries 5 mA and dissipates 25 mW. These are model calculations; a physical build also requires component ratings and safe connections.

PRACTICE + PASS CRITERION

Simulate a two-resistor divider and verify node voltages and current conservation. Pass: predictions include units, the source/load assumptions are stated, and a safety checklist precedes any physical experiment.

ASSIGNED STUDY

R30: lumped-circuit abstraction, resistive networks and basic circuit analysis.

CHECK YOUR UNDERSTANDING

1. What current flows through 1,000 ohms at 5 V?

2. What power is dissipated in that ideal resistor?

3. Why can putting a meter in the wrong current-measurement configuration be dangerous?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 31.2 • MODULE 31

Capacitors, inductors, transients and frequency response

Learning goal Relate stored energy to timing and filtering behavior.

A capacitor stores energy in an electric field and relates current to voltage change through i=C*dv/dt in the ideal model. An inductor relates voltage to current change through v=L*di/dt. These elements make circuit behavior depend on time and initial conditions. An RC network is a useful first-order model for charging, filtering and delay.

For a simple charging RC circuit, the time constant is tau=RC. Frequency-domain analysis uses complex impedance to describe sinusoidal steady-state behavior, but it does not replace startup analysis. Real parasitic resistance, inductance and capacitance matter increasingly as signal edges become fast. A model should state which nonideal effects it intentionally omits.

WORKED EXAMPLE

With R=10,000 ohms and C=1 microfarad, tau=0.01 s, or 10 ms. Starting from zero, the ideal capacitor reaches about 63.2% of its final step-response voltage after one time constant.

PRACTICE + PASS CRITERION

Simulate an RC step response and a sinusoidal frequency sweep. Pass: the measured time constant agrees with the ideal calculation within numerical tolerance, and the report distinguishes transient from steady-state behavior.

ASSIGNED STUDY

R30: capacitors, inductors, first- and second-order systems; R10: frequency response.

CHECK YOUR UNDERSTANDING

1. What is tau for 10 kohm and 1 microfarad?

2. What fraction of a zero-to-final RC charging step is reached at one tau?

3. Why do initial conditions matter?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 31.3 • MODULE 31

Diodes, transistors, CMOS and analog interfaces

Learning goal Understand the physical devices beneath digital abstractions.

A diode has a nonlinear current-voltage relationship; it is not a perfect one-way switch. A transistor can control current and serve as an amplifier or switch depending on its operating region and circuit. MOS devices form complementary networks in CMOS logic. The digital 0/1 abstraction hides voltage ranges, delay, leakage and transition behavior.

Analog interfaces may use amplifiers, comparators and reference voltages. An ideal operational-amplifier model is useful only while feedback and operating limits support its assumptions. Real inputs and outputs have range, bandwidth and loading constraints. Read datasheets as contracts, including absolute maximum ratings and recommended operation, rather than choosing parts from one headline specification.

WORKED EXAMPLE

A logic input specified for one voltage domain is not automatically safe when driven by a higher-voltage output. A logical high in both systems does not imply electrical compatibility.

PRACTICE + PASS CRITERION

Simulate a CMOS inverter and one simple amplifier or comparator stage. Pass: plot input/output behavior, identify valid operating regions and explain one failure of the idealized model.

ASSIGNED STUDY

R30: MOS transistors, amplifiers, digital circuits and operational amplifiers.

CHECK YOUR UNDERSTANDING

1. Why is a transistor not only a binary software switch?

2. Does a logical high guarantee compatibility across voltage domains?

3. Why distinguish absolute maximum ratings from recommended operating conditions?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 31.4 • MODULE 31

Sampling, ADCs, DACs and noisy controller inputs

Learning goal Convert physical signals into stable digital measurements.

An analog-to-digital converter samples and quantizes an input relative to a reference. Resolution describes code granularity, not guaranteed accuracy. Noise, offset, reference error, nonlinearity and acquisition behavior can dominate the nominal bit count. A digital-to-analog converter performs a related reconstruction task with its own filtering and output limits.

An analog signal needs bandwidth control before sampling when aliasing is a concern. Digital filtering afterward cannot generally undo already aliased information. A controller input also needs calibration, dead zones and careful response tuning. Switch debouncing and analog filtering solve different problems. Preserve enough timing information to measure the latency added by filtering.

WORKED EXAMPLE

An ideal 12-bit converter spanning 0 to 3.3 V has a nominal step of about 3.3/4096 = 0.806 mV under that quantization convention. Real usable precision may be lower.

PRACTICE + PASS CRITERION

Simulate a noisy joystick signal, quantization and a simple filter. Pass: report step size, calibration error, added response delay and behavior near the dead-zone boundary.

ASSIGNED STUDY

R30: mixed-signal and measurement concepts; R10: sampling and filtering.

CHECK YOUR UNDERSTANDING

1. How many distinct codes does a 12-bit converter provide?

2. Does 12-bit resolution guarantee 12-bit accuracy?

3. Why filter before sampling to prevent aliasing?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 32

Digital logic, state and timing

Prerequisites Modules 06, 15-16 and 31.

Four lessons

32.1 Boolean logic and combinational building blocks

Translate a logical specification into a circuit with no stored state.

32.2 Registers, finite-state machines and synchronous timing

Model state transitions at clock boundaries.

32.3 Binary arithmetic, fixed point and overflow policies

Choose representations that preserve the needed range and precision.

32.4 Clock domains, reset and metastability

Recognize where synchronous assumptions stop applying.

Module project

Build and test an arithmetic unit, a finite-state controller and a synchronous queue model. For every block, provide a truth table or transition specification before coding its implementation.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R31 Noam Nisan and Shimon Schocken — Nand to Tetris

R35 RISC-V International — RISC-V ISA specifications

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 32.1 • MODULE 32

Boolean logic and combinational building blocks

Learning goal Translate a logical specification into a circuit with no stored state.

Combinational logic computes outputs from current inputs. Boolean algebra and truth tables specify behavior independently of a particular gate arrangement. Multiplexers select among inputs; decoders map a code to selected outputs; adders combine bit patterns according to arithmetic rules. Equivalent logic can have different area, delay and power.

Minimization is not only reducing the printed expression. Fanout, gate availability and signal arrival times influence implementation. A combinational loop usually indicates a design error in beginner synchronous work. Unknown or invalid input codes require a defined policy. Verify small blocks exhaustively before composing them into larger systems.

WORKED EXAMPLE

A two-input AND function has output 1 only for input pair 11. A two-to-one multiplexer computes output=a when select=0 and output=b when select=1.

PRACTICE + PASS CRITERION

Implement a small multiplexer, decoder and four-bit adder in a logic simulator or model. Pass: exhaustive truth-table tests cover every supported input and no state is required for a result.

ASSIGNED STUDY

R15: combinational logic; R31: Boolean logic and arithmetic projects.

CHECK YOUR UNDERSTANDING

1. What distinguishes combinational from sequential logic?

2. What does a two-to-one multiplexer do?

3. Does the shortest Boolean expression always imply the fastest circuit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 32.2 • MODULE 32

Registers, finite-state machines and synchronous timing

Learning goal Model state transitions at clock boundaries.

A register stores state across clock events. A synchronous design computes next-state values from current state and inputs, then captures them at the active clock edge. A finite-state machine makes this transition function explicit. Moore-style outputs depend on state; Mealy-style outputs can also depend directly on inputs, with timing consequences.

Combinational paths between registers must settle within the available timing budget, including clock-to-output, logic, setup and clock uncertainty. Hold requirements constrain how soon data may change after capture. Slowing a clock does not generally fix hold violations. Define reset state and behavior for illegal states rather than relying on power-up luck.

WORKED EXAMPLE

If clock-to-output is 1 ns, logic delay is 6 ns and setup is 1 ns, a simplified zero-skew setup budget needs at least 8 ns per cycle before additional margin, corresponding to at most 125 MHz.

PRACTICE + PASS CRITERION

Design a controller with idle, running and done states, including reset and abort. Pass: a transition table matches simulation and a timing worksheet includes setup and hold as different checks.

ASSIGNED STUDY

R15: sequential logic and finite-state machines; R31: sequential logic.

CHECK YOUR UNDERSTANDING

1. What does a register capture in a synchronous model?

2. What simplified maximum frequency follows from an 8 ns period?

3. Does slowing the clock automatically fix hold violations?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 32.3 • MODULE 32

Binary arithmetic, fixed point and overflow policies

Learning goal Choose representations that preserve the needed range and precision.

Unsigned n-bit arithmetic represents values from zero through 2^n-1. Two's-complement signed arithmetic represents -2^(n-1) through 2^(n-1)-1. Hardware addition can use similar structures for both, but overflow interpretation differs. Explicitly decide whether results wrap, saturate or signal an error.

Fixed-point representations assign an implicit scale to integers. Multiplication changes the scale and usually needs a wider intermediate result. Rounding and truncation introduce error; repeated biased truncation can accumulate. Hardware width affects area and timing as well as numerical behavior. Compare a fixed-point implementation with a higher-precision reference over the complete supported input range where feasible.

WORKED EXAMPLE

A signed 8-bit two's-complement value ranges from -128 to 127. In a format representing integer_value/16, stored integer 24 represents 1.5, and multiplication requires rescaling the product.

PRACTICE + PASS CRITERION

Implement wrapping and saturating addition plus a fixed-point multiply. Pass: boundary cases are exhaustive for a small width and an error report quantifies rounding behavior against a floating-point reference.

ASSIGNED STUDY

R15: arithmetic; R31: ALU construction; R35: integer arithmetic semantics as a software/hardware contract.

CHECK YOUR UNDERSTANDING

1. What is the signed 8-bit two's-complement range?

2. What does stored value 24 mean with scale 1/16?

3. Why use a wider intermediate for multiplication?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 32.4 • MODULE 32

Clock domains, reset and metastability

Learning goal Recognize where synchronous assumptions stop applying.

An asynchronous input can change near a sampling edge and violate a register's timing requirements, allowing metastable behavior. A synchronizer reduces the probability that instability propagates, but does not make the probability mathematically zero. Digital functional simulation generally does not reproduce analog metastability faithfully.

Single-bit control crossings, pulses and multi-bit data need different protocols. Synchronizing every bit of a changing bus independently does not guarantee a coherent word. Handshakes or asynchronous FIFOs transfer data with explicit ownership and timing assumptions. Reset assertion and deassertion also need a domain-aware policy. A clock-domain-crossing review is a design activity, not only a waveform inspection.

WORKED EXAMPLE

A binary counter changing from 0111 to 1000 flips several bits. Independently sampled bits can briefly form a word that was never the intended source value.

PRACTICE + PASS CRITERION

Model a request/acknowledge crossing and explain its stability assumptions. Pass: the receiver never accepts a partially updated word in the protocol model, and the report states that simulation is not an analog metastability proof.

ASSIGNED STUDY

R15: synchronization and asynchronous inputs; deepen with R50 in later verification lessons.

CHECK YOUR UNDERSTANDING

1. Does a two-register synchronizer eliminate all metastability risk?

2. Why not independently synchronize every bit of a changing binary bus?

3. Can ordinary digital simulation prove an analog metastability failure rate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 33

SystemVerilog and verification-driven RTL design

Prerequisites Module 32; Python and C++ foundations complete.

Four lessons

33.1 RTL semantics and synthesizable SystemVerilog

Write a hardware description rather than a sequential software imitation.

33.2 Testbenches, scoreboards and constrained stimulus

Check behavior against an independent specification.

33.3 Assertions, coverage and formal reasoning

Distinguish observed tests from properties proven under assumptions.

33.4 Synthesis, constraints and FPGA resource mapping

Connect verified behavior to a realizable implementation.

Module project

Implement a parameterized synchronous FIFO in RTL with a Python or C++ reference model, automated tests, assertions and synthesis reports. Keep testbench behavior separate from synthesizable hardware.

Core references

R32 Verilator project — Verilator user guide

R33 YosysHQ — Yosys documentation

R34 cocotb project — cocotb documentation

R50 YosysHQ — SymbiYosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 33.1 • MODULE 33

RTL semantics and synthesizable SystemVerilog

Learning goal Write a hardware description rather than a sequential software imitation.

Register-transfer-level code describes concurrent hardware blocks and state updates. Combinational processes should assign every output on every path unless a latch is intentionally specified. Clocked processes describe stored state. Blocking and nonblocking assignments have simulation scheduling semantics; a common disciplined style uses blocking assignments for combinational calculations and nonblocking assignments for clocked state.

Not every language construct is synthesizable, and supported subsets vary by tool. Delays useful in a testbench are not a portable way to create hardware timing. Width, signedness and implicit extension need explicit review. A loop with a fixed elaboration bound may replicate hardware rather than run sequentially over clock cycles. Read synthesis output to check what was actually inferred.

WORKED EXAMPLE

In a clocked block, a<=b; b<=a; with nonblocking assignments exchanges the old register values at the update event. Sequential software assignment with no temporary would behave differently.

PRACTICE + PASS CRITERION

Implement a counter and two-register swap using a simulator. Pass: tests explain old/new values at each edge, lint warnings are resolved or justified, and no unintended latches are inferred.

ASSIGNED STUDY

R32: language support, warnings and simulation; R33: synthesis concepts and supported constructs.

CHECK YOUR UNDERSTANDING

1. Why assign every combinational output on every path?

2. What does the nonblocking swap example do?

3. Does a source-code loop necessarily take multiple hardware clock cycles?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 33.2 • MODULE 33

Testbenches, scoreboards and constrained stimulus

Learning goal Check behavior against an independent specification.

A testbench drives inputs and observes outputs under a timing protocol. A scoreboard compares accepted transactions with a reference model. The reference should express the specification more simply than the RTL, not copy its internal implementation bug for bug. Directed tests target known boundaries; randomized tests explore combinations within a controlled legal domain.

Record random seeds and failing transaction traces so failures are reproducible. Backpressure, reset during activity and simultaneous operations are important for queues and streaming blocks. Sampling before outputs settle can make a correct design appear wrong; testbench scheduling must match simulator semantics. Verification code deserves review and tests of its own.

WORKED EXAMPLE

A FIFO scoreboard enqueues only when a write handshake is accepted and dequeues only when a read handshake is accepted. Counting attempted writes during full conditions would incorrectly blame the device for dropped data.

PRACTICE + PASS CRITERION

Create a FIFO testbench with empty, full, simultaneous read/write and reset cases. Pass: an intentionally corrupted RTL version is detected, and a seed plus trace reproduces the first mismatch.

ASSIGNED STUDY

R34: writing testbenches, triggers and simulator timing; R32: simulation and tracing.

CHECK YOUR UNDERSTANDING

1. Should a scoreboard count attempted or accepted transactions?

2. Why save random seeds?

3. Why deliberately inject a known bug?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 33.3 • MODULE 33

Assertions, coverage and formal reasoning

Learning goal Distinguish observed tests from properties proven under assumptions.

An assertion states a property that should hold, such as occupancy remaining within bounds. Temporal properties describe relationships over time, including data stability while stalled or eventual completion under assumptions. Coverage asks whether important scenarios occurred or are reachable; it is not the same as correctness.

Formal tools explore a mathematical model under explicit assumptions. Bounded checking proves only the explored bound unless stronger reasoning establishes an unbounded result. Overconstraining inputs can make a broken design pass vacuously. Include cover goals and review assumptions separately from assertions. A proof of one interface property is not proof of the whole chip, toolchain or physical implementation.

WORKED EXAMPLE

If a FIFO depth is 8, assert 0<=occupancy<=8. A separate cover goal should demonstrate reaching full and later returning to empty under legal traffic, so the environment is not constrained to do nothing.

PRACTICE + PASS CRITERION

Add occupancy, ordering and stalled-data assertions. Pass: bounded results state their depth, at least two meaningful cover goals are reached, and each environment assumption is justified.

ASSIGNED STUDY

R50: getting started, FIFO example, safety, cover and liveness; R34: assertion-oriented simulation checks.

CHECK YOUR UNDERSTANDING

1. Does reaching 100% of chosen coverage bins prove correctness?

2. Why can overconstrained assumptions hide a bug?

3. What limitation must accompany a bounded proof?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 33.4 • MODULE 33

Synthesis, constraints and FPGA resource mapping

Learning goal Connect verified behavior to a realizable implementation.

Synthesis transforms a supported hardware description into a logic network mapped to available resources. FPGA resources include lookup tables, registers, memories and specialized arithmetic blocks, depending on the device. Coding style can influence whether a memory or multiplier is inferred efficiently, but reports must confirm the result.

Timing constraints tell implementation tools which clocks and paths matter. Missing or incorrect constraints can make a timing report misleading. Functional simulation, synthesis success and timing closure are separate milestones. Compare equivalent implementations using the same device, constraints and tool settings. A small resource count is not useful if the design fails its target timing or interface behavior.

WORKED EXAMPLE

A queue described as individual registers may use a very different amount of FPGA fabric than one mapped to a supported memory primitive. Both can be functionally correct but differ in ports and read latency.

PRACTICE + PASS CRITERION

Synthesize the FIFO and compare two storage implementations or depths. Pass: record device/model, tool version, inferred memory, registers and timing constraints; reconcile any post-synthesis behavior differences.

ASSIGNED STUDY

R33: synthesis and memory mapping; R32: lint; selected FPGA tool documentation only after a target is chosen.

CHECK YOUR UNDERSTANDING

1. Does successful synthesis establish timing closure?

2. Why inspect inferred memory resources?

3. Why use identical constraints when comparing designs?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 34

Designing a small processor

Prerequisites Modules 16 and 32-33. Implement a documented subset, not an unqualified full-ISA claim.

Four lessons

34.1 Instruction sets, datapaths and single-cycle execution

Separate programmer-visible behavior from its implementation.

34.2 Pipelining, forwarding, stalls and branch recovery

Preserve architectural behavior while overlapping instructions.

34.3 Memory interfaces, caches and protection boundaries

Design memory behavior as a protocol rather than an instantaneous array.

34.4 Processor verification, toolchains and architectural evidence

Demonstrate instruction correctness with reproducible programs and traces.

Module project

Create a simple processor model and RTL implementation for a clearly specified instruction subset. Run the same short programs on an instruction-level reference and the RTL, comparing architectural state at retirement.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R31 Noam Nisan and Shimon Schocken — Nand to Tetris

R35 RISC-V International — RISC-V ISA specifications

R32 Verilator project — Verilator user guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 34.1 • MODULE 34

Instruction sets, datapaths and single-cycle execution

Learning goal Separate programmer-visible behavior from its implementation.

An instruction set defines observable operations, encodings, registers and behavior. A microarchitecture implements that contract through a datapath and control. A simple processor may fetch, decode, read operands, execute and update state in one conceptual cycle. Its clock period is limited by the longest active path, so simplicity of control can cost frequency.

Choose a small supported subset and document unsupported instructions, alignment and exception behavior. A processor executing a few RISC-V-like operations is not automatically a compliant RISC-V implementation. Keep the reference interpreter independent from the RTL. Instruction tests should inspect final and intermediate architectural state rather than only whether a display changes.

WORKED EXAMPLE

An ADD instruction might read registers x2 and x3, add their bit patterns under the ISA rule and write x1. The ISA specifies that result; it does not require a particular adder topology.

PRACTICE + PASS CRITERION

Implement arithmetic, load/store and one branch form in a reference model and datapath. Pass: hand-encoded programs produce the expected register and memory state and unsupported encodings have defined behavior.

ASSIGNED STUDY

R35: selected base integer instructions; R15: processor design; R31: computer architecture projects.

CHECK YOUR UNDERSTANDING

1. How does an ISA differ from a microarchitecture?

2. What limits a simple single-cycle clock period?

3. Why avoid claiming full ISA compliance for a small subset?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 34.2 • MODULE 34

Pipelining, forwarding, stalls and branch recovery

Learning goal Preserve architectural behavior while overlapping instructions.

Pipelining overlaps stages from different instructions to improve throughput. It does not necessarily reduce the latency of one instruction. Data hazards arise when a consumer needs a result that is not yet available; forwarding and stalls resolve them under precise timing rules. Structural hazards occur when instructions compete for a resource.

Control hazards follow branches and other changes in instruction flow. A simple design can stall until the branch is known, while more advanced designs predict and recover. Flushed instructions must not create architectural side effects. Track valid bits and distinguish a bubble from a real instruction encoding. Compare retirement behavior with the reference model after every accepted instruction.

WORKED EXAMPLE

If instruction B uses a register produced by A immediately before it, B may need a forwarded result or a stall. Reading the old register-file value produces a data-dependent failure.

PRACTICE + PASS CRITERION

Pipeline the processor or model a pipeline with explicit valid bits. Pass: dependent arithmetic, load-use, branch-taken and back-to-back memory tests match the architectural reference.

ASSIGNED STUDY

R15: pipelined processors; R35: instruction semantics; R32: tracing and debugging.

CHECK YOUR UNDERSTANDING

1. Does pipelining always shorten one instruction's latency?

2. Why must flushed instructions suppress writes?

3. What is a load-use hazard?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 34.3 • MODULE 34

Memory interfaces, caches and protection boundaries

Learning goal Design memory behavior as a protocol rather than an instantaneous array.

A real memory interface can accept requests and return responses later, with stalls or errors. The processor must preserve request identity and avoid repeating side effects while waiting. Separate instruction/data ports simplify some designs but still require a stated memory model. Memory-mapped device registers may have effects that ordinary RAM does not.

A cache holds copies of memory blocks and needs tag comparison, replacement and write policies. Dirty data must reach the appropriate backing store according to the policy. Address translation and protection are additional functions, not inherent properties of every small CPU. An educational core may deliberately omit an MMU while documenting its consequences and deployment limits.

WORKED EXAMPLE

A store request held high during a multi-cycle stall must not become three separate writes if the interface accepts it only once. A ready/valid handshake defines the acceptance event.

PRACTICE + PASS CRITERION

Add a variable-latency memory model and an optional direct-mapped cache. Pass: randomized stalls preserve program results, each accepted store occurs exactly as specified and cache tests include conflicting addresses.

ASSIGNED STUDY

R15: caches, memory and I/O; R35: memory access and selected privilege concepts as an extension.

CHECK YOUR UNDERSTANDING

1. Why is an instantaneous array an incomplete memory model?

2. What does a cache tag identify?

3. Does a small CPU necessarily need an MMU?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 34.4 • MODULE 34

Processor verification, toolchains and architectural evidence

Learning goal Demonstrate instruction correctness with reproducible programs and traces.

Processor testing combines directed instruction cases, randomized legal programs, reference-model comparison and interface assertions. Compare architectural state at defined retirement points, not every internal signal against a different microarchitecture. A useful failure report includes the program, initial state, seed and first mismatching retirement.

A software toolchain adds assembler, linker layout, startup code and calling-convention assumptions. A C compiler may emit instructions outside a tiny core's subset unless configured and checked. Inspect generated disassembly. Formal or compliance suites can strengthen evidence, but their scope and configuration must be recorded. Passing selected tests is not a universal correctness certificate.

WORKED EXAMPLE

A compiled division operation may call a runtime helper or emit an instruction the core does not support. The executable must be inspected and linked against compatible support code.

PRACTICE + PASS CRITERION

Run at least twenty directed programs and a reproducible randomized set against the reference. Pass: mismatch localization works, binary layout is documented and a supported compiled or assembled program runs end to end.

ASSIGNED STUDY

R35: exact chosen ISA revision; R32: simulation; R15: software/processor interface.

CHECK YOUR UNDERSTANDING

1. Why compare at retirement rather than arbitrary internal cycles?

2. Why inspect compiler-generated instructions?

3. What belongs in a reproducible processor failure report?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 35

FPGA-based game subsystems and integration

Prerequisites Modules 21, 31-34. Physical deployment is optional; simulated I/O satisfies the core project.

Four lessons

35.1 Memory-mapped I/O, timers, interrupts and DMA

Explain how software controls asynchronous hardware.

35.2 Framebuffers, scanout and pixel pipelines

Produce timed pixel data with correct storage and scheduling.

35.3 A small accelerator and its software contract

Measure whether specialized hardware improves a complete operation.

35.4 Integrated digital-design review and optional board bring-up

Close the loop between specification, verification and implementation.

Module project

Gate G: demonstrate a verified RTL game subsystem, such as a sprite compositor, controller interface or bounded vector unit. Include a software reference, interface specification, automated verification and synthesis evidence. A physical board is an extension, not a prerequisite.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R32 Verilator project — Verilator user guide

R33 YosysHQ — Yosys documentation

R34 cocotb project — cocotb documentation

R35 RISC-V International — RISC-V ISA specifications

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 35.1 • MODULE 35

Memory-mapped I/O, timers, interrupts and DMA

Learning goal Explain how software controls asynchronous hardware.

Memory-mapped I/O exposes device control through addressable registers with defined access widths and side effects. Reads can acknowledge status or pop data; writes can launch work. Ordinary RAM assumptions may therefore be unsafe. Language volatile alone does not provide inter-thread synchronization or every device-ordering guarantee; use the platform's specified access and barrier mechanisms.

Timers and interrupts notify software of events, while direct memory access moves data without having the CPU copy every element. DMA still needs valid buffers, ownership, addressing and cache-coherence handling. An interrupt handler should perform bounded urgent work and defer expensive processing. A device that can stall or fail requires timeout and recovery policy.

WORKED EXAMPLE

A DMA engine reading a buffer cannot safely use it after software frees or repurposes it. Completion must transfer ownership back before reuse.

PRACTICE + PASS CRITERION

Implement a simulated timer and memory-mapped control block. Pass: register side effects are specified, interrupts are acknowledged without losing new events and buffers remain owned until completion.

ASSIGNED STUDY

R15: I/O and interrupts; R35: ordering concepts for the chosen platform; R32: device testbench modeling.

CHECK YOUR UNDERSTANDING

1. Why are device registers not ordinary memory variables?

2. When may software reuse a DMA source buffer?

3. Does C++ volatile make a shared device driver thread-safe?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 35.2 • MODULE 35

Framebuffers, scanout and pixel pipelines

Learning goal Produce timed pixel data with correct storage and scheduling.

A framebuffer stores pixels, while scanout emits them according to a display timing contract. Active pixels may occupy only part of a complete line/frame interval; blanking and synchronization are additional timing responsibilities. Use documented board/display interfaces rather than inventing electrical signaling. A simulator can capture emitted pixels into image files without any display hardware.

Pixel pipelines must deliver data at the required rate, including memory latency. Double buffering separates the image being displayed from the one being written, but swapping needs a safe synchronization point to avoid tearing. Resolution, color depth and refresh rate determine storage and minimum active-data bandwidth. Real interfaces add overhead and may require more bandwidth.

WORKED EXAMPLE

A 320 x 240 framebuffer at 16 bits per pixel needs 153,600 bytes. Reading only active pixels at 60 frames per second requires 9.216 MB/s, excluding blanking and other traffic.

PRACTICE + PASS CRITERION

Create a simulated scanout counter and color-bar/sprite generator. Pass: exact active dimensions and frame boundaries are verified, captured images match the reference and buffer swapping occurs only at the declared boundary.

ASSIGNED STUDY

R15: digital-system timing; R32 and R34: timed testbenches; use the selected board vendor's documented video reference only for a physical extension.

CHECK YOUR UNDERSTANDING

1. How many bytes does the example framebuffer require?

2. Is active-pixel bandwidth the whole physical interface bandwidth?

3. Why synchronize framebuffer swaps?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 35.3 • MODULE 35

A small accelerator and its software contract

Learning goal Measure whether specialized hardware improves a complete operation.

An accelerator implements a selected computation in dedicated hardware. Its value depends on data reuse, parallelism, setup, transfer and synchronization costs. A tiny arithmetic kernel may run quickly in isolation yet lose to the CPU after communication. Choose a bounded operation with a clear reference, such as vector addition, sprite blending or a fixed-size filter.

Specify input format, address limits, alignment, completion, errors and cancellation. Fixed-point arithmetic must include rounding and saturation rules. Backpressure lets downstream capacity limit upstream production without dropping data. Measure latency to first result, steady-state throughput and total end-to-end time separately. A faster clock is only one possible improvement.

WORKED EXAMPLE

If setup and transfer take 80 microseconds and computation takes 20, the operation takes 100 microseconds. Halving computation alone reduces it to 90, not 50.

PRACTICE + PASS CRITERION

Build a streaming arithmetic or pixel block plus a software driver model. Pass: outputs match the reference over boundary cases, backpressure preserves order and end-to-end cost includes setup and transfer.

ASSIGNED STUDY

R33: synthesis and arithmetic mapping; R34: scoreboard verification; R15: parallelism and I/O.

CHECK YOUR UNDERSTANDING

1. What total time follows from halving the example 20 microsecond compute phase?

2. What does backpressure accomplish?

3. Why report latency and throughput separately?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 35.4 • MODULE 35

Integrated digital-design review and optional board bring-up

Learning goal Close the loop between specification, verification and implementation.

Integration checks clocks, resets, widths, address maps and ownership across blocks. A design can pass every isolated unit test and still fail because two units disagree about a handshake or reset sequence. Maintain an interface table and test the complete path with randomized stalls and reset events.

For a physical extension, use a documented educational board, approved power source and reviewed pin constraints. Verify voltage standards before connecting peripherals. Begin with a known-good reference and add one subsystem at a time. Simulation and FPGA operation provide different evidence; neither validates an ASIC fabrication flow by itself. The final report should clearly label simulated, synthesized and physically measured results.

WORKED EXAMPLE

A functional model may assume memory responds in one cycle while an integrated controller takes several. An end-to-end randomized-latency test exposes that assumption before hardware deployment.

PRACTICE + PASS CRITERION

Complete Gate G with an integration test, synthesis report and short demonstration. Pass: another person reruns the simulation, all timing assumptions are listed and any physical measurements are distinguished from modeled estimates.

ASSIGNED STUDY

R32-R34: repeatable simulation and synthesis; R15: system integration; handbook safety review for physical extensions.

CHECK YOUR UNDERSTANDING

1. Why are unit tests insufficient for integration?

2. What must be checked before wiring a peripheral?

3. Does a successful FPGA demonstration prove an ASIC is ready to fabricate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 08 OF 10

Gaming Platforms and Hardware/Software Co-design

Upper undergraduate CE / capstone

Study boards, memory, GPU architecture, performance, input/display pipelines and integrated design.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 36 Embedded platforms, drivers and board design

36.1 Serial interfaces and peripheral contracts • 36.2 Firmware, RTOS scheduling and driver lifecycle • 36.3 Schematics, PCB layout, power integrity and signal integrity • 36.4 Bring-up, instrumentation and fault isolation

Module 37 GPU microarchitecture and memory systems

37.1 SIMT execution, divergence, occupancy and latency hiding • 37.2 Coalescing, local storage, caches and shared-memory banks • 37.3 Raster, texture, matrix and ray-tracing units • 37.4 CPU-GPU sharing, discrete memory and synchronization

Module 38 Quantitative performance engineering

38.1 Amdahl, roofline and bottleneck models • 38.2 Profilers, counters and trustworthy benchmark design • 38.3 SIMD, cache blocking, NUMA and job scheduling • 38.4 Energy, thermal behavior and sustained performance

Module 39 Gaming-platform constraints and interactive latency

39.1 Input-to-display latency and frame pacing • 39.2 Mobile and handheld systems: memory, power and lifecycle • 39.3 XR spaces, stereo views, pose timing and comfort constraints • 39.4 Portability, platform services and console-development boundaries

Module 40 Hardware/software co-design capstone

40.1 Workload characterization and measurable requirements • 40.2 Architecture alternatives, arithmetic precision and dataflow • 40.3 Interface design, virtualization and end-to-end evaluation • 40.4 Design review, evidence tiers and undergraduate CE completion

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 36

Embedded platforms, drivers and board design

Prerequisites Modules 17-18 and 31-35. Physical work remains optional and supervised.

Four lessons

36.1 Serial interfaces and peripheral contracts

Choose a communication interface using timing and electrical constraints.

36.2 Firmware, RTOS scheduling and driver lifecycle

Bound embedded work and make device state transitions explicit.

36.3 Schematics, PCB layout, power integrity and signal integrity

Translate a circuit into a manufacturable layout without ignoring physics.

36.4 Bring-up, instrumentation and fault isolation

Turn an unresponsive device into a sequence of testable hypotheses.

Module project

Design a simulated controller or peripheral with firmware, a host interface and a schematic-level review. A physical board extension begins only after the electrical, layout and bring-up plans receive qualified review.

Core references

R36 KiCad project — KiCad documentation

R51 Linux kernel contributors — Driver implementer API guide

R53 Zephyr Project — Zephyr documentation

R30 MIT OpenCourseWare — 6.002 Circuits and Electronics

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 36.1 • MODULE 36

Serial interfaces and peripheral contracts

Learning goal Choose a communication interface using timing and electrical constraints.

UART, SPI and I2C offer different ways to exchange information. UART communication depends on agreed framing and baud timing without a shared clock. SPI commonly uses a controller-provided clock and device-select signals, with mode and timing choices. I2C uses addressed devices and shared lines with electrical and protocol requirements such as pull-ups. These names do not specify every device-specific command or voltage level.

A robust driver documents pin assignment, voltage domain, clock limits, transfer ordering, error conditions and device register semantics. USB and PCIe add more complex enumeration and protocol layers; use supported controllers and documented software interfaces rather than treating them as generic serial wires. Validate a protocol model before attaching physical devices.

WORKED EXAMPLE

Two SPI endpoints can use the same bit rate but disagree on clock phase, sampling the wrong edge. Matching only the frequency is not enough to establish compatibility.

PRACTICE + PASS CRITERION

Build a simulated peripheral transaction decoder and driver for one interface. Pass: incorrect framing or mode is detected, timeouts are bounded and a complete interface-contract table is provided.

ASSIGNED STUDY

R53: device drivers, UART, SPI and I2C APIs; R51: bus-level driver documentation. For hardware, add the exact peripheral datasheet.

CHECK YOUR UNDERSTANDING

1. Why can two SPI devices fail at the same clock rate?

2. Does an interface name guarantee voltage compatibility?

3. Why begin with one interface rather than every bus?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 36.2 • MODULE 36

Firmware, RTOS scheduling and driver lifecycle

Learning goal Bound embedded work and make device state transitions explicit.

Firmware initializes hardware, configures peripherals and runs application behavior under platform constraints. A real-time operating system provides scheduling and synchronization mechanisms, but deadlines still depend on task design, interrupt load and resource contention. Assign responsibilities to interrupt context, worker threads and periodic tasks deliberately.

A driver lifecycle includes initialization, use, suspend/resume, removal and failure. Device state may disappear while a request is pending. Buffer ownership and cancellation must remain correct across those transitions. Watchdogs detect certain failures only when serviced meaningfully; repeatedly feeding a watchdog from a stuck loop defeats the intended supervision. Kernel experiments belong in disposable or simulated environments, not a primary computer.

WORKED EXAMPLE

A high-priority input task that waits on a low-priority logging task can miss its deadline despite using an RTOS. Moving logging off the critical path addresses the structural problem.

PRACTICE + PASS CRITERION

Use a supported RTOS simulator or a deterministic scheduler model to implement input, processing and logging tasks. Pass: deadlines are evaluated under injected load and removal/cancellation leaves no live invalid requests.

ASSIGNED STUDY

R53: kernel scheduling, synchronization and device model; R51: driver lifecycle and power management.

CHECK YOUR UNDERSTANDING

1. Does an RTOS automatically make all tasks meet deadlines?

2. Why not feed a watchdog unconditionally from a stuck loop?

3. What must happen to outstanding requests when a device disappears?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 36.3 • MODULE 36

Schematics, PCB layout, power integrity and signal integrity

Learning goal Translate a circuit into a manufacturable layout without ignoring physics.

A schematic specifies electrical connectivity and component intent; a PCB layout determines physical geometry and interconnect. Footprint pin numbering, connector orientation and power-domain assignment must match actual parts. Electrical-rule and design-rule checks catch some mistakes, but they cannot prove the circuit is sensible or safe.

Fast edges make traces and return paths important even when the logical clock seems modest. Signal integrity concerns waveform quality, impedance, reflections and crosstalk; power integrity concerns supply stability under changing current demand. Decoupling, plane continuity, stackup and placement are interconnected choices. High-speed memory and serial links need deeper analysis and qualified review. Start with a low-speed educational design, not a first-attempt graphics card.

WORKED EXAMPLE

A correct net name does not guarantee a usable signal if the physical return path is badly interrupted. Conversely, a visually tidy board can still have swapped connector pins or incompatible voltage domains.

PRACTICE + PASS CRITERION

Create a low-speed peripheral schematic and unrouted or simple two-layer layout in KiCad. Pass: footprints are checked against datasheets, ERC/DRC findings are resolved or justified and a reviewer inspects power and return paths before fabrication.

ASSIGNED STUDY

R36: schematic editor, PCB editor, ERC and DRC; R30: nonideal circuit behavior; R49: interconnect and packaging extension.

CHECK YOUR UNDERSTANDING

1. Does passing DRC prove electrical correctness?

2. Why do return paths matter?

3. Why verify footprints against datasheets?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 36.4 • MODULE 36

Bring-up, instrumentation and fault isolation

Learning goal Turn an unresponsive device into a sequence of testable hypotheses.

Bring-up checks power, clocks, reset, communication and function in an intentional order. A known-good firmware image and a minimal heartbeat separate basic execution from the full application. Logic analyzers observe digital timing under their bandwidth and threshold limits; oscilloscopes reveal analog waveform behavior but require safe probing and interpretation.

Use current-limited approved low-voltage supplies only under a reviewed plan for physical work. Do not probe mains or modify high-energy power systems. Record expected observations before powering hardware and stop when results exceed safe limits. A fault tree helps separate software, interface and electrical hypotheses. Preserve evidence and change one variable at a time rather than repeatedly reflashing and rewiring at random.

WORKED EXAMPLE

If no serial output appears, verify the expected voltage domain, correct pins, configured baud rate, reset state and program execution before assuming the application algorithm is wrong.

PRACTICE + PASS CRITERION

Write and rehearse a bring-up procedure against a simulated device with injected clock, reset and protocol faults. Pass: each fault has a safe diagnostic branch and the procedure includes explicit stop conditions.

ASSIGNED STUDY

R53: board support and debugging; R36: fabrication outputs and design review; selected instrument manuals for any physical extension.

CHECK YOUR UNDERSTANDING

1. Why test power, clock and reset before complex application behavior?

2. What is the value of an expected-observation table?

3. What should happen when a physical observation exceeds a reviewed safe limit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 37

GPU microarchitecture and memory systems

Prerequisites Modules 16, 22, 32 and 34.

Four lessons

37.1 SIMT execution, divergence, occupancy and latency hiding

Explain why many threads do not guarantee useful parallel work.

37.2 Coalescing, local storage, caches and shared-memory banks

Match data layout to the memory transactions actually performed.

37.3 Raster, texture, matrix and ray-tracing units

Understand fixed-function acceleration as specialized tradeoffs.

37.4 CPU-GPU sharing, discrete memory and synchronization

Treat shared addressability separately from ownership and cost.

Module project

Create a GPU architecture workbook connecting shader behavior to execution groups, registers, memory transactions and fixed-function resources. Measure one accessible GPU or clearly label a simulator-only study.

Core references

R37 NVIDIA — CUDA Programming Guide

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R21 Khronos Group — Vulkan tutorial

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 37.1 • MODULE 37

SIMT execution, divergence, occupancy and latency hiding

Learning goal Explain why many threads do not guarantee useful parallel work.

GPUs organize work into groups that execute under architecture-specific scheduling rules. SIMT exposes individual threads while grouping their instruction execution. Divergent control flow can require different paths to be executed for different active lanes, reducing useful work. Exact group size and behavior depend on the target architecture and API capabilities.

Occupancy describes how much execution state is resident relative to a hardware limit, not the fraction of arithmetic units doing useful work. Registers, shared memory and block size can constrain residency. More resident work can help hide latency, but maximizing occupancy may increase spills or reduce per-thread efficiency. Profile the actual kernel and distinguish occupancy, utilization and achieved throughput.

WORKED EXAMPLE

A group with half its lanes taking an expensive branch and half taking another may perform both paths with different active masks. The cost is not necessarily the average of the two branches.

PRACTICE + PASS CRITERION

Compare coherent and divergent workloads, varying register pressure or group size where supported. Pass: outputs agree, device group assumptions are recorded and the report does not equate occupancy with performance.

ASSIGNED STUDY

R37: programming model, execution and hardware implementation; R48: vector and multithreaded architecture foundations.

CHECK YOUR UNDERSTANDING

1. What is branch divergence in grouped GPU execution?

2. Does high occupancy guarantee high throughput?

3. Why can reducing register use sometimes hurt?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 37.2 • MODULE 37

Coalescing, local storage, caches and shared-memory banks

Learning goal Match data layout to the memory transactions actually performed.

Adjacent threads accessing adjacent locations can allow efficient grouped memory transactions. Strided or scattered access may require more transactions for the same useful bytes. Alignment, element size and architecture rules influence the result. Do not infer transaction count solely from source-code load count.

On-chip shared storage can enable reuse and communication within a group, but its banks and synchronization requirements matter. Bank conflicts can serialize accesses under the hardware's rules. Caches help some patterns and not others; a cache hit also has a finite cost. Data-layout changes must preserve semantics and include conversion overhead when comparing end-to-end performance.

WORKED EXAMPLE

If a group needs neighboring float values but the structure stores a large unrelated record between them, the useful bytes can be spread across many transactions. A structure-of-arrays layout may reduce wasted traffic for that access pattern.

PRACTICE + PASS CRITERION

Benchmark contiguous, strided and tiled access with identical useful work. Pass: count useful bytes, state the relevant memory level and include any layout-conversion cost in the application comparison.

ASSIGNED STUDY

R37: device memory access, shared memory and performance considerations; R43: operational intensity.

CHECK YOUR UNDERSTANDING

1. What does coalescing improve?

2. Why can shared memory still cause stalls?

3. Why include layout conversion in a comparison?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 37.3 • MODULE 37

Raster, texture, matrix and ray-tracing units

Learning goal Understand fixed-function acceleration as specialized tradeoffs.

GPUs contain resources specialized for operations such as rasterization, texture filtering or particular matrix arithmetic; some also accelerate parts of ray tracing. The presence, capability and limits of these units vary. A shader can become limited by a specialized unit rather than general arithmetic.

Specialization trades flexibility for efficiency. Supported formats, tile shapes, precision and data movement determine whether an operation maps well. Ray-tracing acceleration does not remove shader work, scene-build cost or memory traffic. Matrix throughput figures are meaningful only with their stated data types and operation-count conventions. Compare equivalent numerical quality and complete work, not incompatible headline peaks.

WORKED EXAMPLE

A fast low-precision matrix path may require packing, conversion and an accuracy check. For a tiny matrix, those costs can outweigh the accelerated arithmetic.

PRACTICE + PASS CRITERION

Analyze one texture-heavy, arithmetic-heavy or ray-query workload. Pass: identify the likely resource limits using measurements or a documented model and state supported precision and feature assumptions.

ASSIGNED STUDY

R37: architecture and specialized operations; R21: graphics/compute capabilities; R23: ray intersection and acceleration reference.

CHECK YOUR UNDERSTANDING

1. Does a ray-tracing unit eliminate the rest of rendering work?

2. Why compare the same numerical precision or quality?

3. When might specialized hardware lose on a small workload?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 37.4 • MODULE 37

CPU-GPU sharing, discrete memory and synchronization

Learning goal Treat shared addressability separately from ownership and cost.

A discrete GPU may have memory reached through explicit transfers, while integrated or unified-memory systems expose different sharing arrangements. Shared addressing does not imply uniform access cost or automatic synchronization. Data migration, cache behavior and coherency rules depend on the platform and API.

Ownership still matters: the CPU must not modify data while the GPU consumes it unless the contract permits that access. DMA and cache maintenance may require specific operations. Pinned or mapped memory has tradeoffs and resource limits. Measure transfer, compute and synchronization separately, then report the end-to-end path. A portable engine should choose strategies by capabilities and workload rather than assuming every GPU behaves like one desktop card.

WORKED EXAMPLE

An integrated GPU and CPU can share physical memory yet contend for the same bandwidth. Eliminating an explicit copy does not create unlimited throughput.

PRACTICE + PASS CRITERION

Compare an upload/compute/readback path with an available shared-memory path, or model both with stated assumptions. Pass: correct ownership, separate phase timings and no claim that unified means zero-cost.

ASSIGNED STUDY

R37: memory management and synchronization; R21, R24 or R25: the selected platform's resource-sharing contract.

CHECK YOUR UNDERSTANDING

1. Does shared addressing mean zero data-access cost?

2. Why is CPU modification during GPU use risky?

3. What must an end-to-end accelerator benchmark include?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 38

Quantitative performance engineering

Prerequisites Modules 09, 16, 18, 30 and 37.

Four lessons

38.1 Amdahl, roofline and bottleneck models

Use upper bounds to reject implausible optimization claims.

38.2 Profilers, counters and trustworthy benchmark design

Distinguish observation from an explanation that still needs testing.

38.3 SIMD, cache blocking, NUMA and job scheduling

Optimize locality and parallelism together.

38.4 Energy, thermal behavior and sustained performance

Evaluate useful work over time rather than only a short peak.

Module project

Produce a performance investigation with a hypothesis, controlled workload, raw measurements, uncertainty, a bottleneck model and a verified optimization. A negative result is acceptable when the evidence is rigorous.

Core references

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R37 NVIDIA — CUDA Programming Guide

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R26 Epic Games — Programming with C++ in Unreal Engine

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 38.1 • MODULE 38

Amdahl, roofline and bottleneck models

Learning goal Use upper bounds to reject implausible optimization claims.

Amdahl's law limits total speedup when only part of a workload improves. For serial fraction s and ideal acceleration p of the remainder, speedup is 1/(s+(1-s)/p). This simplified model assumes the work decomposition remains valid and ignores new overhead unless added explicitly.

A roofline model bounds throughput by compute capacity and memory bandwidth times operational intensity. Define intensity at a particular memory level using actual data movement, not only source-level array sizes. The model is a ceiling, not a performance prediction or proof of the bottleneck. Latency, dependencies, occupancy and instruction mix can keep measured results below both roofs.

WORKED EXAMPLE

For a 1,000 GFLOP/s compute ceiling, 100 GB/s bandwidth and intensity 2 FLOP/byte, the bandwidth roof is 200 GFLOP/s. The simplified upper bound is min(1000,200)=200 GFLOP/s.

PRACTICE + PASS CRITERION

Build a roofline worksheet for one kernel and an Amdahl estimate for one frame optimization. Pass: units cancel correctly, modeled ceilings use the declared memory level and observed deviations are explained as hypotheses.

ASSIGNED STUDY

R43: original Roofline model; R48: quantitative architecture; revisit Module 18 parallel limits.

CHECK YOUR UNDERSTANDING

1. What is the example roofline bound?
2. What is the maximum speedup with 20% unchanged serial work and infinitely fast remaining work?
3. Does being below the roof prove the model is wrong?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 38.2 • MODULE 38

Profilers, counters and trustworthy benchmark design

Learning goal Distinguish observation from an explanation that still needs testing.

Hardware counters and traces expose events such as cache misses, stalls and executed work under tool-specific definitions. Counters can be multiplexed, sampled or unavailable. A metric name does not guarantee identical meaning across vendors or generations. Read the collection method and avoid treating one counter as a complete causal explanation.

Benchmark design controls workload, warmup, power state, compiler options and timing boundaries. Prevent dead-code elimination by validating outputs and making results observable. Repeated trials should be sufficiently independent for the statistical claim. Preserve raw data and record failed or anomalous runs with a declared exclusion policy. Compare medians or distributions appropriately rather than cherry-picking the best run.

WORKED EXAMPLE

A loop whose result is never used may be optimized away, making a timing appear spectacular. Checking a result checksum outside the measured region helps establish that the intended work occurred.

PRACTICE + PASS CRITERION

Create a benchmark harness with warmup, repeated runs, output validation and metadata. Pass: an intentional no-op or wrong-result variant is rejected and all included/excluded samples are documented.

ASSIGNED STUDY

R26: profiling; R37: performance analysis; R09: inference and uncertainty.

CHECK YOUR UNDERSTANDING

1. Why validate benchmark output?

2. Are similarly named counters identical across all processors?

3. Why declare an exclusion policy before inspecting results?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 38.3 • MODULE 38

SIMD, cache blocking, NUMA and job scheduling

Learning goal Optimize locality and parallelism together.

SIMD exploits regular operations across lanes, while threading distributes work across cores. Cache blocking reorganizes computation to reuse data before eviction. Each technique can alter numerical ordering and needs correctness tolerances or exactness requirements. Avoid undefined aliasing or out-of-bounds tricks to obtain a benchmark win.

Non-uniform memory access systems can make data placement and worker affinity important, but binding threads without measurement can also reduce flexibility. Job granularity balances scheduling overhead against parallel opportunities and load balance. False sharing can cause cache-coherence traffic even when threads modify different logical variables. Optimize the actual data ownership pattern, not only the number of threads.

WORKED EXAMPLE

A job requiring 1 microsecond of useful work with 2 microseconds of scheduling overhead is a poor unit of work under that model. Batching can improve efficiency, but overly large batches can create an imbalanced tail.

PRACTICE + PASS CRITERION

Optimize a particle or transform workload using one locality change and one scheduling change. Pass: test tail sizes, report numerical differences, and measure small as well as large workloads to expose overhead.

ASSIGNED STUDY

R37: parallel memory patterns; R48: multiprocessors and memory systems; R11: safe C++ and concurrency.

CHECK YOUR UNDERSTANDING

1. What is false sharing?

2. Why can very small jobs be inefficient?

3. Why test small workloads after a parallel optimization?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 38.4 • MODULE 38

Energy, thermal behavior and sustained performance

Learning goal Evaluate useful work over time rather than only a short peak.

Power is an instantaneous rate of energy use; energy is its integral over time. A design consuming more power can still use less energy for a task if it finishes sufficiently sooner, while sustained operation can be limited by thermal conditions. Performance-per-watt claims need a measurement boundary and a workload definition.

Short benchmark bursts may not represent steady operation after temperature and frequency settle. Record ambient or available temperature conditions, duration and power source without overriding safety protections. Software counters estimate power differently from external instruments; label the method and uncertainty. Dynamic quality scaling can preserve experience within a thermal budget, but should avoid distracting oscillation or violating simulation correctness.

WORKED EXAMPLE

Ignoring idle transitions, a task at 50 W for 2 seconds uses 100 J, while 80 W for 1 second uses 80 J. Lower instantaneous power is not automatically lower task energy.

PRACTICE + PASS CRITERION

Run a sustained workload within normal device limits or analyze a synthetic power trace. Pass: distinguish watts from joules, report the measurement boundary and compare early versus sustained performance.

ASSIGNED STUDY

R48: quantitative architecture; R51: device power management; R37: GPU execution and performance context.

CHECK YOUR UNDERSTANDING

1. How much energy is 50 W for 2 seconds?

2. Which example uses less task energy?

3. Why not disable thermal protections for a curriculum benchmark?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 39

Gaming-platform constraints and interactive latency

Prerequisites Modules 22, 27, 29 and 36-38.

Four lessons

39.1 Input-to-display latency and frame pacing

Account for the whole interaction path rather than only frame rate.

39.2 Mobile and handheld systems: memory, power and lifecycle

Design for suspension, limited budgets and sustained use.

39.3 XR spaces, stereo views, pose timing and comfort constraints

Render from runtime-provided timing and coordinate contracts.

39.4 Portability, platform services and console-development boundaries

Build transferable systems without assuming proprietary access.

Module project

Create a platform capability and latency report for the engine. Implement one non-desktop constraint in simulation or on an available device: touch input, suspend/resume, restricted memory, variable presentation or XR timing.

Core references

R22 SDL project — SDL3 documentation

R24 Microsoft — Direct3D 12 graphics

R25 Apple — Metal developer resources

R29 Microsoft — Xbox Accessibility Guidelines

R52 Khronos Group — OpenXR registry and specifications

R53 Zephyr Project — Zephyr documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 39.1 • MODULE 39

Input-to-display latency and frame pacing

Learning goal Account for the whole interaction path rather than only frame rate.

Input-to-display latency includes sensing, device polling, event delivery, simulation scheduling, rendering, queues, scanout and display response. These stages can overlap and their timing depends on when an input arrives. A high average frame rate is not a complete latency measurement. Distinguish throughput, frame pacing and the age of the displayed input state.

Buffering can improve utilization while increasing latency. Synchronization and presentation modes affect tearing, pacing and queue depth under platform-specific rules. Variable refresh can help presentation timing within supported ranges but does not remove CPU or GPU work. Measure an end-to-end path with appropriate instrumentation or clearly label a timing model as an estimate.

WORKED EXAMPLE

At 60 Hz, one full frame interval is about 16.67 ms. Two extra queued frame intervals can add roughly 33.3 ms of delay in a simplified model, even though the output still averages 60 frames per second.

PRACTICE + PASS CRITERION

Instrument input receipt, simulation consumption, render submission and presentation-related timestamps. Pass: explain which stages are actually measured and which remain estimated, and show a frame-pacing distribution.

ASSIGNED STUDY

R22: input and event timing; R24 or R25: presentation; R52: frame timing as an advanced comparison.

CHECK YOUR UNDERSTANDING

1. Is 60 frames per second a complete latency measurement?

2. What is two 60 Hz frame intervals?

3. Why label timestamp boundaries precisely?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 39.2 • MODULE 39

Mobile and handheld systems: memory, power and lifecycle

Learning goal Design for suspension, limited budgets and sustained use.

Mobile and handheld devices impose different input, memory, thermal and lifecycle constraints from an always-active desktop process. An application may lose focus, suspend or need to restore resources. Save enough state under the platform contract without assuming unlimited time during shutdown. Touch interaction and small displays require different interface testing.

Tile-oriented rendering architectures and shared memory can change the cost of passes and attachments, but device behavior must be verified rather than inferred from a label. Sustained frame targets should respect power and temperature limits. Quality tiers can adjust rendering work while preserving game rules and readability. Test low-memory and interrupted-session behavior deliberately.

WORKED EXAMPLE

A renderer that reloads every texture synchronously after resume may cause a long stall or memory spike. Staged restoration with placeholders can provide a more controlled return to play.

PRACTICE + PASS CRITERION

Add suspend/resume simulation and a constrained memory mode. Pass: progress survives the documented lifecycle, resources are restored safely and quality changes do not alter authoritative simulation rules.

ASSIGNED STUDY

R25: platform graphics resources; R22: application lifecycle events; R29: text and input usability; R51: power-management concepts.

CHECK YOUR UNDERSTANDING

1. Why not assume shutdown always provides unlimited save time?

2. Why measure sustained rather than only initial performance?

3. What should rendering quality tiers preserve?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 39.3 • MODULE 39

XR spaces, stereo views, pose timing and comfort constraints

Learning goal Render from runtime-provided timing and coordinate contracts.

Extended-reality rendering combines multiple views with tracked poses and runtime-managed presentation. A pose belongs to a coordinate space and a time. Using a correct position from the wrong space or an outdated time can still produce visibly incorrect motion. View configuration, field of view and projection should come from the runtime contract rather than hard-coded assumptions.

Prediction and late-stage updates reduce some effects of tracking and rendering delay, but they do not eliminate errors or every source of discomfort. Foveated rendering allocates detail according to a policy and available capabilities, with quality and tracking implications. Keep a desktop simulated-view route for learning before purchasing a headset. Physical testing must be voluntary, brief and stopped when uncomfortable; no medical benefit is implied.

WORKED EXAMPLE

A head pose sampled for an earlier frame should not be described as the pose at display time. The runtime's predicted display timing establishes which estimate the application should use.

PRACTICE + PASS CRITERION

Implement two simulated camera views and a pose-time log, or a minimal OpenXR scene on available supported hardware. Pass: spaces and units are explicit, frame timing follows the runtime and loss of tracking has defined behavior.

ASSIGNED STUDY

R52: spaces, view configuration, frame loop and timing; R29: interaction and accessibility review.

CHECK YOUR UNDERSTANDING

1. Why is a pose incomplete without a space and time?

2. Should stereo projection be hard-coded for every headset?

3. Does prediction remove all latency or discomfort risk?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 39.4 • MODULE 39

Portability, platform services and console-development boundaries

Learning goal Build transferable systems without assuming proprietary access.

Portability starts with explicit contracts for input, files, threading, graphics, audio and lifecycle. A platform layer should isolate differences while allowing necessary capabilities to remain visible. A lowest-common-denominator abstraction can hide performance opportunities, while scattered platform conditionals can make maintenance fragile. Test the boundary with a second implementation or a mock.

Console development may involve access-controlled SDKs, agreements and platform-specific requirements. Public self-study cannot provide those permissions or substitute for official documentation. Learn transferable concepts through desktop, mobile or open hardware, then use authorized platform materials when available. Record dependency licenses and distribution constraints for each target. Do not use leaked SDKs or assume technical compatibility grants distribution rights.

WORKED EXAMPLE

A filesystem interface must distinguish persistent saves from packaged read-only assets. Mapping both to one desktop directory can conceal a portability problem until deployment.

PRACTICE + PASS CRITERION

Write a platform interface and capability matrix, then implement a second backend or strict mock. Pass: assumptions about paths, input and lifecycle are exposed, and proprietary requirements are marked unavailable until authorized.

ASSIGNED STUDY

R22, R24 and R25: platform interfaces; R29: cross-device usability; official authorized platform documentation only for restricted targets.

CHECK YOUR UNDERSTANDING

1. Why can a strict mock help portability?

2. Does public self-study grant access to proprietary console SDKs?

3. Why separate packaged assets from persistent saves?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 40

Hardware/software co-design capstone

Prerequisites Gates F and G; Modules 36-39.

Four lessons

40.1 Workload characterization and measurable requirements

Derive hardware goals from a real software operation.

40.2 Architecture alternatives, arithmetic precision and dataflow

Explore several designs before committing to one.

40.3 Interface design, virtualization and end-to-end evaluation

Include the host software in the hardware result.

40.4 Design review, evidence tiers and undergraduate CE completion

Defend a co-design decision with limitations and reproducible artifacts.

Module project

Gate H: choose one game workload and compare a software baseline with a specialized hardware or architecture alternative. Deliver a reference implementation, model or RTL, software interface, verification evidence and an end-to-end tradeoff report.

Core references

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R38 gem5 project — gem5 documentation

R33 YosysHQ — Yosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 40.1 • MODULE 40

Workload characterization and measurable requirements

Learning goal Derive hardware goals from a real software operation.

Co-design begins with a workload, not a favorite chip block. Identify input sizes, data distributions, frequency of use, latency needs, quality constraints and failure behavior. Measure the baseline and determine whether the operation limits the complete application. A subsystem with an attractive microbenchmark may be irrelevant to the frame critical path.

Requirements should include throughput, worst supported latency, numerical error, storage, interface costs and maintainability. Separate hard constraints from preferences. Choose representative and adverse inputs, keeping a held-out set for final evaluation. A successful design may be one that rejects unnecessary hardware after showing that a software improvement is cheaper and sufficient.

WORKED EXAMPLE

Accelerating a 0.2 ms operation in a 20 ms frame can save at most 1% of frame time under a simple additive model. A larger memory or synchronization bottleneck may deserve attention first.

PRACTICE + PASS CRITERION

Select a workload from the engine and write a requirement sheet plus baseline measurements. Pass: each target has units and a test, and the maximum plausible application benefit is estimated.

ASSIGNED STUDY

R43: bottleneck modeling; R48: quantitative design; R49: project methodology.

CHECK YOUR UNDERSTANDING

1. Why start with workload measurements?

2. What maximum fraction of the example frame can removing 0.2 ms save?

3. Can rejecting a hardware proposal be a successful outcome?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 40.2 • MODULE 40

Architecture alternatives, arithmetic precision and dataflow

Learning goal Explore several designs before committing to one.

A design space includes parallel lanes, pipeline depth, local storage, data reuse, arithmetic precision and the boundary between hardware and software. Each choice affects more than one metric. Wider parallelism can demand more memory bandwidth; deeper pipelines can improve frequency while increasing latency and control complexity.

Quantization requires an error budget tied to the application. Test saturation, rounding and extreme values against a reference. A dataflow diagram should show where values are produced, reused and transferred. Analytical models help eliminate infeasible candidates before expensive implementation. Record assumptions and compare a simple baseline with at least two alternatives rather than presenting one design as inevitable.

WORKED EXAMPLE

Doubling arithmetic lanes does not double useful throughput if the memory interface can supply only the original amount of data and the computation has no additional reuse.

PRACTICE + PASS CRITERION

Model three variants, such as scalar, pipelined and multi-lane. Pass: resource and bandwidth estimates are explicit, fixed-point error meets a chosen application tolerance and rejected variants have documented reasons.

ASSIGNED STUDY

R49: architecture and RTL modeling; R43: bandwidth limits; R33: resource inference.

CHECK YOUR UNDERSTANDING

1. Why might twice as many lanes fail to double throughput?

2. What should determine acceptable quantization error?

3. Why explore more than one architecture?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 40.3 • MODULE 40

Interface design, virtualization and end-to-end evaluation

Learning goal Include the host software in the hardware result.

The host interface specifies work descriptors, buffers, alignment, ownership, completion and errors. Long-running or shared accelerators also need fairness, isolation and recovery policy. A malformed descriptor must not grant arbitrary access to unrelated memory. Use a simulated device or established protected driver framework for educational experiments.

End-to-end evaluation includes packaging data, transfer, queueing, computation, synchronization and result consumption. Pipeline overlap can improve throughput without equally reducing one request's latency. Measure at realistic batch sizes and include a fallback path. Hardware that works only when every input is perfect is not an integrated engineering result.

WORKED EXAMPLE

A kernel with a 5x arithmetic speedup may deliver only 1.2x application speedup after transfers and setup. Both numbers can be correct if their boundaries are stated.

PRACTICE + PASS CRITERION

Implement a host driver model with invalid-descriptor, timeout and reset tests. Pass: buffer bounds are enforced, ownership survives failure and results include both kernel and end-to-end timing.

ASSIGNED STUDY

R51: DMA and device interfaces; R49: system integration; R38: modeled systems when hardware is unavailable.

CHECK YOUR UNDERSTANDING

1. Why validate device work descriptors?

2. Can kernel and application speedups legitimately differ?

3. Why test realistic batch sizes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 40.4 • MODULE 40

Design review, evidence tiers and undergraduate CE completion

Learning goal Defend a co-design decision with limitations and reproducible artifacts.

A design review connects requirements to evidence: functional results, numerical quality, timing, resources, integration and risk. Label measurements according to their source. An analytical estimate, cycle-accurate model, synthesized netlist and physical instrument reading support different claims. Device-specific timing or area should not be generalized without a justified model.

The final comparison needs identical workloads, a competent baseline and disclosure of failed cases. Discuss cost beyond arithmetic, including tool dependence, verification effort, portability and maintenance. Preserve source, scripts, raw data and exact configuration. An independent reviewer should be able to repeat the central experiment and challenge at least one assumption. This checkpoint prepares the learner for research-level specialization, not independent production-chip signoff.

WORKED EXAMPLE

A synthesis estimate from an educational library is useful for comparing variants under that setup. It is not a vendor quotation or a guarantee of commercial silicon performance.

PRACTICE + PASS CRITERION

Complete Gate H and a design-review presentation. Pass: every major claim traces to an artifact, the baseline is reproduced and a limitations section clearly separates educational estimates from production claims.

ASSIGNED STUDY

R49: design-project reporting; R48: architectural tradeoffs; revisit the evidence and project rubrics in the handbook.

CHECK YOUR UNDERSTANDING

1. Why label simulation and physical measurements separately?

2. What makes a baseline competent?

3. Does Gate H authorize production chip fabrication?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 09 OF 10

Graduate Game-Engine and Graphics Engineering

Master's-level specialization

Derive algorithms, reproduce research, optimize parallel systems and defend measured engineering choices.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 41 Graduate numerical methods and research practice

41.1 Reading research as claims, assumptions and evidence • 41.2 Conditioning, sparse systems and iterative solvers • 41.3 Constrained optimization, derivatives and optimality conditions • 41.4 Experimental design, uncertainty and reproducible graduate work

Module 42 Advanced parallel and GPU algorithms

42.1 Parallel reductions, scans and work-span analysis • 42.2 Tiling, data reuse and asynchronous pipelines • 42.3 Irregular workloads, wavefront scheduling and load balance • 42.4 Portable performance and integration-quality evaluation

Module 43 Advanced rendering and reconstruction

43.1 Monte Carlo light transport and multiple importance sampling • 43.2 Real-time ray tracing, dynamic acceleration and hybrid pipelines • 43.3 Spatiotemporal reuse, reservoirs and denoising • 43.4 Differentiable and neural rendering under engineering constraints

Module 44 Advanced engine systems and simulation research

44.1 Large-world scheduling, streaming and critical-path control • 44.2 Deformable simulation, variational models and solver tradeoffs • 44.3 Distributed game state, consistency and overload behavior • 44.4 Formal specifications, replay and reliability-oriented engine design

Module 45 Compilers, shader languages and engine specialization

45.1 Intermediate representations, SSA and dataflow analysis • 45.2 Alias analysis, vectorization and legal optimization • 45.3 Domain-specific languages, specialization and safe JIT execution • 45.4 Compiler/hardware co-design and graduate artifact review

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 41

Graduate numerical methods and research practice

Prerequisites Gates B, F and H; rigorous linear algebra, calculus, probability and systems competence.

Four lessons

41.1 Reading research as claims, assumptions and evidence

Extract an implementable argument rather than only a paper summary.

41.2 Conditioning, sparse systems and iterative solvers

Choose a solver from mathematical properties and error requirements.

41.3 Constrained optimization, derivatives and optimality conditions

Derive and verify an optimization method for an engineering objective.

41.4 Experimental design, uncertainty and reproducible graduate work

Make a performance or quality claim that survives independent scrutiny.

Module project

Produce a graduate numerical notebook with derivations, reference implementations, conditioning experiments and a preregistered evaluation plan. Complete substantial assigned problem sets; the short lesson quizzes are only formative checks.

Core references

R42 Stephen Boyd and Lieven Vandenberghe — Convex Optimization

R54 MIT OpenCourseWare — 18.085 Computational Science and Engineering I

R09 MIT OpenCourseWare — 18.05 Introduction to Probability and Statistics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 41.1 • MODULE 41

Reading research as claims, assumptions and evidence

Learning goal Extract an implementable argument rather than only a paper summary.

A research paper makes claims under assumptions using a method and evidence. First identify the problem, baseline, objective and scope. Then reconstruct the central derivation or algorithm without looking at the authors' code. Track definitions, units and where approximations enter. A result can be valuable while narrower than its abstract initially suggests.

Build a claim-evidence table: each claim points to an equation, proof, experiment or stated limitation. Distinguish reproduction of reported results from an independent implementation and from a new extension. Read related work to identify the actual contribution. Select original papers and author artifacts where available, and record versions. A negative reproduction result needs diagnostic investigation before concluding the original claim is false.

WORKED EXAMPLE

A method reported faster at equal image error is not necessarily faster at equal sample count. Reproducing the wrong comparison target tests a different claim.

PRACTICE + PASS CRITERION

Analyze one paper associated with R23, R44 or R55. Pass: a two-page claim map, a reconstructed central algorithm, at least three assumptions and a feasible reproduction target are reviewed by another reader.

ASSIGNED STUDY

R23: Further Reading sections for a chosen rendering topic; R44 or R55: one named original paper; handbook research-reading template.

CHECK YOUR UNDERSTANDING

1. Why distinguish equal-error from equal-sample comparisons?

2. What makes a claim-evidence table useful?

3. Does a failed reproduction immediately disprove a paper?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 41.2 • MODULE 41

Conditioning, sparse systems and iterative solvers

Learning goal Choose a solver from mathematical properties and error requirements.

A linear system Ax=b has numerical behavior governed partly by conditioning: sensitivity of the solution to changes in data. Residual r=b-Ax measures equation mismatch, but a small residual need not imply a small forward error for an ill-conditioned system. Scaling and problem formulation matter before choosing a faster kernel.

Sparse storage exploits structure, while iterative methods use repeated matrix-vector operations. Conjugate gradient requires a symmetric positive-definite setting for its standard guarantees. Preconditioning changes the system seen by the iteration to improve convergence without changing the intended solution when correctly applied. State stopping criteria, norm, precision and maximum work. Test a known solution and increasingly difficult conditioning, not only one friendly matrix.

WORKED EXAMPLE

If A=diag(1,10^-6), its 2-norm condition number is 10^6. A small perturbation in the second equation can produce a much larger relative change in its associated solution component.

PRACTICE + PASS CRITERION

Implement a sparse matrix-vector product and conjugate gradient for a verified SPD system. Pass: derive the relevant assumptions, compare residual and forward error, and evaluate a simple preconditioner over several conditioning levels.

ASSIGNED STUDY

R54: linear systems, positive definiteness and numerical methods; R07: orthogonality and eigenvalues; R42: numerical optimization background.

CHECK YOUR UNDERSTANDING

1. What is the 2-norm condition number of diag(1,10^-6)?

2. Why can a small residual coexist with substantial solution error?

3. What matrix property is required by standard conjugate gradient?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 41.3 • MODULE 41

Constrained optimization, derivatives and optimality conditions

Learning goal Derive and verify an optimization method for an engineering objective.

Optimization requires a defined objective, feasible set and numerical tolerance. For differentiable unconstrained problems, stationary points have zero gradient but may be minima, maxima or saddles. Convexity provides stronger global conclusions under the appropriate assumptions. Equality and inequality constraints introduce Lagrange multipliers and Karush-Kuhn-Tucker conditions, whose necessity or sufficiency depends on regularity and convexity.

For nonlinear least squares, residual Jacobians support Gauss-Newton-style approximations; poor scaling or rank deficiency can require damping or regularization. Check analytic or automatic derivatives against finite differences over sensible step sizes. Gradient checks themselves have truncation and roundoff error. Report convergence histories and failed initializations rather than only a final successful parameter set.

WORKED EXAMPLE

Minimize x^2+y^2 subject to x+y=1. The Lagrangian gives 2x+lambda=0 and 2y+lambda=0; together with the constraint, x=y=0.5 and lambda=-1.

PRACTICE + PASS CRITERION

Derive the example, then solve a constrained fitting or IK problem with a numerical method. Pass: derivative checks, feasibility residual, convergence criteria and at least one failure or sensitivity case are reported.

ASSIGNED STUDY

R42: convexity, duality, optimality and numerical methods; R41: gradients and constrained differentials.

CHECK YOUR UNDERSTANDING

1. What is the constrained minimizer in the example?

2. Does zero gradient prove a general nonconvex point is a minimum?

3. Why vary finite-difference step size in a derivative check?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 41.4 • MODULE 41

Experimental design, uncertainty and reproducible graduate work

Learning goal Make a performance or quality claim that survives independent scrutiny.

Define the primary outcome and comparison before examining final results. Choose workloads that represent the intended claim, plus adverse and held-out cases. Repeated frames within one run can be correlated, so the statistical unit might be a run, scene or device rather than every frame. A large count of correlated observations does not automatically provide strong independent evidence.

Use confidence intervals or other justified uncertainty summaries, report effect sizes and preserve raw observations. Ablations remove or replace components to test which parts cause an improvement. Multiple comparisons and parameter tuning can overfit the benchmark suite. Keep final evaluation data separate from tuning when feasible. State external validity limits, including devices, workloads and quality settings not tested.

WORKED EXAMPLE

Ten thousand frames from one scene are not equivalent to ten thousand independent games. A scene-level comparison across diverse scenes can better test a claim about broad engine behavior.

PRACTICE + PASS CRITERION

Preregister a small experiment with baseline, primary metric, trial unit and exclusion policy. Pass: raw data and analysis scripts reproduce the tables, and an ablation isolates at least one proposed mechanism.

ASSIGNED STUDY

R09: statistical inference; R42: experimental examples; handbook benchmark and research templates.

CHECK YOUR UNDERSTANDING

1. Why identify the statistical unit?

2. What does an ablation test?

3. Why reserve held-out workloads?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 42

Advanced parallel and GPU algorithms

Prerequisites Modules 18, 37-38 and 41.

Four lessons

42.1 Parallel reductions, scans and work-span analysis

Prove the structure of a parallel primitive before tuning it.

42.2 Tiling, data reuse and asynchronous pipelines

Overlap movement and computation without creating races.

42.3 Irregular workloads, wavefront scheduling and load balance

Design around variable work rather than assuming identical threads.

42.4 Portable performance and integration-quality evaluation

Separate algorithmic benefit from one device-specific tuning win.

Module project

Implement a reusable GPU reduction, scan or compaction primitive and integrate it into an engine workload. Compare a simple reference, a competent library baseline when available and your implementation at equal correctness requirements.

Core references

R37 NVIDIA — CUDA Programming Guide

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 42.1 • MODULE 42

Parallel reductions, scans and work-span analysis

Learning goal Prove the structure of a parallel primitive before tuning it.

A reduction combines many values with an operator; a scan produces prefix combinations. Associativity supports regrouping mathematically, but floating-point addition is not exactly associative. Decide whether the contract requires bitwise reproducibility, bounded error or an exact integer result. A tree reduction can reduce dependency depth from linear to logarithmic while retaining linear total work.

Work is the total number of operations; span is the longest dependency chain. More processors cannot beat the span without changing the algorithm. A naive parallel scan can have extra logarithmic work, while work-efficient constructions achieve linear work with logarithmic span under their model. Handle partial groups, empty inputs and irregular sizes explicitly. Cross-group coordination must follow the API rather than assuming a global barrier inside an ordinary dispatch.

WORKED EXAMPLE

A balanced reduction over 1,024 elements has 10 combination levels under a binary-tree model and 1,023 total pairwise combinations. Its implementation still pays memory and synchronization costs.

PRACTICE + PASS CRITERION

Implement reduction and either scan or compaction. Pass: derive work/span bounds, test sizes 0, 1 and non-powers of two, and report the numerical/reproducibility contract.

ASSIGNED STUDY

R37: cooperative execution and synchronization; R48: parallel architectures; derive algorithm bounds using Module 14 methods.

CHECK YOUR UNDERSTANDING

1. How many levels are in a balanced binary reduction of 1,024 values?

2. How many pairwise combines does that reduction require?

3. Why can parallel floating-point sums differ from serial sums?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 42.2 • MODULE 42

Tiling, data reuse and asynchronous pipelines

Learning goal Overlap movement and computation without creating races.

Tiling partitions a computation so values can be reused in a faster memory level. Matrix multiplication, image filters and neighborhood operations benefit when the tile fits available storage and exposes sufficient parallelism. Larger tiles can increase reuse but also raise register or shared-memory demand and reduce residency.

Double buffering can overlap preparation of one tile with computation on another. The pipeline needs explicit ownership: no consumer reads an incomplete tile and no producer overwrites data still in use. Asynchronous copy features vary by device; maintain a correct portable baseline. Account for boundary tiles, alignment, synchronization and extra staging work. Compare arithmetic intensity at the memory level actually being optimized.

WORKED EXAMPLE

For C=A*B, reusing a loaded A or B tile across many multiply-adds can raise useful operations per byte. A tile that spills heavily can erase that advantage.

PRACTICE + PASS CRITERION

Implement a tiled matrix multiply or image filter, then one overlap variant where supported. Pass: adversarial dimensions match the reference, synchronization is checked and the end-to-end comparison includes staging overhead.

ASSIGNED STUDY

R37: shared memory, synchronization and asynchronous operations where supported; R43: operational intensity.

CHECK YOUR UNDERSTANDING

1. Why can larger tiles hurt performance?

2. What ownership rule is essential for double buffering?

3. Why retain a non-asynchronous baseline?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 42.3 • MODULE 42

Irregular workloads, wavefront scheduling and load balance

Learning goal Design around variable work rather than assuming identical threads.

Ray traversal, graph processing and variable-length simulation tasks have irregular work. A fixed assignment can leave some workers idle while a few long tasks dominate the tail. Work queues, compaction and wavefront processing reorganize active tasks, trading scheduling and memory traffic for better coherence or load balance.

Persistent-worker designs keep workers alive to fetch tasks, but need bounded queues, termination detection and a memory-ordering contract. Global progress cannot depend on every workgroup being resident when the hardware does not guarantee that. Sorting tasks by similarity can improve coherence but has a cost and can alter latency. Evaluate both balanced and skewed distributions, including rare expensive tasks.

WORKED EXAMPLE

If 99 tasks take 1 unit and one takes 100 units, static one-task assignments can leave the final completion dominated by the long task. Splitting or scheduling work differently may help only when the task is divisible.

PRACTICE + PASS CRITERION

Implement an irregular workload using static assignment and a queue or compaction alternative. Pass: prove or test termination, handle queue capacity safely and measure tail behavior as well as mean throughput.

ASSIGNED STUDY

R37: execution, atomics and synchronization; R48: multithreaded scheduling; R23: wavefront rendering as an application.

CHECK YOUR UNDERSTANDING

1. Why can average task cost hide poor completion time?

2. Why avoid a global spin barrier requiring every group to be resident?

3. Is sorting tasks always beneficial?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 42.4 • MODULE 42

Portable performance and integration-quality evaluation

Learning goal Separate algorithmic benefit from one device-specific tuning win.

A performance-portable implementation preserves correctness and obtains useful efficiency across its stated target set. This does not mean identical code or identical speed everywhere. Capability dispatch can select specialized paths while keeping a tested fallback. Tuning parameters should be bounded and recorded rather than silently selected from the final test set.

Compare against a competent baseline and include small, large and irregular workloads. Report speed, memory, numerical error and integration cost. A kernel that is faster only after excluding transfer or conversion work should make that boundary explicit. Reproducible artifacts include build scripts, device capability reports, raw data and failure cases. Lack of access to multiple GPUs is a limitation to disclose, not a reason to invent portability evidence.

WORKED EXAMPLE

A tile size tuned on one GPU can reduce occupancy or exceed shared-memory limits on another. Capability checks and independent tuning prevent that choice from becoming an invalid universal default.

PRACTICE + PASS CRITERION

Integrate the selected primitive into the engine and evaluate at least two workload regimes. Pass: application impact is measured, unsupported features fall back safely and any untested device claims are explicitly excluded.

ASSIGNED STUDY

R37 and R21: capability contracts; R43: performance models; Module 41 experimental protocol.

CHECK YOUR UNDERSTANDING

1. Does portable performance require one fixed launch configuration?

2. What should be said when only one GPU was tested?

3. Why measure integration cost?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 43

Advanced rendering and reconstruction

Prerequisites Modules 09-10, 21-23 and 41-42.

Four lessons

43.1 Monte Carlo light transport and multiple importance sampling

Derive estimators with correct densities and measures.

43.2 Real-time ray tracing, dynamic acceleration and hybrid pipelines

Budget traversal, building and shading as one system.

43.3 Spatiotemporal reuse, reservoirs and denoising

Evaluate variance reduction together with bias, history validity and motion.

43.4 Differentiable and neural rendering under engineering constraints

Test learned or gradient-based methods against explicit baselines.

Module project

Reproduce a bounded rendering result and add one justified extension. Derive the estimator or reconstruction objective, compare equal-quality cost, include temporal failure cases and preserve source scenes and evaluation scripts.

Core references

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

R44 Benedikt Bitterli and coauthors — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

R47 Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola — Dive into Deep Learning

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 43.1 • MODULE 43

Monte Carlo light transport and multiple importance sampling

Learning goal Derive estimators with correct densities and measures.

The rendering equation expresses outgoing radiance as emission plus an integral of incident radiance, scattering and geometry factors. Monte Carlo estimation samples an integral: for samples from density p, average f(x)/p(x) where p is positive over the relevant nonzero support. Correct units and probability measures matter; area and solid-angle densities are not interchangeable without a Jacobian.

Importance sampling aims to reduce variance by matching useful structure. Multiple importance sampling combines strategies with weights that sum appropriately under the estimator. The balance heuristic uses relative weighted densities, but sample counts and support must be included correctly. Russian roulette can terminate paths without bias when surviving contributions receive the proper probability compensation. Finite-variance independent sampling gives the familiar inverse-square-root standard-error scaling; do not assume it for every estimator.

WORKED EXAMPLE

With one sample from each of two strategies and densities p1=2 and p2=6 at a point, the balance weight for strategy 1 is 2/(2+6)=0.25.

PRACTICE + PASS CRITERION

Implement a path tracer with explicit light sampling and MIS. Pass: derive the estimator, verify a simple known integral, compare variance at equal cost and test emission and grazing-angle edge cases.

ASSIGNED STUDY

R23: Monte Carlo Integration, Light Transport and sampling strategies. Work the associated mathematical exercises, not only the implementation.

CHECK YOUR UNDERSTANDING

1. What is the balance weight for strategy 1 in the example?

2. Why is p(x)>0 on relevant support important?

3. What sample-count increase ideally halves standard error for independent finite-variance samples?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 43.2 • MODULE 43

Real-time ray tracing, dynamic acceleration and hybrid pipelines

Learning goal Budget traversal, building and shading as one system.

Ray-tracing acceleration structures trade build cost, memory and query cost. Dynamic scenes may update or rebuild structures; the best policy depends on movement, geometry and traversal degradation. A refit can be cheaper initially but leave poor bounds after large changes. Hardware-assisted traversal still needs correct synchronization and resource lifetime.

Hybrid renderers combine rasterized visibility with ray-traced shadows, reflections or indirect lighting. Each path needs a consistent material and coordinate model. Screen-space or cached approximations require fallbacks where information is absent. Measure acceleration build/update, tracing, shading, denoising and synchronization separately, then report total frame impact. Compare equivalent scene motion and image quality rather than static screenshots alone.

WORKED EXAMPLE

A faster traversal variant that adds 4 ms of build work to save 1 ms of tracing is a regression for a scene rebuilt every frame, but might help a scene reused for many frames.

PRACTICE + PASS CRITERION

Add one ray-traced effect to the engine or build a software hybrid prototype. Pass: dynamic-scene tests include total update cost, missing-information fallbacks and resource synchronization evidence.

ASSIGNED STUDY

R23: intersection acceleration and light transport; R21, R24 or R25: supported ray-tracing interfaces for an optional hardware route.

CHECK YOUR UNDERSTANDING

1. Why can a cheap BVH refit become a poor long-term choice?

2. Is saving 1 ms of tracing worth 4 ms of per-frame rebuild cost?

3. Why compare animated scenes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 43.3 • MODULE 43

Spatiotemporal reuse, reservoirs and denoising

Learning goal Evaluate variance reduction together with bias, history validity and motion.

Temporal and spatial reuse exploit related samples across frames or nearby locations. Reusing data changes the sampling and correlation structure, so the estimator and weights must be derived rather than improvised. Reservoir sampling can retain a compact representative sample under a defined weighted selection rule; its rendering use requires additional treatment of target distributions and reuse.

Denoising reconstructs a signal from noisy observations using assumptions about spatial, temporal or learned structure. It can suppress noise while introducing bias, blur or ghosting. Visibility changes and incorrect motion vectors invalidate history. Distinguish estimator bias from reconstruction bias and assess moving sequences at equal total cost. The original 2020 ReSTIR paper is a historical reproduction option, not a claim to represent the latest technique.

WORKED EXAMPLE

If a moving object reveals a background, old foreground samples are not valid background evidence merely because they occupy the same pixel coordinate. History acceptance must account for correspondence and visibility.

PRACTICE + PASS CRITERION

Reproduce a small weighted-reservoir example or temporal denoiser, then evaluate on motion and disocclusion sequences. Pass: the selection/weight rule is derived, bias limitations are stated and results include sequence-level failure cases.

ASSIGNED STUDY

R44: original ReSTIR paper and its estimator assumptions; R23: sampling theory; read only the selected bounded method before expanding scope.

CHECK YOUR UNDERSTANDING

1. Why cannot reused samples always be treated as new independent samples?

2. Why is same-pixel history not enough to establish correspondence?

3. What should be compared in a denoised rendering benchmark?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 43.4 • MODULE 43

Differentiable and neural rendering under engineering constraints

Learning goal Test learned or gradient-based methods against explicit baselines.

Differentiable rendering studies how outputs change with scene or model parameters. Visibility discontinuities, stochastic sampling and approximations complicate gradients. A useful derivative must match the actual objective and parameterization; automatic differentiation alone does not guarantee the right physical or statistical estimator. Validate on small scenes with finite differences and known cases.

Neural rendering or learned reconstruction can trade preprocessing, training and memory for runtime quality or speed. Training data coverage, scene specificity, temporal stability and inference hardware constrain deployment. Compare a non-learned baseline and include training or preprocessing cost where the claim requires it. Separate interpolation within familiar conditions from generalization to new scenes, lighting or motion.

WORKED EXAMPLE

A reconstruction network tested on frames from its own training sequence may perform well without generalizing to a new level. Holding out entire scenes tests a stronger claim.

PRACTICE + PASS CRITERION

Implement a small inverse-rendering optimization or learned reconstruction experiment. Pass: gradient or generalization checks, a non-learned baseline, compute/memory accounting and a distribution-shift failure analysis are included.

ASSIGNED STUDY

R47: automatic differentiation, optimization and generalization; R23: light transport and sampling foundations; select one original method with Module 41's paper-review protocol.

CHECK YOUR UNDERSTANDING

1. Does automatic differentiation guarantee the desired gradient estimator?

2. Why hold out entire scenes for some rendering evaluations?

3. When should training cost be included?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 44

Advanced engine systems and simulation research

Prerequisites Gates F-H; Modules 41-43.

Four lessons

44.1 Large-world scheduling, streaming and critical-path control

Design a bounded system under changing content and workload.

44.2 Deformable simulation, variational models and solver tradeoffs

Derive a physical or constraint model before comparing visual results.

44.3 Distributed game state, consistency and overload behavior

Specify which guarantees survive network delay and partial failure.

44.4 Formal specifications, replay and reliability-oriented engine design

Use precise invariants to reduce the space of subtle failures.

Module project

Conduct an engine-system experiment with a formal model or numerical argument, a reference implementation and stress-tested integration. Choose one main depth area: world streaming, simulation, distributed state or reliability.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R54 MIT OpenCourseWare — 18.085 Computational Science and Engineering I

R55 Matthias Mueller and collaborators — Author publication archive: position-based simulation and XPBD

R50 YosysHQ — SymbiYosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 44.1 • MODULE 44

Large-world scheduling, streaming and critical-path control

Learning goal Design a bounded system under changing content and workload.

Large-world engines coordinate visibility, simulation, streaming and resource residency under changing demand. A frame graph can make dependencies explicit, but hidden shared state or unpredictable tasks can still expand the critical path. Partition work by ownership and locality while controlling cross-partition communication.

World-coordinate precision can degrade far from the origin; origin rebasing, hierarchical coordinates or higher precision each affect physics, rendering and networking interfaces. Streaming priorities should reflect gameplay deadlines and resource availability, not only distance. Build a workload model with bursts, cancellation and scarce memory. Evaluate tail latency and quality degradation under stress rather than only steady travel through a preloaded scene.

WORKED EXAMPLE

A camera teleport invalidates a distance-based prefetch assumption and can create a sudden working-set change. The engine needs placeholders and a bounded recovery policy rather than an unlimited synchronous load.

PRACTICE + PASS CRITERION

Implement a bounded world-streaming scheduler or large-coordinate experiment. Pass: teleport and memory-pressure tests preserve correctness, peak resources are measured and the critical path is explained with a trace.

ASSIGNED STUDY

R26: world, task and streaming systems for comparison; R20: engine architecture; R54: numerical conditioning where coordinate precision is studied.

CHECK YOUR UNDERSTANDING

1. Why can a camera teleport defeat ordinary prefetching?

2. Why is origin rebasing an interface issue?

3. What distinguishes a bounded degradation policy?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 44.2 • MODULE 44

Deformable simulation, variational models and solver tradeoffs

Learning goal Derive a physical or constraint model before comparing visual results.

Deformable simulation represents degrees of freedom and a model of forces, energies or constraints. Spatial discretization, boundary conditions and time integration together define the numerical problem. Finite-element or mass-spring approaches approximate different structures; a visually similar result does not prove equivalent material behavior.

Implicit methods can improve stability for stiff systems while requiring nonlinear or linear solves and introducing other errors. Position-based and compliant-constraint methods offer alternative formulations with their own parameters and convergence behavior. Derive one simple energy or constraint, explain its units and study timestep/iteration sensitivity. Separate stability, accuracy and physical fidelity. A stable method can be inaccurate, and an accurate local solver can still use an inappropriate material model.

WORKED EXAMPLE

For spring energy E=0.5*k*(x-rest)^2, the force is -dE/dx=-k*(x-rest). This derivative supplies a check before extending to a multi-particle system.

PRACTICE + PASS CRITERION

Implement a small cloth, elastic bar or compliant-constraint model and compare two integration/solver choices. Pass: derive the core equations, check gradients, quantify constraint or energy error and reproduce one bounded result from the assigned research.

ASSIGNED STUDY

R54: equilibrium, variational and differential-equation methods; R55: 2016 XPBD and related position-based simulation course notes.

CHECK YOUR UNDERSTANDING

1. What force follows from the example spring energy?

2. Does numerical stability guarantee physical accuracy?

3. Why vary timestep and solver iterations independently?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 44.3 • MODULE 44

Distributed game state, consistency and overload behavior

Learning goal Specify which guarantees survive network delay and partial failure.

Distributed engine services divide state across processes or machines. Some operations require a single authoritative order, while others tolerate delayed or approximate views. Consistency is a property of a specified operation and model, not a single adjective that applies equally to every subsystem. During communication failure, availability and agreement requirements can conflict.

Partition ownership, migration and recovery need explicit protocols. A world handoff must not create two authorities granting the same irreversible outcome. Queues require bounds and backpressure; retry storms can magnify overload. Test fault sequences involving delayed messages, crashes and duplicate requests. Use local process simulation before public infrastructure, and never claim internet-scale behavior from a two-process happy-path demo.

WORKED EXAMPLE

If two servers both accept ownership of the same entity during a failed handoff, both might award a unique item. A versioned ownership or fencing mechanism must prevent stale authority from committing that effect.

PRACTICE + PASS CRITERION

Model a two-partition entity handoff with injected failure. Pass: a stated safety invariant holds, recovery assumptions are explicit and overload is bounded without unbounded retry amplification.

ASSIGNED STUDY

R17-R18: transport and transactions; R20: game-state architecture; use Module 41's claim and experiment discipline for the distributed model.

CHECK YOUR UNDERSTANDING

1. Why is duplicate authority dangerous?

2. What is a retry storm?

3. Can a small local demo establish internet-scale reliability?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 44.4 • MODULE 44

Formal specifications, replay and reliability-oriented engine design

Learning goal Use precise invariants to reduce the space of subtle failures.

A formal model abstracts a system to states and transitions so properties can be examined precisely. Safety states that something bad never happens; liveness states that something good eventually happens under assumptions. An engine resource manager can have safety invariants about ownership and liveness obligations about eventual completion or cancellation.

Model checking explores the chosen abstraction, not every detail of the production implementation. Connect model transitions to real code and tests. Deterministic replay, fault injection and property-based testing complement formal reasoning by exercising implemented behavior. Avoid vacuous proofs from impossible environment assumptions. A reliability claim should identify the property, model, bound and implementation evidence separately.

WORKED EXAMPLE

For an asset request, safety might require that a released owner never receives a callback touching its memory. Liveness might require that each accepted request eventually completes or cancels, assuming the worker continues to run.

PRACTICE + PASS CRITERION

Model a small engine lifecycle or queue protocol and connect it to implementation tests. Pass: at least one safety and one conditional liveness property are defined, a deliberate violation is found and abstraction limits are documented.

ASSIGNED STUDY

R50: safety, liveness and assumptions as a formal-methods reference; R11 and Module 18: ownership/concurrency contracts; R26: system comparison.

CHECK YOUR UNDERSTANDING

1. How do safety and liveness differ?

2. Why must liveness assumptions mention scheduling or worker progress?

3. Does proving an abstract model automatically prove all implementation code?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 45

Compilers, shader languages and engine specialization

Prerequisites Modules 15-16, 18, 37 and 41-42.

Four lessons

45.1 Intermediate representations, SSA and dataflow analysis

Represent program meaning so transformations can be justified.

45.2 Alias analysis, vectorization and legal optimization

Separate transformations that are fast from transformations that preserve meaning.

45.3 Domain-specific languages, specialization and safe JIT execution

Trade generality for predictable game-engine computation.

45.4 Compiler/hardware co-design and graduate artifact review

Connect a transformation to architecture and application evidence.

Module project

Gate I: deliver a research-quality reproduction and extension in rendering, simulation, GPU algorithms or compilation. This module also requires a small compiler optimization with semantic tests, generated-code inspection and end-to-end measurements.

Core references

R14 LLVM Project — Kaleidoscope and LLVM tutorials

R11 C++ Core Guidelines editors — C++ Core Guidelines

R37 NVIDIA — CUDA Programming Guide

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 45.1 • MODULE 45

Intermediate representations, SSA and dataflow analysis

Learning goal Represent program meaning so transformations can be justified.

An intermediate representation makes control flow, values and operations explicit. Static single assignment gives each value one defining assignment; merge points use a representation such as phi nodes to select values from predecessor paths. SSA simplifies many analyses but does not by itself establish program correctness.

Dataflow analysis computes properties over a control-flow graph, often through iterative fixed points on a chosen domain. Liveness identifies values that may be used later; constant propagation tracks values known under the analysis. Soundness means the analysis does not exclude behavior relevant to its guarantee, while precision controls how much useful information it retains. Dominance and control-flow edges must remain consistent after transformations.

WORKED EXAMPLE

After an if/else assigning x=1 or x=2, a merge can represent x=phi(1 from then, 2 from else). Replacing it with 1 unconditionally would be wrong unless the else path is proven unreachable.

PRACTICE + PASS CRITERION

Extend the tiny compiler with a control-flow IR and one dataflow analysis. Pass: derive transfer functions and a fixed-point stopping condition, test loops and merges, and verify transformed programs against an interpreter.

ASSIGNED STUDY

R14: LLVM frontend, control flow and optimization tutorials; R48: compiler/architecture interaction.

CHECK YOUR UNDERSTANDING

1. What does SSA require of each value?

2. Why is the example phi node not always constant 1?

3. What is the role of a fixed point in dataflow analysis?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 45.2 • MODULE 45

Alias analysis, vectorization and legal optimization

Learning goal Separate transformations that are fast from transformations that preserve meaning.

An optimization must preserve the language's permitted observable behavior under its assumptions. Alias analysis determines which references may identify overlapping storage. Incorrectly assuming no aliasing can make loop reordering or vectorization wrong. Undefined behavior, overflow rules and floating-point options affect what transformations are legal, so record the compilation contract.

Vectorization groups iterations only when dependencies and semantics permit it. Reductions, exceptional values and rounding can require special handling. Register allocation maps values to a limited physical register set; spills introduce memory traffic. Inspect generated code and test adversarial inputs. A faster compiler output that silently changes required results is a correctness failure, not an optimization.

WORKED EXAMPLE

For a[i]=a[i-1]+1, iterations depend on previous results. Treating all iterations as independent changes behavior unless a valid alternative algorithm preserves the recurrence.

PRACTICE + PASS CRITERION

Implement or study a loop transformation with an explicit legality check. Pass: overlapping-memory and boundary cases are tested, compiler flags are recorded and speed claims exclude transformations that violate the required numerical contract.

ASSIGNED STUDY

R14: optimization and code generation; R11: aliasing, lifetime and arithmetic guidance; R37: vector/parallel execution consequences.

CHECK YOUR UNDERSTANDING

1. Why is alias analysis important for reordering loads and stores?

2. Are iterations of a[i]=a[i-1]+1 independent?

3. Why record floating-point compiler options?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 45.3 • MODULE 45

Domain-specific languages, specialization and safe JIT execution

Learning goal Trade generality for predictable game-engine computation.

A domain-specific language restricts expression to make authoring, validation or optimization easier. A material graph, particle formula or animation expression can expose structure that a general compiler cannot easily infer. Define types, allowed effects and resource limits before implementing code generation.

Specialization precomputes decisions known from material or scene parameters, potentially reducing runtime branches at the cost of more variants and compilation work. Just-in-time compilation introduces startup latency, code-cache management and execution-security concerns. Cache keys must include meaningful inputs and tool versions. Untrusted scripts or graphs need capability and complexity limits. A restricted interpreter remains a useful fallback and semantic reference.

WORKED EXAMPLE

Specializing four binary material options can create up to 2^4=16 variants before other parameters are considered. Uncontrolled variant growth can increase build time and runtime cache misses.

PRACTICE + PASS CRITERION

Build a typed expression or material DSL with constant folding and a bounded interpreter or JIT route. Pass: invalid graphs fail safely, variant counts are measured and compiled outputs match the reference.

ASSIGNED STUDY

R14: language implementation and JIT; R26: material and scripting concepts for comparison; Module 20: trust boundaries.

CHECK YOUR UNDERSTANDING

1. How many combinations do four independent binary options create?

2. Why limit the effects allowed in a DSL?

3. Why keep an interpreter alongside compiled execution?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 45.4 • MODULE 45

Compiler/hardware co-design and graduate artifact review

Learning goal Connect a transformation to architecture and application evidence.

Compiler decisions influence memory layout, instruction mix, register demand and parallel scheduling. Hardware features are useful only when software can express and exploit them efficiently. A co-design experiment changes one interface, representation or instruction opportunity and measures its complete effect. Avoid attributing a gain to hardware when it actually comes from a stronger software baseline.

Graduate review requires a defensible claim, mathematical or semantic argument, implementation and reproducible evaluation. Gate I can focus on one earlier graduate topic, but must include baselines, ablations, uncertainty and limitations. Publication acceptance is not required; credible independent evaluation is. Preserve unsuccessful experiments and distinguish a reproduced known result from the learner's actual extension.

WORKED EXAMPLE

A new vector instruction that saves arithmetic but increases packing and spills may lose in the full engine. The experiment must measure both the generated kernel and its application integration.

PRACTICE + PASS CRITERION

Complete Gate I and defend one compiler transformation's legality. Pass: an independent reader reruns the central experiment, the claimed extension is precisely identified and raw evidence supports the conclusions.

ASSIGNED STUDY

R14 and R48: compilation and architecture; R49: co-design methodology; Module 41: research evaluation protocol.

CHECK YOUR UNDERSTANDING

1. Why can a faster instruction fail to improve the application?

2. What distinguishes reproduction from extension?

3. Is publication acceptance mandatory for Gate I?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

VOLUME 10 OF 10

Graduate Gaming Hardware and Research Thesis

Master's-level specialization / thesis

Evaluate advanced architectures, physical design and verification, then complete and defend a research artifact.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 46 Advanced computer architecture for gaming workloads

46.1 Cache coherence, memory consistency and synchronization • 46.2 Out-of-order execution, speculation and precise state • 46.3 Architecture simulation, traces and model validation • 46.4 Memory systems, chiplets and multi-objective design

Module 47 VLSI, timing and physical implementation

47.1 CMOS logic, transistor sizing and delay models • 47.2 Static timing analysis, clocks and timing exceptions • 47.3 Synthesis, placement, clock trees, routing and physical checks • 47.4 Power, area, process corners and design-space closure

Module 48 Verification, reliability and hardware trust boundaries

48.1 Formal properties, abstraction and assume-guarantee reasoning • 48.2 Clock/reset crossings and asynchronous FIFO verification • 48.3 Fault models, ECC, testability and recovery • 48.4 Hardware security, thermal limits and assurance review

Module 49 Master's thesis: proposal, implementation and evaluation

49.1 Question selection, literature map and thesis scope • 49.2 Proposal review, baselines and risk-reduction milestones • 49.3 Implementation, ablations and controlled evaluation • 49.4 Thesis writing, reproducibility and contribution boundaries

Module 50 Thesis defense, professional practice and continued development

50.1 Oral defense and independent artifact reproduction • 50.2 Mature-codebase integration, review and maintainable contributions • 50.3 Portfolio, technical interviews and role-specific evidence • 50.4 Long-term learning, maintenance and final handoff

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 46

Advanced computer architecture for gaming workloads

Prerequisites Modules 34, 37-40 and 41-42.

Four lessons

46.1 Cache coherence, memory consistency and synchronization

Separate agreement about a location from ordering across locations.

46.2 Out-of-order execution, speculation and precise state

Increase instruction overlap without changing architectural outcomes.

46.3 Architecture simulation, traces and model validation

Use simulation as evidence with a stated fidelity boundary.

46.4 Memory systems, chiplets and multi-objective design

Choose an architecture across performance, power, cost and complexity.

Module project

Perform an architecture study using a pinned simulator or a carefully bounded analytical model. Compare at least three configurations on representative and adverse game-related workloads, and validate the model against a known reference where possible.

Core references

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R38 gem5 project — gem5 documentation

R35 RISC-V International — RISC-V ISA specifications

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 46.1 • MODULE 46

Cache coherence, memory consistency and synchronization

Learning goal Separate agreement about a location from ordering across locations.

Cache coherence governs how cached copies of a location interact, including ownership and visibility of writes. Memory consistency defines which orderings of operations across locations are observable. A coherent machine can still expose behaviors disallowed by a stronger consistency model. Programming-language memory rules add another layer between source code and hardware.

Protocols such as MESI use states and messages to preserve coherence invariants, but real implementations include transient states and races among requests. A useful invariant is that no two participants simultaneously hold conflicting write ownership under the protocol. Directory designs track sharers while snooping designs observe suitable interconnect traffic. False sharing and invalidation storms are performance effects of otherwise correct coherence. Use small litmus tests and state models to connect abstract rules to software synchronization.

WORKED EXAMPLE

A producer writes data and then publishes a flag. Correct consumer behavior depends on an appropriate release/acquire or equivalent synchronization contract; coherent caches alone do not justify ordinary unsynchronized C++ access.

PRACTICE + PASS CRITERION

Model a small coherence protocol and evaluate message-passing litmus tests under stated language and hardware models. Pass: ownership invariants hold, transient races are considered and coherence is not confused with data-race freedom.

ASSIGNED STUDY

R48: multiprocessors, coherence and consistency; R35: selected memory-ordering specification; R11: language-level concurrency.

CHECK YOUR UNDERSTANDING

1. How do coherence and consistency differ?

2. Do coherent caches make unsynchronized C++ shared access safe?

3. Why include transient states in a protocol model?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 46.2 • MODULE 46

Out-of-order execution, speculation and precise state

Learning goal Increase instruction overlap without changing architectural outcomes.

Out-of-order processors schedule ready operations while older operations may be waiting. Register renaming removes false name dependencies such as write-after-read and write-after-write, but true data dependencies remain. A reorder or retirement mechanism preserves the required architectural order and supports precise exceptions.

Branch prediction and memory speculation create work before all conditions are known. Incorrect speculation must discard inappropriate architectural effects, while microarchitectural traces may still matter to security and performance. More execution width helps only when useful parallelism, instruction supply and memory behavior support it. Analyze queue capacity, issue limits and dependency chains rather than assuming a larger core always improves a latency-sensitive game thread.

WORKED EXAMPLE

For two instructions that both write architectural register r1 but whose values are consumed separately, renaming can assign different physical destinations. A later instruction that truly consumes the first result still waits for that result.

PRACTICE + PASS CRITERION

Use a simulator or detailed scheduling model to compare in-order and out-of-order execution on independent and dependent instruction streams. Pass: architectural results agree and bottleneck explanations include true dependencies and resource limits.

ASSIGNED STUDY

R48: superscalar, out-of-order, prediction and exception handling; R38: CPU models and configuration.

CHECK YOUR UNDERSTANDING

1. Which dependencies does register renaming primarily remove?

2. Why retire architectural effects in a controlled order?

3. Why may a wider core fail to improve one game thread?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 46.3 • MODULE 46

Architecture simulation, traces and model validation

Learning goal Use simulation as evidence with a stated fidelity boundary.

A functional simulator checks what computation does; a timing model estimates when events occur under modeled resources. A cycle-oriented model is not automatically accurate for every real device. Workload inputs, cache warmup, sampled regions and operating-system effects influence results. Trace-driven studies can lose feedback between timing and program behavior.

Validate simple cases against analytical expectations and available measured hardware before trusting complex comparisons. State which pipeline, memory and interconnect mechanisms are modeled or omitted. Sensitivity analysis varies uncertain parameters to see whether conclusions survive. Preserve simulator revision, configuration and workload binaries. Relative comparisons can still be misleading when a model error favors one design disproportionately.

WORKED EXAMPLE

If a simulated memory latency is uncertain, sweep a plausible declared range. A proposed design that wins only at one finely chosen value has a weaker claim than one that remains useful across the range.

PRACTICE + PASS CRITERION

Run a cache, scheduling or accelerator study with at least one validation case and a sensitivity sweep. Pass: raw configurations reproduce results, modeled versus measured quantities are labeled and conclusions do not exceed the tested fidelity.

ASSIGNED STUDY

R38: simulation models, statistics and configuration; R48: architecture evaluation; R43: analytical bounds.

CHECK YOUR UNDERSTANDING

1. Does cycle-oriented simulation guarantee real-hardware accuracy?

2. Why perform sensitivity analysis?

3. What can a trace-driven model omit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 46.4 • MODULE 46

Memory systems, chiplets and multi-objective design

Learning goal Choose an architecture across performance, power, cost and complexity.

Memory bandwidth, latency, capacity and energy are distinct design dimensions. More channels or wider interfaces can improve bandwidth while increasing pins, routing, power and controller complexity. Cache size changes miss behavior but also area and access characteristics. Packaging and chiplet approaches introduce interconnect and partitioning tradeoffs; no packaging label guarantees a superior system.

A Pareto frontier contains designs not dominated on every chosen objective. Define constraints and compare alternatives at consistent workload quality and manufacturing assumptions. Cost and yield models require credible inputs; educational estimates should not be presented as production quotations. Include verification and software complexity in the design discussion because a theoretically efficient architecture may be impractical to support.

WORKED EXAMPLE

Design A takes 10 ms at 40 J per task; B takes 8 ms at 50 J. Neither dominates the other on latency and energy. A third design at 12 ms and 60 J is dominated by both under those two metrics.

PRACTICE + PASS CRITERION

Construct a Pareto comparison of at least three architecture configurations. Pass: objectives and assumptions are explicit, dominated designs are identified correctly and a recommendation changes appropriately when a hard constraint changes.

ASSIGNED STUDY

R48: memory and quantitative design; R49: packaging and system tradeoffs; R38: architecture alternatives.

CHECK YOUR UNDERSTANDING

1. Does A dominate B in the example?

2. Why is the 12 ms, 60 J design dominated?

3. Why not present an educational chip-cost model as a quote?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 47

VLSI, timing and physical implementation

Prerequisites Modules 31-34, 40 and 46. Use educational libraries and licensed or open tools only.

Four lessons

47.1 CMOS logic, transistor sizing and delay models

Connect gate-level choices to electrical load and timing.

47.2 Static timing analysis, clocks and timing exceptions

Check every required path under the correct timing contract.

47.3 Synthesis, placement, clock trees, routing and physical checks

Understand how a logical netlist becomes a physical layout.

47.4 Power, area, process corners and design-space closure

Report PPA with enough assumptions to make comparison meaningful.

Module project

Carry a verified small RTL block through an educational synthesis and physical-design flow. Report constraints, timing, area and power-estimation assumptions. The result is a learning artifact, not fabrication-ready commercial signoff.

Core references

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R39 OpenROAD project — OpenROAD documentation

R30 MIT OpenCourseWare — 6.002 Circuits and Electronics

R33 YosysHQ — Yosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 47.1 • MODULE 47

CMOS logic, transistor sizing and delay models

Learning goal Connect gate-level choices to electrical load and timing.

CMOS gates charge and discharge capacitances through transistor networks. Propagation delay depends on drive strength, load, input transition, topology and process/voltage/temperature conditions. Increasing device size can reduce effective resistance while increasing input capacitance and area, moving cost to the previous stage.

Logical-effort-style models separate gate topology, electrical effort and parasitic delay to reason about sizing and staging. These are approximations, not substitutes for characterized library timing. Series transistor stacks and fanout affect delay differently. Dynamic switching, leakage and short-circuit effects contribute to power. Study an inverter chain and a small logic path before optimizing a large synthesized block.

WORKED EXAMPLE

In the simplified normalized model d=g*h+p, a stage with g=1, h=4 and p=1 has delay 5 units. Those normalized units require a technology-dependent scale before becoming nanoseconds.

PRACTICE + PASS CRITERION

Analyze and simulate an inverter chain or compare library-cell drive strengths under several loads. Pass: assumptions are stated, input loading is included and model predictions are compared with available characterized or simulated results.

ASSIGNED STUDY

R49: CMOS devices, combinational circuits and delay; R30: MOS circuit foundations.

CHECK YOUR UNDERSTANDING

1. What is d for g=1, h=4, p=1?

2. Why can enlarging one gate slow an earlier stage?

3. Can normalized logical-effort delay be reported directly as nanoseconds?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 47.2 • MODULE 47

Static timing analysis, clocks and timing exceptions

Learning goal Check every required path under the correct timing contract.

Static timing analysis evaluates path delays without enumerating every input waveform. Setup checks constrain maximum data arrival before capture; hold checks constrain minimum delay around the capture edge. Clock-to-output, combinational delay, setup/hold requirements, skew and uncertainty all participate. Different corners can dominate different checks.

Clocks, generated clocks, input/output delays and exceptions define what the tool analyzes. An unconstrained path is not a fast path; it may be an unexamined path. False-path and multicycle exceptions must follow real functional guarantees, not serve as a way to hide violations. Review exception reachability and clock-domain boundaries. Functional correctness and timing correctness require separate evidence.

WORKED EXAMPLE

With zero skew and uncertainty, tCQ_max=0.10 ns, logic_max=0.50 ns and setup=0.05 ns require T>=0.65 ns. Real signoff adds the relevant clock, interconnect and corner details.

PRACTICE + PASS CRITERION

Constrain a small sequential block and inspect worst setup and hold paths. Pass: no unexplained unconstrained endpoints, every exception is justified and a deliberately tightened clock produces an understood violation.

ASSIGNED STUDY

R39: timing analysis and flow constraints; R49: sequential timing and clocking.

CHECK YOUR UNDERSTANDING

1. What minimum simplified period follows from the example?

2. Does marking a path false make a real timing problem disappear physically?

3. Why examine both maximum and minimum delays?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 47.3 • MODULE 47

Synthesis, placement, clock trees, routing and physical checks

Learning goal Understand how a logical netlist becomes a physical layout.

The implementation flow maps logic, places cells, constructs clock distribution and routes interconnect under design constraints. Placement changes wire length and congestion; routing adds parasitic resistance and capacitance that affect timing and power. Clock-tree design must distribute a usable clock while controlling skew and other requirements.

Physical checks include geometry rules and connectivity consistency between layout and the intended circuit. Passing one tool stage does not establish all manufacturing, electrical or reliability requirements. Educational process kits and libraries have specific scope and licensing. Preserve exact versions and distinguish pre-route estimates from extracted post-route results. A failed route or timing closure attempt is useful evidence about congestion or unrealistic constraints.

WORKED EXAMPLE

A netlist can meet an idealized pre-route timing estimate and fail after long interconnect is included. The logical cell count alone does not reveal that physical bottleneck.

PRACTICE + PASS CRITERION

Run a small block through an educational OpenROAD-compatible flow or inspect a provided open example. Pass: capture stage reports, identify a routing/timing change and state which physical checks were and were not performed.

ASSIGNED STUDY

R39: flow stages, placement, clock tree and routing; R49: physical-design methodology and EDA.

CHECK YOUR UNDERSTANDING

1. Why can timing worsen after routing?

2. Does synthesis success prove a manufacturable layout exists?

3. Why preserve process-kit and library versions?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 47.4 • MODULE 47

Power, area, process corners and design-space closure

Learning goal Report PPA with enough assumptions to make comparison meaningful.

Power-performance-area evaluation depends on workload activity, clocking, voltage, cell library and physical implementation. A common simplified dynamic-power model is P=alpha*C*V^2*f, where alpha represents switching activity under a stated convention. Leakage and other contributions are additional. A power estimate without activity assumptions can be badly misleading.

Process, voltage and temperature corners model variation but do not cover every reliability or manufacturing issue. Compare designs under consistent constraints and report whether timing actually closes. Clock gating and operand isolation can reduce unnecessary switching but require functional and timing verification. An educational area reduction is meaningful only alongside performance, power and correctness effects. Keep fabrication and production signoff outside the claim.

WORKED EXAMPLE

Under the simplified dynamic model with unchanged alpha, C and f, reducing voltage to 0.9 of its original value scales dynamic power to 0.9^2=0.81. In practice the achievable frequency and margins may also change.

PRACTICE + PASS CRITERION

Compare two verified RTL implementations using identical flow assumptions. Pass: report area, worst timing and activity-based power estimates where supported, with uncertainty and no unsupported commercial-silicon claim.

ASSIGNED STUDY

R49: power, timing and implementation tradeoffs; R39: reports and analysis; R30: energy and electrical models.

CHECK YOUR UNDERSTANDING

1. What dynamic-power factor follows from 0.9x voltage under fixed other factors?

2. Why is an activity assumption necessary for power estimation?

3. Why can lower voltage invalidate a timing target?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 48

Verification, reliability and hardware trust boundaries

Prerequisites Modules 33-35, 46-47.

Four lessons

48.1 Formal properties, abstraction and assume-guarantee reasoning

Prove useful properties without hiding failures in assumptions.

48.2 Clock/reset crossings and asynchronous FIFO verification

Check digital protocol correctness while acknowledging analog risk.

48.3 Fault models, ECC, testability and recovery

Design for detectable failures rather than assuming perfect hardware.

48.4 Hardware security, thermal limits and assurance review

Include isolation and failure containment in the design contract.

Module project

Create a verification dossier for the hardware capstone: requirements-to-tests traceability, assertions, coverage, clock/reset review, fault injection and a clearly bounded assurance claim.

Core references

R50 YosysHQ — SymbiYosys documentation

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R39 OpenROAD project — OpenROAD documentation

R51 Linux kernel contributors — Driver implementer API guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 48.1 • MODULE 48

Formal properties, abstraction and assume-guarantee reasoning

Learning goal Prove useful properties without hiding failures in assumptions.

Formal verification reasons about a mathematical model and properties under assumptions. Decompose a large system into contracts: a block assumes valid inputs and guarantees specified outputs or progress. The system review must show that connected blocks actually satisfy each other's assumptions. Circular or inconsistent contracts can make local proofs meaningless.

Bounded model checking, induction and equivalence checking answer different questions. A timeout is not a proof or a counterexample. Vacuity checks and cover properties help reveal environments that never exercise the intended behavior. Abstraction can make proof tractable but must conservatively preserve the property of interest. Track proof scripts, tool versions and unresolved properties as part of the artifact.

WORKED EXAMPLE

An arbiter property stating that every request is eventually granted becomes vacuous if the environment assumes requests are always zero. A cover goal requiring contention exposes that missing behavior.

PRACTICE + PASS CRITERION

Prove a queue, arbiter or transaction property and inject a violation. Pass: assumptions are reviewed independently, contention is covered and bounded, proven, failed and unknown results are reported separately.

ASSIGNED STUDY

R50: safety, liveness, induction, cover and assumptions; R49: verification methodology.

CHECK YOUR UNDERSTANDING

1. Is a solver timeout evidence that the property is true?

2. Why can the no-request assumption make an arbiter proof vacuous?

3. What must assume-guarantee integration establish?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 48.2 • MODULE 48

Clock/reset crossings and asynchronous FIFO verification

Learning goal Check digital protocol correctness while acknowledging analog risk.

Clock-domain crossings require both a sound transfer protocol and an appropriate physical synchronizer implementation. Asynchronous FIFOs often use local binary pointers with Gray-coded values transferred between domains, because adjacent Gray values change one bit. Correct full/empty detection, pointer widths, reset behavior and synchronization delay still need verification.

Gray coding is not a substitute for routing/skew constraints or correct sampling assumptions. Multi-bit coherence and pulse capture need distinct reasoning. Reset deassertion must be coordinated with each domain's requirements, and resetting one side of a live link needs a recovery policy. Digital formal analysis proves the modeled protocol; analog metastability probability needs device and timing information beyond that model.

WORKED EXAMPLE

A one-bit source pulse can occur entirely between two destination clock edges and be missed. A level handshake or suitable event-transfer scheme is needed when every event must be observed.

PRACTICE + PASS CRITERION

Model an asynchronous FIFO or event handshake with independent clocks and reset sequences. Pass: no loss/duplication under the stated protocol, full/empty behavior is checked and physical synchronizer assumptions are listed separately.

ASSIGNED STUDY

R50: multiclock modeling and verification; R49: sequential circuits and synchronization; revisit Module 32 timing boundaries.

CHECK YOUR UNDERSTANDING

1. Why is Gray code useful for pointer transfer?

2. Can an unsynchronized one-cycle pulse be missed by another domain?

3. Does a digital FIFO proof quantify real metastability failure rate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 48.3 • MODULE 48

Fault models, ECC, testability and recovery

Learning goal Design for detectable failures rather than assuming perfect hardware.

A fault model specifies the failures considered, such as stuck-at logic, transient bit flips or communication corruption. Detection and correction capability depend on the code or mechanism and the fault pattern. Parity detects an odd number of flipped bits but does not locate and correct an arbitrary single error by itself. Error-correcting codes add structured redundancy with explicit guarantees.

Design-for-test techniques improve controllability and observability of internal state, with area, timing and security implications. Built-in self-test and scan are not interchangeable with complete functional verification. A detected error needs policy: retry, isolate, reset, degrade or fail safely. Fault injection tests the policy as well as detection. Report unmodeled common-mode or multi-bit failures instead of assuming independence without evidence.

WORKED EXAMPLE

With one parity bit over a word, a single flipped data bit changes parity and is detectable. Two flipped bits can leave parity unchanged, so the same scheme cannot guarantee detection of every double-bit error.

PRACTICE + PASS CRITERION

Implement a parity and a small single-error-correcting code model, then inject faults. Pass: enumerate the supported correction/detection claims, test multi-bit counterexamples and define a recovery response.

ASSIGNED STUDY

R49: testing, DFT and reliability context; R51: error detection/correction reporting concepts; R06: coding and combinatorial reasoning extension.

CHECK YOUR UNDERSTANDING

1. Can simple parity detect every two-bit error?

2. Why specify a fault model before claiming reliability?

3. What is missing when a system detects an error but has no recovery policy?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 48.4 • MODULE 48

Hardware security, thermal limits and assurance review

Learning goal Include isolation and failure containment in the design contract.

Hardware interfaces control powerful resources: memory access, privilege, interrupts and firmware execution. Validate descriptors, restrict DMA and separate trusted control from untrusted data. Boot authenticity, update integrity and rollback policy require established cryptographic mechanisms and careful key handling, not a homemade cipher. Debug interfaces need a lifecycle access policy.

Shared caches, timing and power behavior can expose information even when architectural results are isolated. Study threat models and defensive mitigations without attacking systems outside an authorized lab. Thermal and electrical protections remain active during experiments. An assurance review connects threats and hazards to requirements, mechanisms, tests and residual risks. Passing a educational checklist is not security certification.

WORKED EXAMPLE

A device accepting an unchecked memory address in a work descriptor can cross the intended buffer boundary. A protected mapping and validated length constrain that authority even when the computation itself is correct.

PRACTICE + PASS CRITERION

Write a threat-and-failure review for the accelerator and test malformed descriptors, denied access and reset under load in simulation. Pass: isolation limits, protection assumptions and unresolved risks are explicit; no unauthorized target testing occurs.

ASSIGNED STUDY

R51: device access, DMA and isolation; R49: implementation and verification; R50: enforceable protocol properties.

CHECK YOUR UNDERSTANDING

1. Why validate both address and length in a work descriptor?

2. Can correct architectural output coexist with a side-channel risk?

3. Does this review constitute a security certification?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 49

Master's thesis: proposal, implementation and evaluation

Prerequisites Gate I and relevant advanced-track modules; independent technical mentorship strongly recommended.

Four lessons

49.1 Question selection, literature map and thesis scope

Turn a broad interest into a falsifiable, feasible research question.

49.2 Proposal review, baselines and risk-reduction milestones

Make failure cheap before the expensive implementation phase.

49.3 Implementation, ablations and controlled evaluation

Execute the study without changing the question to fit a favorable result.

49.4 Thesis writing, reproducibility and contribution boundaries

Write a technical argument another engineer can audit.

Module project

Prepare and execute a thesis-scale project in one primary specialization. Budget 400-800 focused hours as a planning range, with scope adjusted through review. The deliverable is a defensible contribution and reproducible artifact, not a promise of publication.

Core references

R42 Stephen Boyd and Lieven Vandenberghe — Convex Optimization

R44 Benedikt Bitterli and coauthors — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R55 Matthias Mueller and collaborators — Author publication archive: position-based simulation and XPBD

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 49.1 • MODULE 49

Question selection, literature map and thesis scope

Learning goal Turn a broad interest into a falsifiable, feasible research question.

A thesis question identifies a problem, intervention or method, evaluation setting and meaningful outcome. Narrowness improves rigor: one well-tested engine subsystem or hardware tradeoff is preferable to a promise to build an entire commercial console. Distinguish an engineering integration contribution, a new method, a replication study and a negative result.

Build a literature map with seminal work, strong baselines and relevant newer work reviewed when the thesis begins. The dated anchor papers in this curriculum are not a complete current literature review. Identify the gap and explain why it matters. Check data, hardware, tool access, licenses and mentorship before committing. Write explicit exclusions so the project remains feasible.

WORKED EXAMPLE

Instead of "make a faster engine," ask whether a bounded task-scheduling policy reduces p99 frame time under specified streaming bursts without exceeding a declared memory budget, compared with two competent baselines.

PRACTICE + PASS CRITERION

Write a 3-5 page proposal with a research question, 8-15 relevant sources, baseline plan, constraints and exclusions. Pass: a reviewer can explain what observation would weaken or refute the main hypothesis.

ASSIGNED STUDY

Use Module 41's paper method and the chosen track's original papers; R44, R48, R49 or R55 are starting anchors, not a substitute for the literature search.

CHECK YOUR UNDERSTANDING

1. What makes a research question falsifiable?

2. Why state explicit exclusions?

3. Are the curriculum's historical anchor papers a complete current literature review?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 49.2 • MODULE 49

Proposal review, baselines and risk-reduction milestones

Learning goal Make failure cheap before the expensive implementation phase.

A credible proposal includes a baseline that already runs, an evaluation protocol and a risk register. Technical risks include insufficient performance, unavailable data, unsupported tool features and an inability to measure the claimed effect. Resolve the highest-impact uncertainties with small prototypes before building a large framework.

Milestones should produce evidence: a reference result, a validated instrument, a working minimal method, an ablation and a reproducible final comparison. A calendar date alone is not a technical milestone. Define stop or pivot criteria in advance. Independent review can reveal an unfair baseline or an untestable objective. Keep project decisions in a versioned log so later changes are transparent.

WORKED EXAMPLE

Before implementing a new GPU scheduler, confirm that the target platform exposes enough timing information to distinguish scheduling delay from execution time. Otherwise the proposed mechanism may be impossible to evaluate directly.

PRACTICE + PASS CRITERION

Deliver a running baseline, risk register and milestone plan. Pass: the primary metric can be measured, a small feasibility experiment exists and each major risk has a mitigation or explicit acceptance decision.

ASSIGNED STUDY

R49: project milestone and reporting structure; Module 41: experimental design; chosen method's primary documentation.

CHECK YOUR UNDERSTANDING

1. Why require a running baseline early?

2. What makes a milestone evidence-based?

3. Why define pivot criteria before final results?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 49.3 • MODULE 49

Implementation, ablations and controlled evaluation

Learning goal Execute the study without changing the question to fit a favorable result.

Implement the smallest method that tests the research question. Keep a correct reference path and add tests before optimizing. Separate development experiments from final evaluation, and record changes to the protocol with reasons. A method that fails on some supported inputs must report those failures rather than silently removing the cases.

Ablations test the contribution of individual components. Sensitivity sweeps examine parameter dependence; held-out workloads examine generalization. Statistical analysis should match the trial structure and metric. Preserve raw data, seeds, binaries or build manifests and analysis scripts. When results are negative, investigate correctness, measurement and assumptions, then report the supported conclusion. Honest negative evidence can be a substantial contribution.

WORKED EXAMPLE

If a new allocator reduces average time but worsens p99 frame time beyond the stated requirement, the original latency hypothesis is not supported even though one secondary number improved.

PRACTICE + PASS CRITERION

Run the approved evaluation with baselines, at least one ablation and one sensitivity study. Pass: scripts regenerate results, failed cases remain visible and claims use the preregistered primary metric or clearly disclose deviations.

ASSIGNED STUDY

R09 and Module 41: uncertainty; chosen track's original methods and implementation references; handbook experiment manifest.

CHECK YOUR UNDERSTANDING

1. Does improving a secondary metric prove the primary hypothesis?

2. Why retain failed supported cases?

3. What should happen after a negative result?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 49.4 • MODULE 49

Thesis writing, reproducibility and contribution boundaries

Learning goal Write a technical argument another engineer can audit.

A thesis report connects motivation, prior work, method, implementation, experiments, results and limitations. Each major claim should point to a derivation, test, table or figure. Separate observations from interpretations and specify uncertainty. Explain enough detail for a competent reader to reconstruct the central method without depending on undocumented conversations.

Include an artifact guide with environment, commands, expected outputs and resource needs. Record third-party code, assets, datasets and AI assistance with their relevant licenses and provenance. Do not include confidential material without authorization. The contribution statement should identify what is new, what is reproduced and what remains unverified. A polished document cannot compensate for missing experiments or an irreproducible implementation.

WORKED EXAMPLE

A report saying "the method is 30% faster" should specify the baseline, workload, metric, aggregation, hardware, quality constraint and uncertainty. Otherwise the number lacks an interpretable claim boundary.

PRACTICE + PASS CRITERION

Produce a thesis-style report, typically 30-60 pages plus appendices as a planning guide, and a complete artifact package. Pass: a reviewer traces five major claims to evidence and reproduces the central table or experiment.

ASSIGNED STUDY

R49: final project reports; chosen original papers for reporting comparisons; handbook thesis outline and evidence checklist.

CHECK YOUR UNDERSTANDING

1. What must accompany a speedup claim?

2. Why distinguish observation from interpretation?

3. What belongs in the contribution statement?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 50

Thesis defense, professional practice and continued development

Prerequisites Module 49 and all prerequisites for the selected specialization.

Four lessons

50.1 Oral defense and independent artifact reproduction

Show that the learner owns the reasoning and implementation.

50.2 Mature-codebase integration, review and maintainable contributions

Transfer a prototype into an engineering environment with other owners.

50.3 Portfolio, technical interviews and role-specific evidence

Present concrete engineering competence without inflated claims.

50.4 Long-term learning, maintenance and final handoff

Keep the knowledge and artifacts useful after the course ends.

Module project

Gate J: defend the thesis, pass an independent artifact reproduction, complete a maintainable handoff and publish an honest portfolio where rights permit. This completes the self-study program, not an accredited degree.

Core references

R01 ACM / IEEE-CS / AAAI — CS2023 knowledge areas

R02 ACM / IEEE Computer Society — Computer Engineering Curricula 2016

R05 Scott Chacon and Ben Straub — Pro Git

R11 C++ Core Guidelines editors — C++ Core Guidelines

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 50.1 • MODULE 50

Oral defense and independent artifact reproduction

Learning goal Show that the learner owns the reasoning and implementation.

An oral defense tests the relationship between the research question, assumptions, implementation and conclusions. Begin with the central claim and evidence, then invite scrutiny of the weakest assumptions. Be prepared to derive a key equation, explain a failed experiment and modify a small part of the implementation live. Saying that a tool generated the code is not an explanation of its correctness.

An independent reproduction uses the supplied artifact instructions without hidden files or undocumented manual repairs. Record what the reviewer could reproduce and what failed. A defense can pass with limitations when the claims are narrowed honestly; it should not pass on presentation alone while the central evidence remains unavailable. Revisions should respond directly to review findings.

WORKED EXAMPLE

A reviewer changes a supported workload parameter. The learner should predict which resource or error term may change, run the test and explain agreement or disagreement with that prediction.

PRACTICE + PASS CRITERION

Hold a 30-45 minute technical defense with a qualified reviewer or peer panel and a separate artifact run. Pass: the central result is reproduced, major questions receive evidence-based answers and required revisions are completed.

ASSIGNED STUDY

R49: design-project evaluation as a reference; handbook defense questions and rubric.

CHECK YOUR UNDERSTANDING

1. What does a live modification test beyond a prepared presentation?

2. Must every thesis result be positive to pass?

3. Why separate independent reproduction from the author's demo?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 50.2 • MODULE 50

Mature-codebase integration, review and maintainable contributions

Learning goal Transfer a prototype into an engineering environment with other owners.

A research prototype and a maintained engine component face different requirements. Integration needs style consistency, tests, platform support, migration behavior, documentation and ownership. Study an existing codebase through a narrow feature path: entry point, data flow, invariants, tests and build configuration. Avoid rewriting a subsystem before understanding why it exists.

A good contribution starts with a focused issue and a reproducible case. Discuss design where project norms require it, make a small patch and respond constructively to review. Open-source maintainers are not obligated to accept educational work. A local integration exercise can satisfy this lesson when no appropriate public contribution is available. Respect licenses and never upload employer or proprietary SDK code without authorization.

WORKED EXAMPLE

A faster container may break serialization ordering or iterator expectations elsewhere in the engine. Integration review must examine those contracts before accepting the local benchmark improvement.

PRACTICE + PASS CRITERION

Integrate one tested improvement into a mature open codebase or a maintained local fork. Pass: a focused patch, regression test, design rationale and reviewer response are included, whether or not an upstream project accepts it.

ASSIGNED STUDY

R05: distributed workflows and contributing; R11: interfaces and maintenance; R26: mature-engine comparison where authorized.

CHECK YOUR UNDERSTANDING

1. Why can a locally faster component be an integration regression?

2. Is upstream acceptance required to prove learning?

3. Why avoid a large rewrite as the first contribution?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 50.3 • MODULE 50

Portfolio, technical interviews and role-specific evidence

Learning goal Present concrete engineering competence without inflated claims.

A portfolio should show a small number of complete artifacts with clear ownership, build instructions, tests, measured results and limitations. Tailor emphasis: engine roles need runtime architecture and debugging; rendering roles need graphics mathematics and GPU evidence; platform roles need systems and latency; hardware roles need RTL, verification and timing or board evidence.

Technical interviews often probe fundamentals through unfamiliar problems, so practice explanation, assumptions, edge cases and tests rather than memorizing only known answers. Revisit algorithms, memory, concurrency and mathematics using transfer tasks. Do not label this self-study program an accredited degree or imply professional chip signoff from educational tools. Honest scope and a reproducible artifact are stronger than a list of fashionable technologies.

WORKED EXAMPLE

A portfolio entry can state: "Implemented a verified FIFO and compared two storage mappings under one pinned synthesis flow; physical silicon and production signoff were not performed." That is precise and defensible.

PRACTICE + PASS CRITERION

Prepare three portfolio case studies and complete one systems, one algorithm and one specialization mock interview. Pass: every claimed result is supported by an artifact and assistance or third-party contributions are clearly attributed.

ASSIGNED STUDY

R01-R02: breadth review; R11: engineering explanation and contracts; handbook role map and portfolio template.

CHECK YOUR UNDERSTANDING

1. What makes a portfolio claim defensible?

2. Should completing this program be described as earning an accredited master's degree?

3. Why practice unfamiliar transfer tasks?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 50.4 • MODULE 50

Long-term learning, maintenance and final handoff

Learning goal Keep the knowledge and artifacts useful after the course ends.

Engineering knowledge includes durable concepts and changing interfaces. Preserve principles such as invariants, measurement and ownership while periodically checking toolchains, specifications and platform documentation. A dependency update should run tests and benchmarks rather than assume compatibility from a version number. Maintain an evidence log of what was actually revalidated.

Create a handoff document describing architecture, build, tests, known risks and next decisions. Archive raw research data and source in accessible formats with appropriate backups. Choose a next specialization from demonstrated interest and gaps, not pressure to master every field simultaneously. The final review should identify strengths and unresolved areas honestly. Continuing with mentorship, collaboration and real projects is part of professional development, not a failure of the curriculum.

WORKED EXAMPLE

Updating a graphics SDK can change tutorial requirements or validation behavior. A pinned working environment preserves reproducibility while a separate upgrade branch tests the new toolchain.

PRACTICE + PASS CRITERION

Complete Gate J: archive the thesis artifact, produce a maintainer handoff and write a 90-day learning plan with one primary gap and one real contribution goal. Pass: another person can locate, build and understand the project without the author's machine.

ASSIGNED STUDY

R05: maintenance workflows; R01-R02: periodic breadth review; current official documentation for the selected tools when an update is undertaken.

CHECK YOUR UNDERSTANDING

1. Why keep a pinned environment while testing an upgrade separately?

2. What belongs in a maintainer handoff?

3. Why choose one primary next specialization?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.